AI Research on Deep Convolutional Neural Network Designs

Table of Contents

ChatGPT

1D CNN Architecture for Variable-Length Sequential Data in RL

Overview: Recent research has shown that **1D convolutional networks** can model sequential data as effectively as (or even better than) recurrent networks in many domains [1]. This makes them a promising choice for reinforcement learning (RL) policies that must handle **variable-length sequences** of instructions. Below we describe a **proven Conv1D architecture** (inspired by the Temporal Convolutional Network, TCN) and how to adapt it to an actor-critic (A2C/PPO-style) model for your quantum circuit input. This architecture is *fully convolutional* along the sequence dimension (no RNNs), and is designed to handle arbitrary sequence lengths end-to-end [2].

Temporal Convolutional Network (TCN) – Robust 1D CNN for Sequences

A **Temporal Convolutional Network (TCN)** is a deep 1D CNN architecture that has been empirically shown to outperform RNNs on various sequence modeling tasks [1]. It's well-suited to sequences of *variable length with repeating patterns* (like a series of quantum gate instructions). Key features of a TCN (or similar Conv1D sequence models) include:

- **Stacked 1D Convolutions with Dilation:** The network stacks multiple Conv1D layers that slide over the sequence (time) dimension. Using *dilated* convolutions allows the receptive field to grow exponentially with depth [3]. For example, each layer can have a small kernel (e.g. 3) but with dilation factors doubling at each layer (1, 2, 4, 8, …). This way, a relatively **deep network (e.g. 8–12 layers)** can capture long-range dependencies (hundreds of time-steps) without needing an impractically large kernel [4]. In other words, dilation lets the model **cover very long sequences** while keeping kernels small and efficient.

- **Residual Blocks for Stability:** Rather than plain sequential conv layers, TCNs use **residual blocks** (skip connections) to ease training of deep networks [4]. Each residual block typically contains **two Conv1D layers** (with dilation) plus non-linearities, and the block's input is added to its output (with a 1×1 conv on the input if needed to match dimensions) [5]. This design lets layers **learn modifications to the identity mapping** instead of entire transformations, which greatly stabilizes training for deep CNNs [6] [7]. In fact, experiments found that adding residual connections yields faster convergence and better final performance on sequence tasks [7]. *Recommendation:* Wrap your Conv1d layers in residual blocks if you stack many of them, especially if you increase channel counts in deeper layers.

- **Normalization and Dropout:** Successful Conv1D architectures often include normalization and regularization in each block. The TCN reference model applied **weight normalization** to conv filters and **spatial dropout** (dropout entire channels) after conv layers [8]. In practice, using **LayerNorm + dropout** (or BatchNorm) after conv layers is also effective to stabilize training – you already use

LayerNorm and PReLU, which is consistent with these best practices. These help deal with the high-dimensional input (e.g. 150 features per instruction) and prevent overfitting as the network grows deep.

- **Causal vs. Non-causal:** In some domains (like forecasting), conv layers are made *causal* (no future leakage). For your use-case (analyzing a circuit as a whole to output an action), **causality isn't required** – you can use "same" padding so the conv layers consider the full neighborhood around each instruction. The important part is that **input and output lengths remain the same** through the conv stack (achieved via padding) [9], so that every instruction's effects can propagate to the final representation.

Why this architecture fits: A TCN-style ConvNet naturally handles **arbitrary-length sequences** by sliding the filters over the instruction sequence [2], just as you described (each "cell" of the kernel covers one instruction's features). It can learn repeating local patterns (via small kernel convolutions) and also long-term patterns (via stacked dilations). In fact, one major advantage of Conv1D over RNNs is the **flexible receptive field** – you can increase it by adding layers or larger dilation factors as needed [10]. This means the network's "memory" of past instructions can be tuned to the task's needs more easily than an RNN's fixed state size. And unlike RNNs, a Conv1D processes the whole sequence in parallel, avoiding sequential bottlenecks and vanishing gradients through time [11] [12].

Proven performance: Conv1D architectures with these elements (dilations, residuals, gating or ReLU) have matched or surpassed recurrent nets in **audio generation, language modeling, and translation** tasks [1]. For example, DeepMind's WaveNet (audio) and Facebook's ConvS2S (translation) are fully convolutional sequence models that achieved state-of-art results using ~10–20 layers of 1D conv with residual/gated activations. The TCN formulation (Bai et al. 2018) distilled many of these best practices into a simple architecture that worked across diverse tasks [13] [14]. All this gives confidence that a well-designed 1D CNN can **learn complex sequence patterns** (even something as unusual as quantum gate sequences) given sufficient depth and training.

Designing the Conv1D Network: Depth, Kernel Size, Channels

When building your 1D CNN policy network, consider the following design choices informed by the above architectures:

- **Kernel Size vs. Depth:** Instead of using an extremely large kernel (e.g. 130) in one layer, it's usually better to use a **small kernel (3–5)** and achieve a large receptive field by stacking layers (with dilation). Research shows that **small kernels focus on local patterns** (which is often important for "language-like" sequences), while stacking/dilating layers captures global context [15]. Larger kernels can help very long-memory tasks converge a bit faster, but they didn't always improve final accuracy in language modeling [15]. A reasonable approach is to start with kernel size 3 or 5 for all conv layers. With dilation doubling each layer, the receptive field grows exponentially – for example, 6 layers with kernel=3 gives a field of 127 time-steps, covering most circuits in your case. This way each layer learns **n-gram patterns of instructions**, and deeper layers combine them into longer motifs. (If you truly expect patterns spanning >100 instructions, you can increase depth or kernel slightly. But avoid one giant filter covering the whole sequence, as it may over-smooth or ignore the sequential structure.)

- **Number of Layers:** Use enough layers (with dilation) such that the network's receptive field ≥ the maximum sequence length you need to consider. Bai et al. note that **up to ~12 layers** might be needed if the model should potentially look across an entire long sequence [4]. In practice, you might start with ~6–8 layers and adjust. Each residual block effectively counts as **one layer** in terms of receptive field growth (since both convs in the block typically use the same dilation before it increases in the next block). Ensure dilation resets or increases properly per layer. For example: layers 1–6 with dilation 1,2,4,8,16,32 (then you have receptive field *~2^6*). If the sequence can be extremely long, you could either add more layers or use a second pass of dilation (some implementations cycle dilation after a certain point).

- **Channels (Filters) per Layer:** It's common to start with a moderate number of filters and **increase the channel count in deeper layers** (to capture higher-level features). For example, one successful design used 32 filters in the first conv layer, 64 in the second [16], and you could continue to 128, etc., as depth increases. This is similar to how image CNNs increase channels after each block. More channels = more feature maps to represent different instruction patterns. That said, increasing channels will increase model size, so find a balance. You might try a progression like 64 → 64 → 128 → 128 (depending on your compute budget). The **residual 1×1 conv** in each block will ensure the skip connection matches the new width [5] when you increase channels.

- **Activation Functions:** ReLU is standard (TCN used ReLU [8]), but you are using **PReLU**, which is fine (it can help with negative values and maybe learns a slight bias in activation). The exact choice (ReLU, LeakyReLU, PReLU) is not critical as long as it's applied after each conv. Just ensure it's **within the residual block** (i.e., Conv -> Activation -> Norm -> Conv -> Activation -> Norm, then add skip). Gated activations (e.g. GLU or gated tanh as in WaveNet/GLU) are another option. Some NLP conv models used **GLUs** which multiply a tanh and sigmoid branch for each conv output [17]. Gating can improve modeling of certain data (TCN authors found GLU helped on word language modeling but not on other tasks) [18]. If you're feeling experimental, you could try replacing ReLU with a gated unit in the conv blocks. However, **starting with a simple ReLU/PReLU is reasonable** – the **TCN without gating performed robustly across many tasks** [18].

- **Normalization & Regularization:** Continue using **Layer Normalization** after each conv (or each block) as you have – it helps with training stability, especially given the very different scales (gate indicators vs. continuous parameters) in your 150 features. Dropout is also important for RL to prevent overfitting to early trajectories. TCN used **Spatial Dropout** (drop entire channels) [5], which is effective for conv nets on sequences. In PyTorch, you can use `nn.Dropout1d` on the conv outputs to drop whole feature maps. This forces the network to not rely on any single feature channel too much. A dropout rate around 0.1–0.3 is a typical starting point. *Tip:* Place dropout **within the residual block** (e.g. after the first or second conv before adding the residual back). This was done in the original TCN design [8].

- **Pooling/Downsampling:** Many sequence CNNs (like TCN) do **not** downsample the time dimension at all – they keep output length equal to input length by padding ("same" convolution) [9]. This is useful if you need a prediction at every time-step. In your case, you ultimately need a single output (policy & value) for the whole sequence, not at every step. There are two ways to handle this:

- **Global Pooling:** After the final conv layer, apply a **Global Average Pooling (1D)** across the sequence length, converting an output of shape *(seq_len, channels)* into a single *1×channels* vector. This yields a

fixed-length representation regardless of sequence length [19] [20]. Global Max Pooling is another option (taking the strongest feature activation over time). Global pooling is simple and makes the network truly length-agnostic – it's often used in sequence classification CNNs to aggregate features [19].

- **Flatten with Masking/Padding:** Alternatively, you could pad all circuits to a max length and flatten the conv output, then use a fully-connected layer. However, this is less elegant and can introduce a dependence on the max length. Given you prefer an end-to-end conv solution, **Global Average Pooling is recommended** to summarize the sequence features [19]. (It effectively acts like an adaptive stride that reduces the sequence dimension to 1.)

Both approaches end up with a **fixed-size feature vector** that represents the entire circuit. The rest of the network (policy/value heads) can then be standard fully-connected layers.

Integration into an Actor-Critic (A2C/PPO) Agent

To use this Conv1D architecture in an RL agent, you will treat it as a **feature extractor** that feeds into policy and value outputs:

- **Shared Conv Trunk:** In on-policy algorithms like A2C/PPO, it's common to **share the feature extractor** between the policy and value networks [21]. That is, you apply the Conv1D stack (with all the layers discussed above) to the input sequence *once*, get the final pooled feature vector, and then have **two output heads** on top of that: one for the action probability distribution $\pi(a|s)$, one for the value $V(s)$. Sharing the conv backbone saves computation and tends to work well unless the value function needs a very different representation. (Off-policy methods like SAC sometimes use separate encoders [21], but for simplicity you can start with a shared encoder.)

- **Policy Head:** Take the pooled feature vector and pass it through a fully-connected (dense) layer to produce a logit for each action. For example, a single `Linear(feat_dim, N_actions)` can output unnormalized scores for each of the N possible actions, which then go through a softmax to yield the policy distribution π. If you want a bit more capacity, you could insert one hidden layer (e.g. Linear to 128 → ReLU → Linear to N_actions), but often a direct projection is sufficient when the conv already did heavy lifting.

- **Value Head:** Similarly, take the same feature vector and feed it to a linear layer (or small MLP) to output a single scalar value V. For instance `Linear(feat_dim, 1)` gives the state-value. In an Actor-Critic, this value head shares the conv features but has its own weights for the linear output.

- **End-to-End Training:** The entire network (conv + heads) can be trained end-to-end with the RL algorithm's loss functions (policy gradient loss + value loss, etc.). The conv layers will learn to extract those features from the instruction sequence that are relevant for predicting good actions and state value. Gradient backpropagation works through the conv blocks just like any neural net. Notably, because we use **global pooling**, the conv will receive signal about *the whole sequence's outcome*. (If using "same" conv without pooling, you might choose the *last time-step output* to connect to the value/policy, but with circuits of varying length it's safer to pool or average over all time-steps.)

Performance considerations: A Conv1D feature extractor can be heavier than a simple MLP, so monitor the inference speed if your sequences are long. However, convolution can be parallelized across the

sequence, so with a reasonable GPU this architecture should still run efficiently [11]. Also, by sharing the conv trunk between actor & critic, you roughly halve the computations compared to separate networks. Memory-wise, 1D conv layers are quite lean compared to sequence length – they don't need to unroll steps like RNNs, so memory usage scales with network depth rather than sequence length [22] (aside from the need to hold the whole sequence in the forward pass, which you're already doing).

Summary of the proposed architecture:

- **Input:** Quantum circuit as a sequence of instructions, each encoded as a 150-dim vector (features = gate type one-hot ±1, 3 parameters, 130 qubit role flags). So input tensor shape is `(batch, seq_len, 150)` if using batch-first, or `(batch, 150, seq_len)` for channels-first convention. (Many frameworks use batch×channels×length for Conv1D.)
- **Conv1D Stack:** e.g. 6–8 residual blocks, each with two Conv1D layers. Use kernel size 3–5, stride 1, **padding="same"**. Dilation grows each block (1,2,4,8,...). Start with 64 channels, increase to 128 in deeper layers. Each conv is followed by PReLU (or ReLU) and LayerNorm; each block has a skip connection. Apply dropout in blocks (e.g. 10–20% rate). This yields an output of shape `(batch, seq_len, C_final)` (or `(batch, C_final, seq_len)` in channels-first).
- **Global Pooling:** Average (or max) pool over the sequence dimension to get a `(batch, C_final)` vector. This ensures the network output doesn't depend on sequence length (works for any seq_len). [19]
- **Policy Head:** Dense layer from the pooled vector to N_action outputs, then softmax to get π.
- **Value Head:** Dense layer from the pooled vector to 1 output (linear).

This design gives you a **fully convolutional pipeline** from input sequence to output, with no manual feature engineering. It leverages **known-effective CNN patterns** (dilated conv + residual + normalization) to handle sequential patterns. You can confidently initialize with this architecture since it has been used in many settings (from text to time-series) with good results [13] [14]. Adjust the depth or width as needed based on performance (e.g. if the agent struggles to use very long-term information, you might add more layers or slightly enlarge kernels). But overall, this TCN-based design should provide a **strong starting point** for an RL policy network dealing with variable-length quantum circuit inputs.

Sources:

- Bai et al., *"An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling"* – introduced the TCN architecture and showed Conv1D outperforms RNNs on many sequence tasks [1] [11]. Describes using dilations, residual blocks, etc., for long sequences [4] [8]. Residual connections improved training stability and accuracy [7].
- MATLAB Documentation – example of a 1D CNN for sequence classification (uses two Conv1D+ReLU+LayerNorm layers with small kernel=5, followed by global average pooling and dense output) [16] [19]. Demonstrates a simple but effective Conv1D architecture for variable-length data.
- Stable Baselines3 – RL library noting that for policy networks, CNN feature extractors can be shared between actor and critic in on-policy methods like A2C/PPO [21] (recommending a single shared conv backbone with separate output layers for efficiency). This aligns with the proposed architecture for integrating the Conv1D into your RL agent.

---

1 2 3 4 5 6 7 8 10 11 12 15 17 18 22 [1803.01271] An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling
https://ar5iv.labs.arxiv.org/html/1803.01271

9 13 14 Temporal Convolutional Networks and Forecasting - Unit8
https://unit8.com/resources/temporal-convolutional-networks-and-forecasting/

16 19 20 Sequence Classification Using 1-D Convolutions - MATLAB & Simulink
https://www.mathworks.com/help/deeplearning/ug/sequence-classification-using-1-d-convolutions.html

21 Policy Networks — Stable Baselines3 2.7.1a3 documentation
https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Shaojie Bai [1] **J. Zico Kolter** [2] **Vladlen Koltun** [3]

Abstract

For most deep learning practitioners, sequence modeling is synonymous with recurrent networks. Yet recent results indicate that convolutional architectures can outperform recurrent networks on tasks such as audio synthesis and machine translation. Given a new sequence modeling task or dataset, which architecture should one use? We conduct a systematic evaluation of generic convolutional and recurrent architectures for sequence modeling. The models are evaluated across a broad range of standard tasks that are commonly used to benchmark recurrent networks. Our results indicate that a simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks and datasets, while demonstrating longer effective memory. We conclude that the common association between sequence modeling and recurrent networks should be reconsidered, and convolutional networks should be regarded as a natural starting point for sequence modeling tasks. To assist related work, we have made code available at `http://github.com/locuslab/TCN`.

1. Introduction

Deep learning practitioners commonly regard recurrent architectures as the default starting point for sequence modeling tasks. The sequence modeling chapter in the canonical textbook on deep learning is titled "Sequence Modeling: Recurrent and Recursive Nets" (Goodfellow et al., 2016), capturing the common association of sequence modeling and recurrent architectures. A well-regarded recent online course on "Sequence Models" focuses exclusively on recurrent architectures (Ng, 2018).

[1]Machine Learning Department, Carnegie Mellon University, Pittsburgh, PA, USA [2]Computer Science Department, Carnegie Mellon University, Pittsburgh, PA, USA [3]Intel Labs, Santa Clara, CA, USA. Correspondence to: Shaojie Bai <shaojieb@cs.cmu.edu>, J. Zico Kolter <zkolter@cs.cmu.edu>, Vladlen Koltun <vkoltun@gmail.edu>.

On the other hand, recent research indicates that certain convolutional architectures can reach state-of-the-art accuracy in audio synthesis, word-level language modeling, and machine translation (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b). This raises the question of whether these successes of convolutional sequence modeling are confined to specific application domains or whether a broader reconsideration of the association between sequence processing and recurrent networks is in order.

We address this question by conducting a systematic empirical evaluation of convolutional and recurrent architectures on a broad range of sequence modeling tasks. We specifically target a comprehensive set of tasks that have been repeatedly used to compare the effectiveness of different recurrent network architectures. These tasks include polyphonic music modeling, word- and character-level language modeling, as well as synthetic stress tests that had been deliberately designed and frequently used to benchmark RNNs. Our evaluation is thus set up to compare convolutional and recurrent approaches to sequence modeling on the recurrent networks' "home turf".

To represent convolutional networks, we describe a generic temporal convolutional network (TCN) architecture that is applied across all tasks. This architecture is informed by recent research, but is deliberately kept simple, combining some of the best practices of modern convolutional architectures. It is compared to canonical recurrent architectures such as LSTMs and GRUs.

The results suggest that TCNs convincingly outperform baseline recurrent architectures across a broad range of sequence modeling tasks. This is particularly notable because the tasks include diverse benchmarks that have commonly been used to evaluate recurrent network designs (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Zhang et al., 2016). This indicates that the recent successes of convolutional architectures in applications such as audio processing are not confined to these domains.

To further understand these results, we analyze more deeply the memory retention characteristics of recurrent networks. We show that despite the theoretical ability of recurrent architectures to capture infinitely long history, TCNs exhibit

substantially longer memory, and are thus more suitable for domains where a long history is required.

To our knowledge, the presented study is the most extensive systematic comparison of convolutional and recurrent architectures on sequence modeling tasks. The results suggest that the common association between sequence modeling and recurrent networks should be reconsidered. The TCN architecture appears not only more accurate than canonical recurrent networks such as LSTMs and GRUs, but also simpler and clearer. It may therefore be a more appropriate starting point in the application of deep networks to sequences.

2. Background

Convolutional networks (LeCun et al., 1989) have been applied to sequences for decades (Sejnowski & Rosenberg, 1987; Hinton, 1989). They were used prominently for speech recognition in the 80s and 90s (Waibel et al., 1989; Bottou et al., 1990). ConvNets were subsequently applied to NLP tasks such as part-of-speech tagging and semantic role labelling (Collobert & Weston, 2008; Collobert et al., 2011; dos Santos & Zadrozny, 2014). More recently, convolutional networks were applied to sentence classification (Kalchbrenner et al., 2014; Kim, 2014) and document classification (Zhang et al., 2015; Conneau et al., 2017; Johnson & Zhang, 2015; 2017). Particularly inspiring for our work are the recent applications of convolutional architectures to machine translation (Kalchbrenner et al., 2016; Gehring et al., 2017a;b), audio synthesis (van den Oord et al., 2016), and language modeling (Dauphin et al., 2017).

Recurrent networks are dedicated sequence models that maintain a vector of hidden activations that are propagated through time (Elman, 1990; Werbos, 1990; Graves, 2012). This family of architectures has gained tremendous popularity due to prominent applications to language modeling (Sutskever et al., 2011; Graves, 2013; Hermans & Schrauwen, 2013) and machine translation (Sutskever et al., 2014; Bahdanau et al., 2015). The intuitive appeal of recurrent modeling is that the hidden state can act as a representation of everything that has been seen so far in the sequence. Basic RNN architectures are notoriously difficult to train (Bengio et al., 1994; Pascanu et al., 2013) and more elaborate architectures are commonly used instead, such as the LSTM (Hochreiter & Schmidhuber, 1997) and the GRU (Cho et al., 2014). Many other architectural innovations and training techniques for recurrent networks have been introduced and continue to be actively explored (El Hihi & Bengio, 1995; Schuster & Paliwal, 1997; Gers et al., 2002; Koutnik et al., 2014; Le et al., 2015; Ba et al., 2016; Wu et al., 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018).

Multiple empirical studies have been conducted to evaluate the effectiveness of different recurrent architectures. These studies have been motivated in part by the many degrees of freedom in the design of such architectures. Chung et al. (2014) compared different types of recurrent units (LSTM vs. GRU) on the task of polyphonic music modeling. Pascanu et al. (2014) explored different ways to construct deep RNNs and evaluated the performance of different architectures on polyphonic music modeling, character-level language modeling, and word-level language modeling. Jozefowicz et al. (2015) searched through more than ten thousand different RNN architectures and evaluated their performance on various tasks. They concluded that if there were "architectures much better than the LSTM", then they were "not trivial to find". Greff et al. (2017) benchmarked the performance of eight LSTM variants on speech recognition, handwriting recognition, and polyphonic music modeling. They also found that "none of the variants can improve upon the standard LSTM architecture significantly". Zhang et al. (2016) systematically analyzed the connecting architectures of RNNs and evaluated different architectures on character-level language modeling and on synthetic stress tests. Melis et al. (2018) benchmarked LSTM-based architectures on word-level and character-level language modeling, and concluded that "LSTMs outperform the more recent models".

Other recent works have aimed to combine aspects of RNN and CNN architectures. This includes the Convolutional LSTM (Shi et al., 2015), which replaces the fully-connected layers in an LSTM with convolutional layers to allow for additional structure in the recurrent layers; the Quasi-RNN model (Bradbury et al., 2017) that interleaves convolutional layers with simple recurrent layers; and the dilated RNN (Chang et al., 2017), which adds dilations to recurrent architectures. While these combinations show promise in combining the desirable aspects of both types of architectures, our study here focuses on a comparison of generic convolutional and recurrent architectures.

While there have been multiple thorough evaluations of RNN architectures on representative sequence modeling tasks, we are not aware of a similarly thorough comparison of convolutional and recurrent approaches to sequence modeling. (Yin et al. (2017) have reported a comparison of convolutional and recurrent networks for sentence-level and document-level classification tasks. In contrast, sequence modeling calls for architectures that can synthesize whole sequences, element by element.) Such comparison is particularly intriguing in light of the aforementioned recent success of convolutional architectures in this domain. Our work aims to compare generic convolutional and recurrent architectures on typical sequence modeling tasks that are commonly used to benchmark RNN variants themselves (Hermans & Schrauwen, 2013; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016).

3. Temporal Convolutional Networks

We begin by describing a generic architecture for convolutional sequence prediction. Our aim is to distill the best practices in convolutional network design into a simple architecture that can serve as a convenient but powerful starting point. We refer to the presented architecture as a temporal convolutional network (TCN), emphasizing that we adopt this term not as a label for a truly new architecture, but as a simple descriptive term for a family of architectures. (Note that the term has been used before (Lea et al., 2017).) The distinguishing characteristics of TCNs are: 1) the convolutions in the architecture are causal, meaning that there is no information "leakage" from future to past; 2) the architecture can take a sequence of any length and map it to an output sequence of the same length, just as with an RNN. Beyond this, we emphasize how to build very long effective history sizes (i.e., the ability for the networks to look very far into the past to make a prediction) using a combination of very deep networks (augmented with residual layers) and dilated convolutions.

Our architecture is informed by recent convolutional architectures for sequential data (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b), but is distinct from all of them and was designed from first principles to combine simplicity, autoregressive prediction, and very long memory. For example, the TCN is much simpler than WaveNet (van den Oord et al., 2016) (no skip connections across layers, conditioning, context stacking, or gated activations).

Compared to the language modeling architecture of Dauphin et al. (2017), TCNs do not use gating mechanisms and have much longer memory.

3.1. Sequence Modeling

Before defining the network structure, we highlight the nature of the sequence modeling task. Suppose that we are given an input sequence $x_0, \ldots, x_T$, and wish to predict some corresponding outputs $y_0, \ldots, y_T$ at each time. The key constraint is that to predict the output y_t for some time t, we are constrained to only use those inputs that have been previously observed: $x_0, \ldots, x_t$. Formally, a sequence modeling network is any function $f : \mathcal{X}^{T+1} \to \mathcal{Y}^{T+1}$ that produces the mapping

$$\hat{y}_0, \ldots, \hat{y}_T = f(x_0, \ldots, x_T) \tag{1}$$

if it satisfies the causal constraint that y_t depends only on $x_0, \ldots, x_t$ and not on any "future" inputs $x_{t+1}, \ldots, x_T$. The goal of learning in the sequence modeling setting is to find a network f that minimizes some expected loss between the actual outputs and the predictions, $L(y_0, \ldots, y_T, f(x_0, \ldots, x_T))$, where the sequences and outputs are drawn according to some distribution.

This formalism encompasses many settings such as autoregressive prediction (where we try to predict some signal given its past) by setting the target output to be simply the input shifted by one time step. It does not, however, directly capture domains such as machine translation, or sequence-to-sequence prediction in general, since in these cases the entire input sequence (including "future" states) can be used to predict each output (though the techniques can naturally be extended to work in such settings).

3.2. Causal Convolutions

As mentioned above, the TCN is based upon two principles: the fact that the network produces an output of the same length as the input, and the fact that there can be no leakage from the future into the past. To accomplish the first point, the TCN uses a 1D fully-convolutional network (FCN) architecture (Long et al., 2015), where each hidden layer is the same length as the input layer, and zero padding of length $(\text{kernel size} - 1)$ is added to keep subsequent layers the same length as previous ones. To achieve the second point, the TCN uses *causal convolutions*, convolutions where an output at time t is convolved only with elements from time t and earlier in the previous layer.

To put it simply: TCN = 1D FCN + causal convolutions.

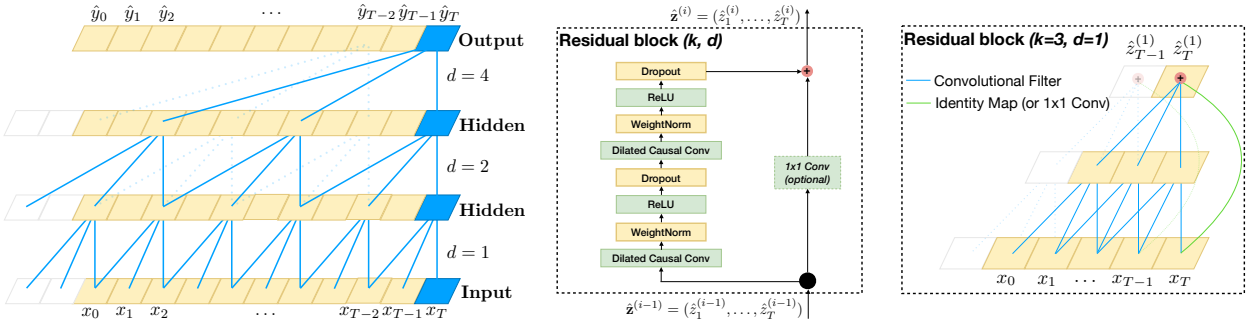
Note that this is essentially the same architecture as the time delay neural network proposed nearly 30 years ago by Waibel et al. (1989), with the sole tweak of zero padding to ensure equal sizes of all layers.

A major disadvantage of this basic design is that in order to achieve a long effective history size, we need an extremely deep network or very large filters, neither of which were particularly feasible when the methods were first introduced. Thus, in the following sections, we describe how techniques from modern convolutional architectures can be integrated into a TCN to allow for both very deep networks and very long effective history.

3.3. Dilated Convolutions

A simple causal convolution is only able to look back at a history with size linear in the depth of the network. This makes it challenging to apply the aforementioned causal convolution on sequence tasks, especially those requiring longer history. Our solution here, following the work of van den Oord et al. (2016), is to employ dilated convolutions that enable an exponentially large receptive field (Yu & Koltun, 2016). More formally, for a 1-D sequence input $\mathbf{x} \in \mathbb{R}^n$ and a filter $f : \{0, \ldots, k-1\} \to \mathbb{R}$, the dilated convolution operation F on element s of the sequence is defined as

$$F(s) = (\mathbf{x} *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i} \tag{2}$$



(a) (b) (c)

Figure 1. Architectural elements in a TCN. (a) A dilated causal convolution with dilation factors $d = 1, 2, 4$ and filter size $k = 3$. The receptive field is able to cover all values from the input sequence. (b) TCN residual block. An 1x1 convolution is added when residual input and output have different dimensions. (c) An example of residual connection in a TCN. The blue lines are filters in the residual function, and the green lines are identity mappings.

where d is the dilation factor, k is the filter size, and $s - d \cdot i$ accounts for the direction of the past. Dilation is thus equivalent to introducing a fixed step between every two adjacent filter taps. When $d = 1$, a dilated convolution reduces to a regular convolution. Using larger dilation enables an output at the top level to represent a wider range of inputs, thus effectively expanding the receptive field of a ConvNet.

This gives us two ways to increase the receptive field of the TCN: choosing larger filter sizes k and increasing the dilation factor d, where the effective history of one such layer is $(k-1)d$. As is common when using dilated convolutions, we increase d exponentially with the depth of the network (i.e., $d = O(2^i)$ at level i of the network). This ensures that there is some filter that hits each input within the effective history, while also allowing for an extremely large effective history using deep networks. We provide an illustration in Figure 1(a).

3.4. Residual Connections

A residual block (He et al., 2016) contains a branch leading out to a series of transformations $\mathcal{F}$, whose outputs are added to the input $\mathbf{x}$ of the block:

$$o = \text{Activation}(\mathbf{x} + \mathcal{F}(\mathbf{x})) \tag{3}$$

This effectively allows layers to learn modifications to the identity mapping rather than the entire transformation, which has repeatedly been shown to benefit very deep networks.

Since a TCN's receptive field depends on the network depth n as well as filter size k and dilation factor d, stabilization of deeper and larger TCNs becomes important. For example, in a case where the prediction could depend on a history of size 2^{12} and a high-dimensional input sequence, a network of up to 12 layers could be needed. Each layer, more specifically, consists of multiple filters for feature extraction. In our design of the generic TCN model, we therefore employ a generic residual module in place of a convolutional layer.

The residual block for our baseline TCN is shown in Figure 1(b). Within a residual block, the TCN has two layers of dilated causal convolution and non-linearity, for which we used the rectified linear unit (ReLU) (Nair & Hinton, 2010). For normalization, we applied weight normalization (Salimans & Kingma, 2016) to the convolutional filters. In addition, a spatial dropout (Srivastava et al., 2014) was added after each dilated convolution for regularization: at each training step, a whole channel is zeroed out.

However, whereas in standard ResNet the input is added directly to the output of the residual function, in TCN (and ConvNets in general) the input and output could have different widths. To account for discrepant input-output widths, we use an additional 1x1 convolution to ensure that element-wise addition $\oplus$ receives tensors of the same shape (see Figure 1(b,c)).

3.5. Discussion

We conclude this section by listing several advantages and disadvantages of using TCNs for sequence modeling.

- **Parallelism**. Unlike in RNNs where the predictions for later timesteps must wait for their predecessors to complete, convolutions can be done in parallel since the same filter is used in each layer. Therefore, in both training and evaluation, a long input sequence can be processed as a whole in TCN, instead of sequentially as in RNN.
- **Flexible receptive field size**. A TCN can change its receptive field size in multiple ways. For instance, stacking more dilated (causal) convolutional layers, using larger dilation factors, or increasing the filter size are all viable options (with possibly different interpretations). TCNs thus afford better control of the model's memory size, and are easy to adapt to different domains.
- **Stable gradients**. Unlike recurrent architectures, TCN has a backpropagation path different from the temporal direction of the sequence. TCN thus avoids the problem

of exploding/vanishing gradients, which is a major issue for RNNs (and which led to the development of LSTM, GRU, HF-RNN (Martens & Sutskever, 2011), etc.).

- **Low memory requirement for training**. Especially in the case of a long input sequence, LSTMs and GRUs can easily use up a lot of memory to store the partial results for their multiple cell gates. However, in a TCN the filters are shared across a layer, with the backpropagation path depending only on network depth. Therefore in practice, we found gated RNNs likely to use up to a multiplicative factor more memory than TCNs.
- **Variable length inputs**. Just like RNNs, which model inputs with variable lengths in a recurrent way, TCNs can also take in inputs of arbitrary lengths by sliding the 1D convolutional kernels. This means that TCNs can be adopted as drop-in replacements for RNNs for sequential data of arbitrary length.

There are also two notable disadvantages to using TCNs.

- **Data storage during evaluation**. In evaluation/testing, RNNs only need to maintain a hidden state and take in a current input x_t in order to generate a prediction. In other words, a "summary" of the entire history is provided by the fixed-length set of vectors h_t, and the actual observed sequence can be discarded. In contrast, TCNs need to take in the raw sequence up to the effective history length, thus possibly requiring more memory during evaluation.
- **Potential parameter change for a transfer of domain**. Different domains can have different requirements on the amount of history the model needs in order to predict. Therefore, when transferring a model from a domain where only little memory is needed (i.e., small k and d) to a domain where much longer memory is required (i.e., much larger k and d), TCN may perform poorly for not having a sufficiently large receptive field.

4. Sequence Modeling Tasks

We evaluate TCNs and RNNs on tasks that have been commonly used to benchmark the performance of different RNN sequence modeling architectures (Hermans & Schrauwen, 2013; Chung et al., 2014; Pascanu et al., 2014; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016). The intention is to conduct the evaluation on the "home turf" of RNN sequence models. We use a comprehensive set of synthetic stress tests along with real-world datasets from multiple domains.

The adding problem. In this task, each input consists of a length-n sequence of depth 2, with all values randomly chosen in $[0, 1]$, and the second dimension being all zeros except for two elements that are marked by 1. The objective is to sum the two random values whose second dimensions are marked by 1. Simply predicting the sum to be 1 should give an MSE of about 0.1767. First introduced by Hochreiter & Schmidhuber (1997), the adding problem has been used repeatedly as a stress test for sequence models (Martens & Sutskever, 2011; Pascanu et al., 2013; Le et al., 2015; Arjovsky et al., 2016; Zhang et al., 2016).

Sequential MNIST and P-MNIST. Sequential MNIST is frequently used to test a recurrent network's ability to retain information from the distant past (Le et al., 2015; Zhang et al., 2016; Wisdom et al., 2016; Cooijmans et al., 2016; Krueger et al., 2017; Jing et al., 2017). In this task, MNIST images (LeCun et al., 1998) are presented to the model as a 784×1 sequence for digit classification. In the more challenging P-MNIST setting, the order of the sequence is permuted at random (Le et al., 2015; Arjovsky et al., 2016; Wisdom et al., 2016; Krueger et al., 2017).

Copy memory. In this task, each input sequence has length $T + 20$. The first 10 values are chosen randomly among the digits $1, \ldots, 8$, with the rest being all zeros, except for the last 11 entries that are filled with the digit '9' (the first '9' is a delimiter). The goal is to generate an output of the same length that is zero everywhere except the last 10 values after the delimiter, where the model is expected to repeat the 10 values it encountered at the start of the input. This task was used in prior works such as Zhang et al. (2016); Arjovsky et al. (2016); Wisdom et al. (2016); Jing et al. (2017).

JSB Chorales and Nottingham. JSB Chorales (Allan & Williams, 2005) is a polyphonic music dataset consisting of the entire corpus of 382 four-part harmonized chorales by J. S. Bach. Each input is a sequence of elements. Each element is an 88-bit binary code that corresponds to the 88 keys on a piano, with 1 indicating a key that is pressed at a given time. Nottingham is a polyphonic music dataset based on a collection of 1,200 British and American folk tunes, and is much larger than JSB Chorales. JSB Chorales and Nottingham have been used in numerous empirical investigations of recurrent sequence modeling (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Greff et al., 2017). The performance on both tasks is measured in terms of negative log-likelihood (NLL).

PennTreebank. We used the PennTreebank (PTB) (Marcus et al., 1993) for both character-level and word-level language modeling. When used as a character-level language corpus, PTB contains 5,059K characters for training, 396K for validation, and 446K for testing, with an alphabet size of 50. When used as a word-level language corpus, PTB contains 888K words for training, 70K for validation, and 79K for testing, with a vocabulary size of 10K. This is a highly studied but relatively small language modeling dataset (Miyamoto & Cho, 2016; Krueger et al., 2017; Merity et al., 2017).

Wikitext-103. Wikitext-103 (Merity et al., 2016) is almost

Table 1. Evaluation of TCNs and recurrent architectures on synthetic stress tests, polyphonic music modeling, character-level language modeling, and word-level language modeling. The generic TCN architecture outperforms canonical recurrent networks across a comprehensive suite of tasks and datasets. Current state-of-the-art results are listed in the supplement. ^{h} means that higher is better. $^\ell$ means that lower is better.

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	**TCN**
Seq. MNIST (accuracy^h)	70K	87.2	96.2	21.5	**99.0**
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	**97.2**
Adding problem T=600 (loss$^\ell$)	70K	0.164	**5.3e-5**	0.177	**5.8e-5**
Copy memory T=1000 (loss)	16K	0.0204	0.0197	0.0202	**3.5e-5**
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	**8.10**
Music Nottingham (loss)	1M	3.29	3.46	4.05	**3.07**
Word-level PTB (perplexity$^\ell$)	13M	**78.93**	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	**45.19**
Word-level LAMBADA (perplexity)	-	4186	-	14725	**1279**
Char-level PTB (bpc$^\ell$)	3M	1.36	1.37	1.48	**1.31**
Char-level text8 (bpc)	5M	1.50	1.53	1.69	**1.45**

110 times as large as PTB, featuring a vocabulary size of about 268K. The dataset contains 28K Wikipedia articles (about 103 million words) for training, 60 articles (about 218K words) for validation, and 60 articles (246K words) for testing. This is a more representative and realistic dataset than PTB, with a much larger vocabulary that includes many rare words, and has been used in Merity et al. (2016); Grave et al. (2017); Dauphin et al. (2017).

LAMBADA. Introduced by Paperno et al. (2016), LAMBADA is a dataset comprising 10K passages extracted from novels, with an average of 4.6 sentences as context, and 1 target sentence the last word of which is to be predicted. This dataset was built so that a person can easily guess the missing word when given the context sentences, but not when given only the target sentence without the context sentences. Most of the existing models fail on LAMBADA (Paperno et al., 2016; Grave et al., 2017). In general, better results on LAMBADA indicate that a model is better at capturing information from longer and broader context. The training data for LAMBADA is the full text of 2,662 novels with more than 200M words. The vocabulary size is about 93K.

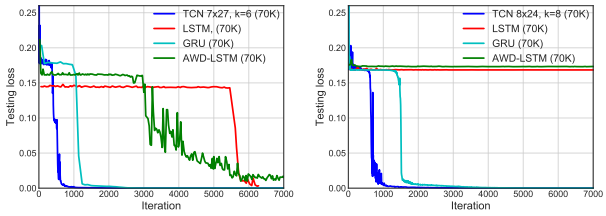
text8. We also used the text8 dataset for character-level language modeling (Mikolov et al., 2012). text8 is about 20 times larger than PTB, with about 100M characters from Wikipedia (90M for training, 5M for validation, and 5M for testing). The corpus contains 27 unique alphabets.

5. Experiments

We compare the generic TCN architecture described in Section 3 to canonical recurrent architectures, namely LSTM, GRU, and vanilla RNN, with standard regularizations. All experiments reported in this section used exactly the same TCN architecture, just varying the depth of the network n and occasionally the kernel size k so that the receptive field covers enough context for predictions. We use an exponential dilation $d = 2^i$ for layer i in the network, and the Adam optimizer (Kingma & Ba, 2015) with learning rate 0.002 for TCN, unless otherwise noted. We also empirically find that gradient clipping helped convergence, and we pick the maximum norm for clipping from $[0.3, 1]$. When training recurrent models, we use grid search to find a good set of hyperparameters (in particular, optimizer, recurrent drop $p \in [0.05, 0.5]$, learning rate, gradient clipping, and initial forget-gate bias), while keeping the network around the same size as TCN. No other architectural elaborations, such as gating mechanisms or skip connections, were added to either TCNs or RNNs. Additional details and controlled experiments are provided in the supplementary material.

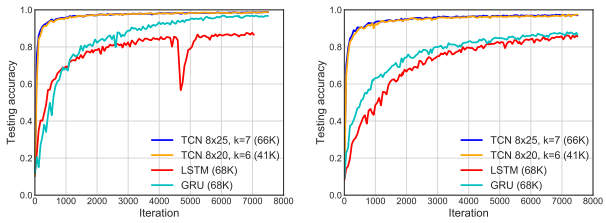
5.1. Synopsis of Results

A synopsis of the results is shown in Table 1. Note that on several of these tasks, the generic, canonical recurrent architectures we study (e.g., LSTM, GRU) are not the state-of-the-art. (See the supplement for more details.) With this caveat, the results strongly suggest that the generic TCN architecture *with minimal tuning* outperforms canonical recurrent architectures across a broad variety of sequence modeling tasks that are commonly used to benchmark the performance of recurrent architectures themselves. We now analyze these results in more detail.



(a) $T = 200$ (b) $T = 600$

Figure 2. Results on the adding problem for different sequence lengths T. TCNs outperform recurrent architectures.



(a) Sequential MNIST (b) P-MNIST

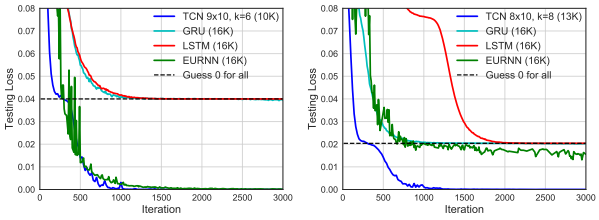
Figure 3. Results on Sequential MNIST and P-MNIST. TCNs outperform recurrent architectures.

5.2. Synthetic Stress Tests

The adding problem. Convergence results for the adding problem, for problem sizes $T = 200$ and 600, are shown in Figure 2. All models were chosen to have roughly 70K parameters. TCNs quickly converged to a virtually perfect solution (i.e., MSE near 0). GRUs also performed quite well, albeit slower to converge than TCNs. LSTMs and vanilla RNNs performed significantly worse.

Sequential MNIST and P-MNIST. Convergence results on sequential and permuted MNIST, run over 10 epochs, are shown in Figure 3. All models were configured to have roughly 70K parameters. For both problems, TCNs substantially outperform the recurrent architectures, both in terms of convergence and in final accuracy on the task. For P-MNIST, TCNs outperform state-of-the-art results (95.9%) based on recurrent networks with Zoneout and Recurrent BatchNorm (Cooijmans et al., 2016; Krueger et al., 2017).

Copy memory. Convergence results on the copy memory task are shown in Figure 4. TCNs quickly converge to correct answers, while LSTMs and GRUs simply converge to the same loss as predicting all zeros. In this case we also compare to the recently-proposed EURNN (Jing et al., 2017), which was highlighted to perform well on this task. While both TCN and EURNN perform well for sequence length $T = 500$, the TCN has a clear advantage for $T = 1000$ and longer (in terms of both loss and rate of convergence).



(a) $T = 500$ (b) $T = 1000$

Figure 4. Result on the copy memory task for different sequence lengths T. TCNs outperform recurrent architectures.

5.3. Polyphonic Music and Language Modeling

We now discuss the results on polyphonic music modeling, character-level language modeling, and word-level language modeling. These domains are dominated by recurrent architectures, with many specialized designs developed for these tasks (Zhang et al., 2016; Ha et al., 2017; Krueger et al., 2017; Grave et al., 2017; Greff et al., 2017; Merity et al., 2017). We mention some of these specialized architectures when useful, but our primary goal is to compare the generic TCN model to similarly generic recurrent architectures, before domain-specific tuning. The results are summarized in Table 1.

Polyphonic music. On Nottingham and JSB Chorales, the TCN with virtually no tuning outperforms the recurrent models by a considerable margin, and even outperforms some enhanced recurrent architectures for this task such as HF-RNN (Boulanger-Lewandowski et al., 2012) and Diagonal RNN (Subakan & Smaragdis, 2017). Note however that other models such as the Deep Belief Net LSTM perform better still (Vohra et al., 2015); we believe this is likely due to the fact that the datasets are relatively small, and thus the right regularization method or generative modeling procedure can improve performance significantly. This is largely orthogonal to the RNN/TCN distinction, as a similar variant of TCN may well be possible.

Word-level language modeling. Language modeling remains one of the primary applications of recurrent networks and many recent works have focused on optimizing LSTMs for this task (Krueger et al., 2017; Merity et al., 2017). Our implementation follows standard practice that ties the weights of encoder and decoder layers for both TCN and RNNs (Press & Wolf, 2016), which significantly reduces the number of parameters in the model. For training, we use SGD and anneal the learning rate by a factor of 0.5 for both TCN and RNNs when validation accuracy plateaus.

On the smaller PTB corpus, an optimized LSTM architecture (with recurrent and embedding dropout, etc.) outperforms the TCN, while the TCN outperforms both GRU and vanilla RNN. However, on the much larger Wikitext-103 corpus and the LAMBADA dataset (Paperno et al., 2016), without any hyperparameter search, the TCN outperforms

the LSTM results of Grave et al. (2017), achieving much lower perplexities.

Character-level language modeling. On character-level language modeling (PTB and text8, accuracy measured in bits per character), the generic TCN outperforms regularized LSTMs and GRUs as well as methods such as Norm-stabilized LSTMs (Krueger & Memisevic, 2015). (Specialized architectures exist that outperform all of these, see the supplement.)

5.4. Memory Size of TCN and RNNs

One of the theoretical advantages of recurrent architectures is their unlimited memory: the theoretical ability to retain information through sequences of unlimited length. We now examine specifically how long the different architectures can retain information in practice. We focus on 1) the copy memory task, which is a stress test designed to evaluate long-term, distant information propagation in recurrent networks, and 2) the LAMBADA task, which tests both local and non-local textual understanding.

The copy memory task is perfectly set up to examine a model's ability to retain information for different lengths of time. The requisite retention time can be controlled by varying the sequence length T. In contrast to Section 5.2, we now focus on the accuracy on the last 10 elements of the output sequence (which are the nontrivial elements that must be recalled). We used models of size 10K for both TCN and RNNs.

The results of this focused study are shown in Figure 5. TCNs consistently converge to 100% accuracy for all sequence lengths, whereas LSTMs and GRUs of the same size quickly degenerate to random guessing as the sequence length T grows. The accuracy of the LSTM falls below 20% for $T < 50$, while the GRU falls below 20% for $T < 200$. These results indicate that TCNs are able to maintain a much longer effective history than their recurrent counterparts.

This observation is backed up on real data by experiments on the large-scale LAMBADA dataset, which is specifically designed to test a model's ability to utilize broad context (Paperno et al., 2016). As shown in Table 1, TCN outperforms LSTMs and vanilla RNNs by a significant margin in perplexity on LAMBADA, with a substantially smaller network and virtually no tuning. (State-of-the-art results on this dataset are even better, but only with the help of additional memory mechanisms (Grave et al., 2017).)

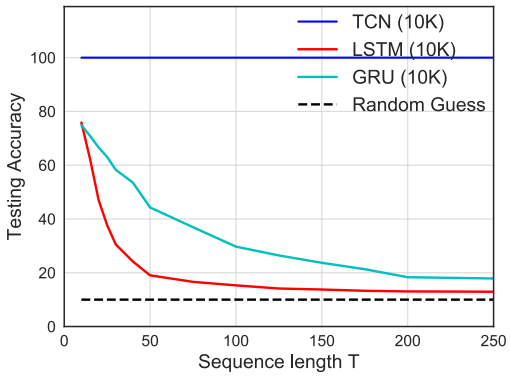


Figure 5. Accuracy on the copy memory task for sequences of different lengths T. While TCN exhibits 100% accuracy for all sequence lengths, the LSTM and GRU degenerate to random guessing as T grows.

6. Conclusion

We have presented an empirical evaluation of generic convolutional and recurrent architectures across a comprehensive suite of sequence modeling tasks. To this end, we have described a simple temporal convolutional network (TCN) that combines best practices such as dilations and residual connections with the causal convolutions needed for autoregressive prediction. The experimental results indicate that TCN models substantially outperform generic recurrent architectures such as LSTMs and GRUs. We further studied long-range information propagation in convolutional and recurrent networks, and showed that the "infinite memory" advantage of RNNs is largely absent in practice. TCNs exhibit longer memory than recurrent architectures with the same capacity.

Numerous advanced schemes for regularizing and optimizing LSTMs have been proposed (Press & Wolf, 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018). These schemes have significantly advanced the accuracy achieved by LSTM-based architectures on some datasets. The TCN has not yet benefitted from this concerted community-wide investment into architectural and algorithmic elaborations. We see such investment as desirable and expect it to yield advances in TCN performance that are commensurate with the advances seen in recent years in LSTM performance. We will release the code for our project to encourage this exploration.

The preeminence enjoyed by recurrent networks in sequence modeling may be largely a vestige of history. Until recently, before the introduction of architectural elements such as dilated convolutions and residual connections, convolutional architectures were indeed weaker. Our results indicate that with these elements, a simple convolutional architecture is more effective across diverse sequence modeling tasks than recurrent architectures such as LSTMs. Due to the comparable clarity and simplicity of TCNs, we conclude that convolutional networks should be regarded as a natural starting point and a powerful toolkit for sequence modeling.

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling
Supplementary Material

A. Hyperparameters Settings

A.1. Hyperparameters for TCN

Table 2 lists the hyperparameters we used when applying the generic TCN model on various tasks and datasets. The most important factor for picking parameters is to make sure that the TCN has a sufficiently large receptive field by choosing k and d that can cover the amount of context needed for the task.

As discussed in Section 5, the number of hidden units was chosen so that the model size is approximately at the same level as the recurrent models with which we are comparing. In Table 2, a gradient clip of N/A means no gradient clipping was applied. In larger tasks (e.g., language modeling), we empirically found that gradient clipping (we randomly picked a threshold from $[0.3, 1]$) helps with regularizing TCN and accelerating convergence.

All weights were initialized from a Gaussian disitribution $\mathcal{N}(0, 0.01)$. In general, we found TCN to be relatively insensitive to hyperparameter changes, as long as the effective history (i.e., receptive field) size is sufficient.

A.2. Hyperparameters for LSTM/GRU

Table 3 reports hyperparameter settings that were used for the LSTM. These values are picked from hyperparameter search for LSTMs that have up to 3 layers, and the optimizers are chosen from {SGD, Adam, RMSprop, Adagrad}. For certain larger datasets, we adopted the settings used in prior work (e.g., Grave et al. (2017) on Wikitext-103). GRU hyperparameters were chosen in a similar fashion, but typically with more hidden units than in LSTM to keep the total network size approximately the same (since a GRU cell is more compact).

B. State-of-the-Art Results

As previously noted, the generic TCN and LSTM/GRU models we used can be outperformed by more specialized architectures on some tasks. State-of-the-art results are summarized in Table 4. The same TCN architecture is used across all tasks. Note that the size of the state-of-the-art model may be different from the size of the TCN.

C. Effect of Filter Size and Residual Block

In this section we briefly study the effects of different components of a TCN layer. Overall, we believe dilation is required for modeling long-term dependencies, and so we mainly focus on two other factors here: the filter size k used by each layer, and the effect of residual blocks.

We perform a series of controlled experiments, with the results of the ablative analysis shown in Figure 6. As before, we kept the model size and depth exactly the same for different models, so that the dilation factor is strictly controlled. The experiments were conducted on three different tasks: copy memory, permuted MNIST (P-MNIST), and Penn Treebank word-level language modeling. These experiments confirm that both factors (filter size and residual connections) contribute to sequence modeling performance.

Filter size k. In both the copy memory and the P-MNIST tasks, we observed faster convergence and better accuracy for larger filter sizes. In particular, looking at Figure 6a, a TCN with filter size ≤ 3 only converges to the same level as random guessing. In contrast, on word-level language modeling, a smaller kernel with filter size of $k = 3$ works best. We believe this is because a smaller kernel (along with fixed dilation) tends to focus more on the local context, which is especially important for PTB language modeling (in fact, the very success of n-gram models suggests that only a relatively short memory is needed for modeling language).

Residual block. In all three scenarios that we compared here, we observed that the residual function stabilized training and brought faster convergence with better final results. Especially in language modeling, we found that residual connections contribute substantially to performance (See Figure 6f).

D. Gating Mechanisms

One component that had been used in prior work on convolutional architectures for language modeling is the gated activation (van den Oord et al., 2016; Dauphin et al., 2017). We have chosen not to use gating in the generic TCN model. We now examine this choice more closely.

Table 2. TCN parameter settings for experiments in Section 5.

TCN SETTINGS							
Dataset/Task	**Subtask**	k	n	**Hidden**	**Dropout**	**Grad Clip**	**Note**
The Adding Problem	$T = 200$	6	7	27	0.0	N/A	
	$T = 400$	7	7	27			
	$T = 600$	8	8	24			
Seq. MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Permuted MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Copy Memory Task	$T = 500$	6	9	10	0.05	1.0	RMSprop 5e-4
	$T = 1000$	8	8	10			
	$T = 2000$	8	9	10			
Music JSB Chorales	-	3	2	150	0.5	0.4	
Music Nottingham	-	6	4	150	0.2	0.4	
Word-level LM	PTB	3	4	600	0.5	0.4	Embed. size 600
	Wiki-103	3	5	1000	0.4		Embed. size 400
	LAMBADA	4	5	500			Embed. size 500
Char-level LM	PTB	3	3	450	0.1	0.15	Embed. size 100
	text8	2	5	520			

Dauphin et al. (2017) compared the effects of gated linear units (GLU) and gated tanh units (GTU), and adopted GLU in their non-dilated gated ConvNet. Following the same choice, we now compare TCNs using ReLU and TCNs with gating (GLU), represented by an elementwise product between two convolutional layers, with one of them also passing through a sigmoid function $\sigma(x)$. Note that the gates architecture uses approximately twice as many convolutional layers as the ReLU-TCN.

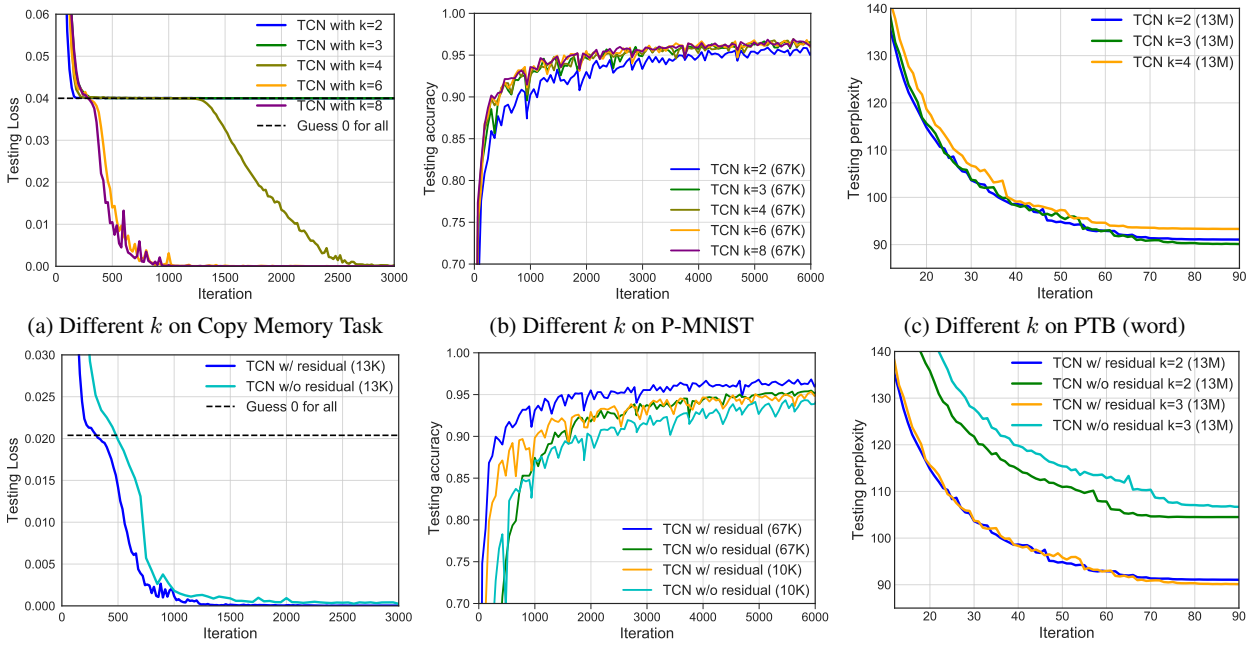
The results are shown in Table 5, where we kept the number of model parameters at about the same size. The GLU does further improve TCN accuracy on certain language modeling datasets like PTB, which agrees with prior work. However, we do not observe comparable benefits on other tasks, such as polyphonic music modeling or synthetic stress tests that require longer information retention. On the copy memory task with $T = 1000$, we found that TCN with gating converged to a worse result than TCN with ReLU (though still better than recurrent models).

Table 3. LSTM parameter settings for experiments in Section 5.

LSTM SETTINGS (KEY PARAMETERS)							
Dataset/Task	**Subtask**	n	**Hidden**	**Dropout**	**Grad Clip**	**Bias**	**Note**
The Adding Problem	$T = 200$	2	77	0.0	50	5.0	SGD 1e-3
	$T = 400$	2	77		50	10.0	Adam 2e-3
	$T = 600$	1	130		5	1.0	-
Seq. MNIST	-	1	130	0.0	1	1.0	RMSprop 1e-3
Permuted MNIST	-	1	130	0.0	1	10.0	RMSprop 1e-3
Copy Memory Task	$T = 500$	1	50	0.05	0.25	-	RMSprop/Adam
	$T = 1000$	1	50		1		
	$T = 2000$	3	28		1		
Music JSB Chorales	-	2	200	0.2	1	10.0	SGD/Adam
Music Nottingham	-	3	280	0.1	0.5	-	Adam 4e-3
		1	500		1	-	
Word-level LM	PTB	3	700	0.4	0.3	1.0	SGD 30, Emb. 700, etc.
	Wiki-103	-	-	-	-	-	Grave et al. (2017)
	LAMBADA	-	-	-	-	-	Grave et al. (2017)
Char-level LM	PTB	2	600	0.1	0.5	-	Emb. size 120
	text8	1	1024	0.15	0.5	-	Adam 1e-2

Table 4. State-of-the-art (SoTA) results for tasks in Section 5.

TCN VS. SOTA RESULTS					
Task	**TCN Result**	**Size**	**SoTA**	**Size**	**Model**
Seq. MNIST (acc.)	99.0	21K	99.0	21K	Dilated GRU (Chang et al., 2017)
P-MNIST (acc.)	97.2	42K	95.9	42K	Zoneout (Krueger et al., 2017)
Adding Prob. 600 (loss)	5.8e-5	70K	5.3e-5	70K	Regularized GRU
Copy Memory 1000 (loss)	3.5e-5	70K	0.011	70K	EURNN (Jing et al., 2017)
JSB Chorales (loss)	8.10	300K	3.47	-	DBN+LSTM (Vohra et al., 2015)
Nottingham (loss)	3.07	1M	1.32	-	DBN+LSTM (Vohra et al., 2015)
Word PTB (ppl)	88.68	13M	47.7	22M	AWD-LSTM-MoS + Dynamic Eval. (Yang et al., 2018)
Word Wiki-103 (ppl)	45.19	148M	40.4	>300M	Neural Cache Model (Large) (Grave et al., 2017)
Word LAMBADA (ppl)	1279	56M	138	>100M	Neural Cache Model (Large) (Grave et al., 2017)
Char PTB (bpc)	1.31	3M	1.22	14M	2-LayerNorm HyperLSTM (Ha et al., 2017)
Char text8 (bpc)	1.45	4.6M	1.29	>12M	HM-LSTM (Chung et al., 2016)



(a) Different k on Copy Memory Task (b) Different k on P-MNIST (c) Different k on PTB (word)

(d) Residual on Copy Memory Task (e) Residual on P-MNIST (f) Residual on PTB (word)

Figure 6. Controlled experiments that study the effect of different components of the TCN model.

Table 5. An evaluation of gating in TCN. A plain TCN is compared to a TCN that uses gated activations.

Task	**TCN**	**TCN + Gating**
Sequential MNIST (acc.)	**99.0**	**99.0**
Permuted MNIST (acc.)	**97.2**	96.9
Adding Problem $T = 600$ (loss)	**5.8e-5**	**5.6e-5**
Copy Memory $T = 1000$ (loss)	**3.5e-5**	0.00508
JSB Chorales (loss)	**8.10**	8.13
Nottingham (loss)	**3.07**	3.12
Word-level PTB (ppl)	88.68	**87.94**
Char-level PTB (bpc)	1.31	**1.306**
Char text8 (bpc)	**1.45**	1.485

Unit8

Temporal Convolutional Networks and Forecasting

Share This Article

Jul 6, 2021

16 min read

Written By

Francesco Lässig

How a convolutional network with some simple adaptations can become a powerful tool for sequence modeling and forecasting.

Although commonly associated with image classification tasks, convolutional neural networks (CNNs) have proven to be valuable tools for sequence modeling and forecasting, given the right modifications. In this article we explore in detail the basic building blocks that a Temporal Convolutional Network (TCN) consists of, and how they all fit together to create a powerful forecasting model. Using our open-source *Darts* TCN implementation, we show that accurate predictions can be achieved on a real-world dataset with only a few lines of code.

The following description of a Temporal Convolutional Network is based on the followir https://arxiv.org/pdf/1803.01271.pdf. References to this paper will be denoted by (*).

Motivation

Up until recently, the topic of sequence modeling in the context of deep learning has been largely associated with recurrent neural network architectures such as LSTM and GRU. S. Bai et al. (*) suggest that this way of thinking is antiquated, and that convolutional networks should be taken into consideration as one of the primary candidates when modeling sequential data. They were able to show that convolutional networks can achieve better performance than RNNs in many tasks while avoiding common drawbacks of recurrent models, such as the exploding/vanishing gradient problem or lacking memory retention. Furthermore, using a convolutional network instead of a recurrent one can lead to performance improvements as it allows for parallel computation of outputs. The architecture they propose is called Temporal Convolutional Network (TCN) and will be explained in the following sections. To facilitate understanding the TCN architecture in conjunction with its *Darts* implementation, this article will use the same model parameter names as seen in the library wherever possible (indicated in bold).

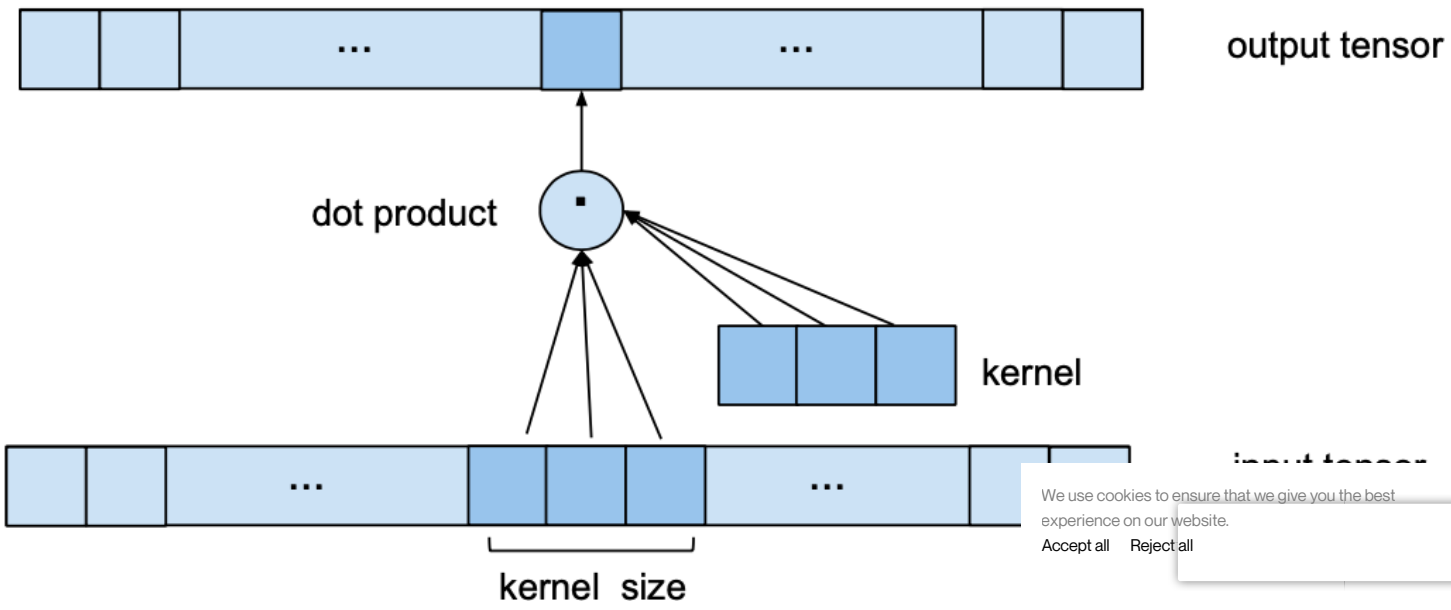
Overview

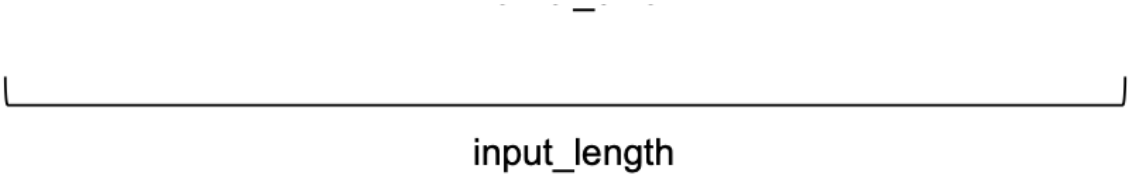
A TCN, short for Temporal Convolutional Network, consists of dilated, causal 1D convolutional layers with the same input and output lengths. The following sections go into detail about what these terms actually mean.

1D Convolutional Network

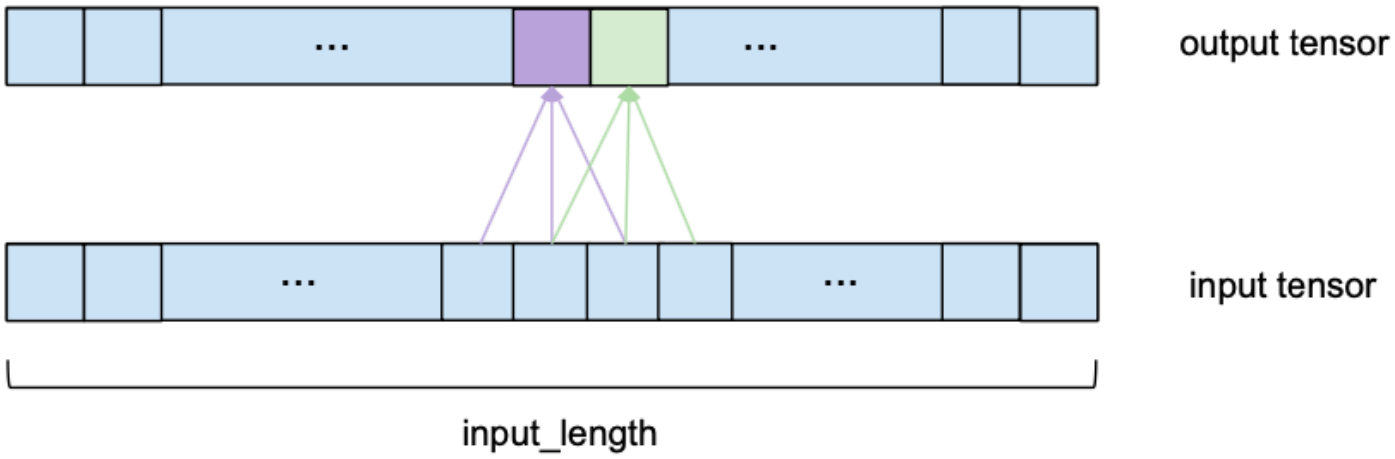
A 1D convolutional network takes as input a 3-dimensional tensor and also outputs a 3-dimensional tensor. The input tensor of our TCN implementation has the shape (batch_size, input_length, input_size) and the output tensor has the shape (batch_size, input_length, output_size). Since every layer in a TCN has the same input and output length, only the third dimension of the input and output tensors differs. In the univariate case, input_size and output_size will both be equal to one. In the more general multivariate case, input_size and output_size might differ since we might not want to forecast every component of the input sequence.

One single 1D convolutional layer receives an input tensor of shape (batch_size, input_length, *nr_input_channels*) and outputs a tensor of shape (batch_size, input_length, *nr_output_channels*). To see how one single layer converts its input into the output, let's look at one element of the batch (the same process takes place for every element in the batch). Let's start with the simplest case where *nr_input_channels* and *nr_output_channels* both equal 1. In this case we are looking at 1D input and output tensors. The following image shows how one element of the output tensor is computed.





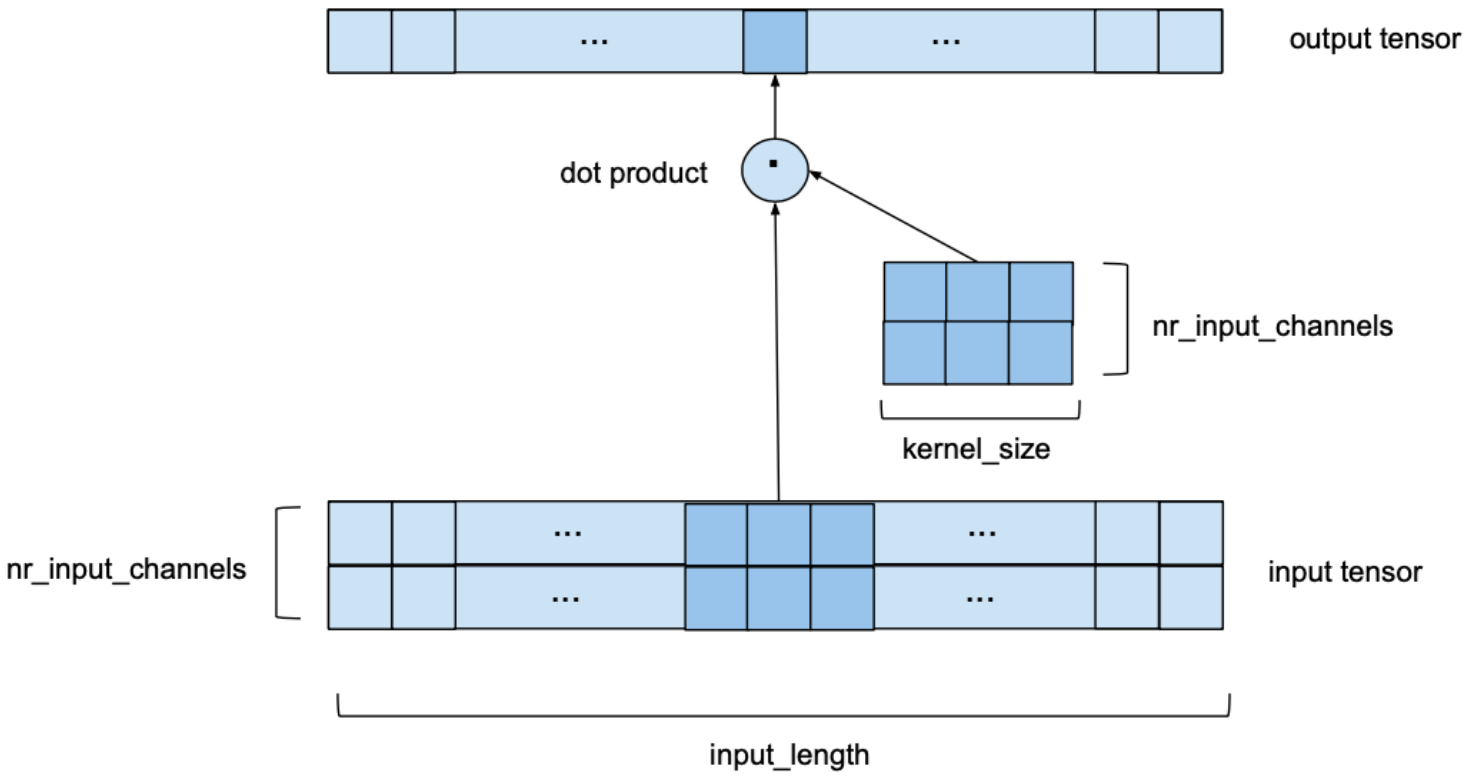
We can see that to compute one element of the output, we look at a series of consecutive elements of length kernel_size of the input. In the above example we chose a kernel_size of 3. To obtain the output, we take the dot product of the subsequence of the input and a kernel vector of learned weights of the same length. To get the next element of the output, the same procedure is applied, but the kernel_size-sized window of the input sequence is shifted to the right by one element (for the purposes of this forecasting model, the stride is always set to 1). Please note that the same set of kernel weights will be used to compute every output of one convolutional layer. The following image shows two consecutive output elements and their respective input subsequences.



To make the visualization simpler, the dot product with the kernel vector is not shown anymore, but takes place for every output element with the same kernel weights.

To make sure that the output sequence has the same length as the input sequence, some zero-padding is applied. This means that additional zero-valued entries are added to either the beginning or the end of the input tensor to ensure that the output has the desired length. How this is done exactly will be explained in later sections.

Now let's look at the case where we have multiple input channels, i.e. *nr_input_channels* is larger than 1. In this case, the process described above is repeated for every single input channel, but each time with a different kernel. This leads to *nr_input_channels* intermediate output vectors and a number of kernel weights of kernel_size * *nr_input_channels*. Then, all the intermediate output vectors are summed up to obtain the final output vector. In some sense this is equivalent to a 2D convolution with an input tensor of shape (input_size, *nr_input_channels*) and a kernel of shape (kernel_size, *nr_input_channels*) as shown in the following image. It's still 1D in the sense that the window only moves along a single axis, but we do have a 2D convolution at every step in the sense that we are using a 2-dimensional kernel matrix.



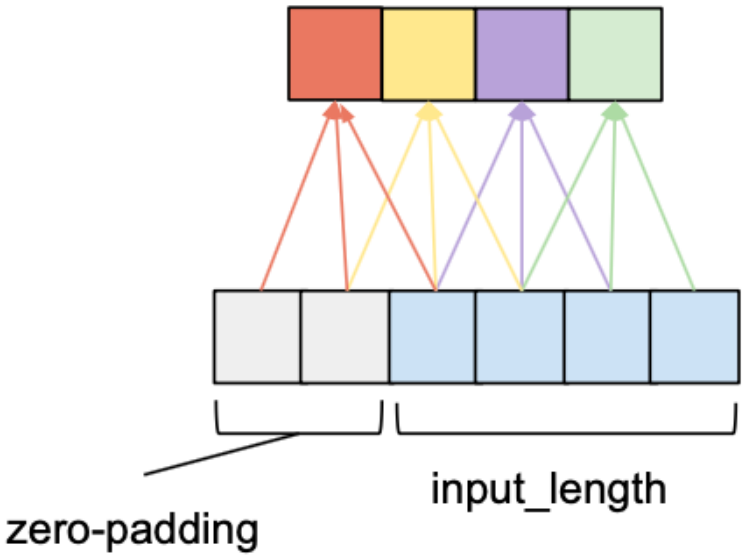
For this example we chose *nr_input_channels* to be equal to 2. Now, instead of a kernel vector sliding over a 1-dimensional input sequence we have a *nr_input_channels* by kernel_size kernel matrix sliding along a *nr_input_channels* wide series of length input_length.

If both *nr_input_channels* and *nr_output_channels* are larger than 1, the above process is simply repeated for every output channel with a different kernel matrix. The output vectors are then stacked on top of each other resulting in an output tensor of shape (input_length, *nr_output_channels*). The number of kernel weights in this case is equal to kernel_size* *nr_input_channels** *nr_output_channels*.

The two variables *nr_input_channels* and *nr_output_channels* depend on the position of the layer within the network. The first layer will have *nr_input_channels* = input_size, and the last layer will have *nr_output_channels* = output_size. All other layers will use the intermediate channel number given by num_filters.

Causal Convolution

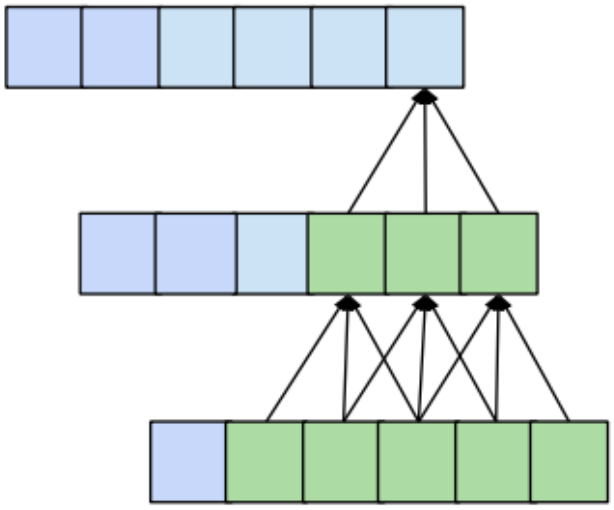
For a convolutional layer to be causal, for every i in {0, ..., input_length — 1} the i'th element of the output sequence may only depend on the elements of the input sequence with indices {0, ..., i}. In other words, an element in the output sequence can only depend on elements that come before it in the input sequence. As mentioned before, to ensure an output tensor has the same length as the input tensor, we need to apply zero padding. If we only apply zero-padding on the left side of the input tensor, then causal convolution will be ensured. To understand this, consider the rightmost output element. Given that there is no padding on the right side of the input sequence, the last element it depends on is the last element of the input. Now consider the second to last output element of the output sequence. Its kernel window is shifted to the left by one compared to the last output element, that means its rightmost dependency in the input sequence is the second to last element of the input sequence. It follows by induction that for every element in the output sequence, its latest dependency in the input sequence has the same index as itself. The following figure shows an exam
input_length of 4 and a kernel_size of 3.



We can see that with a left zero-padding of 2 entries we can achieve the same output length while obeying the causality rule. In fact, without dilation, the number of zero-padding entries required for maintaining the input length is always equal to kernel_size – 1.

Dilation

One desirable quality of a forecasting model is that the value of a specific entry in the output depends on all previous entries in the input, i.e. all entries that have an index smaller or equal to itself. This is achieved when the receptive field, meaning the set of entries of the original input that affect a specific entry of the output, has size input_length. We also call this 'full history coverage'. As we have seen before, one conventional convolutional layer makes an entry in the output dependent on the kernel_size entries of the input that have an index smaller or equal to itself. For instance, if we have a kernel_size of 3, the 5th element in the output will be dependent on elements 3, 4 and 5 of the input. This reach is expanded when we stack multiple layers on top of each other. In the following figure we can see that by stacking two layers with kernel_size 3 we get a receptive field size of 5.



More generally, a 1D convolutional network with n layers and a kernel_size k has a receptive field r of size

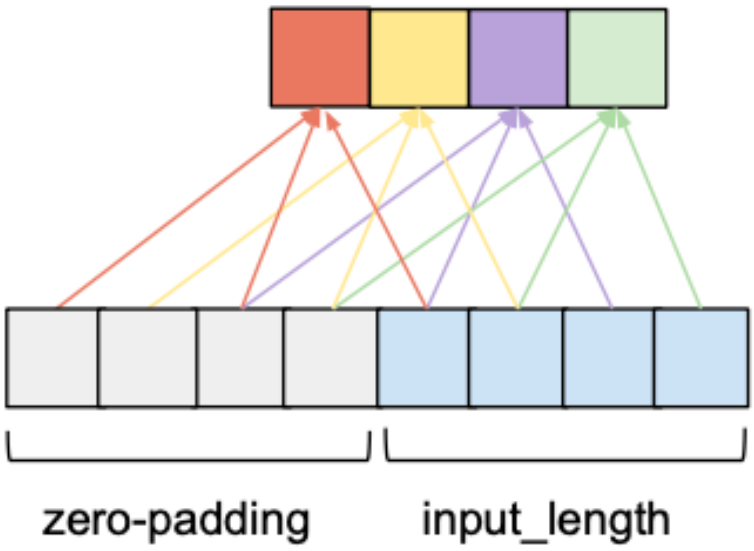
$$r = 1 + n * (k - 1)$$

To know how many layers are needed for full coverage, we can set the receptive field size to input_length l and solve for the number of layers n (we need to round up in case of non-integer values):

$$n = \lceil (l - 1)/(k - 1) \rceil$$

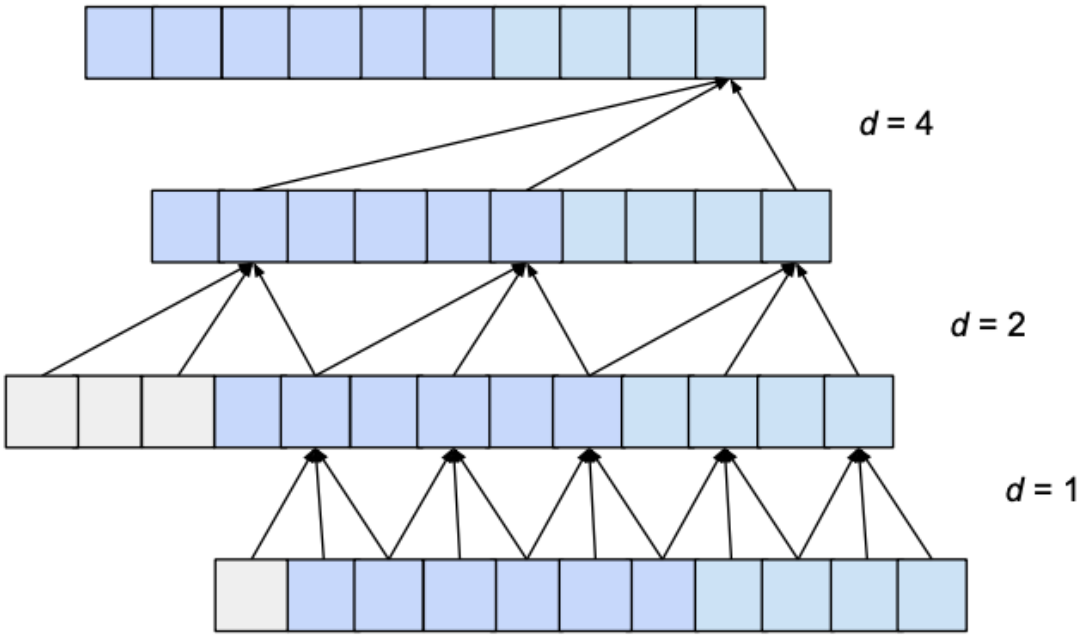
This means that, given a fixed kernel_size, the number of layers required for full history coverage is linear in the length of the input tensor, which will result in networks that become very deep very fast, leading to models with a very large number of parameters that take longer to train. Furthermore, a high number of layers has been shown to lead to degradation problems related to the gradient of the loss function. One way to increase the receptive field size while still keeping the number of layers relatively small is to introduce dilation to the convolutional network.

Dilation in the context of a convolutional layer refers to the distance between the elements of the input sequence that are used to compute one entry of the output sequence. So a conventional convolutional layer could be seen as a 1-dilated layer, since the input elements for 1 output value are adjacent. The following figure shows an example of a 2-dilated layer with an input_length of 4 and a kernel_size of 3.



Compared to the 1-dilated case, this layer has a receptive field that spreads across a length of 5 rather than 3. More generally, a *d*-dilated layer with a kernel size *k* has a receptive field spreading across a length of 1+d*(k-1). If *d* is fixed, this will still require a number linear in the length of the input tensor to achieve full receptive field coverage (we just decreased the constant).

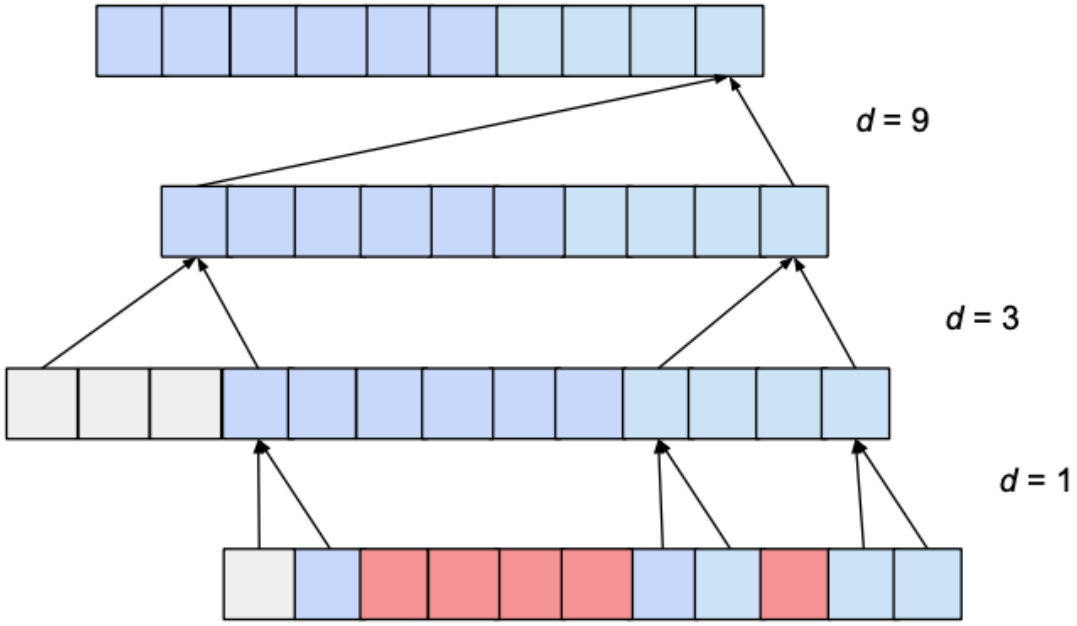
This problem can be addressed by increasing the value of *d* exponentially as we move up through the layers. For this, we choose a constant dilation_base integer *b* which will let us compute the dilation *d* of a specific layer as a function of the number of layers below it, *i*, as *d = b**i*. The following figure shows a network with an input_length of 10, a kernel_size of 3 and a dilation_base of 2 which results in 3 dilated convolutional layers for full coverage.



Here we only show the influence of inputs that affect the last value of the output. Likewise, only zero-padding entries necessary for the last output value are shown. Clearly, the last output value is dependent on the whole input coverage. Actually, given the hyperparameters, an input_length of up to 15 could be used while maintaining full receptive field coverage. Generally speaking, every additional layer adds a value of *d*(k-1)* to the current receptive field width, where *d* is computed as *d=b**i*, with *i* representing the number of layers below our new layer. Consequently, the width of the receptive field *w* of a TCN with exponential dilation of base *b*, kernel size *k* and number of layers *n* is given by

$$w = 1 + \sum_{i=0}^{n-1}(k-1)\cdot b^i = 1 + (k-1)\cdot\frac{b^n - 1}{b-1}$$

However, depending on the values *b* and *k*, this receptive field can have 'holes'. Consider the following network with a dilation_base of 3 and a kernel size of 2:



The receptive field does cover a range that is larger than the input size (namely 15). However, the receptive field has holes in it; that is there are entries in the input sequence that the output value is not dependent on (shown above in red). To solve this problem, we need to either increase the kernel size to 3, or decrease the dilation base to 2. Generally speaking, for a receptive field with no holes, the kernel size k has to be at least as big as the dilation base b.

Considering these observations, we can compute how many layers our network needs for full history coverage. Given a kernel size *k*, dilation base *b*, where $k \geq b$, and input length *l*, the following inequality must hold for full history coverage:

$$1 + (k-1) \cdot \frac{b^n - 1}{b - 1} \geq l$$

We can solve for *n* and get the minimum number of required layers as

$$n = \left\lceil log_b \left(\frac{(l-1) \cdot (b-1)}{(k-1)} + 1 \right) \right\rceil$$

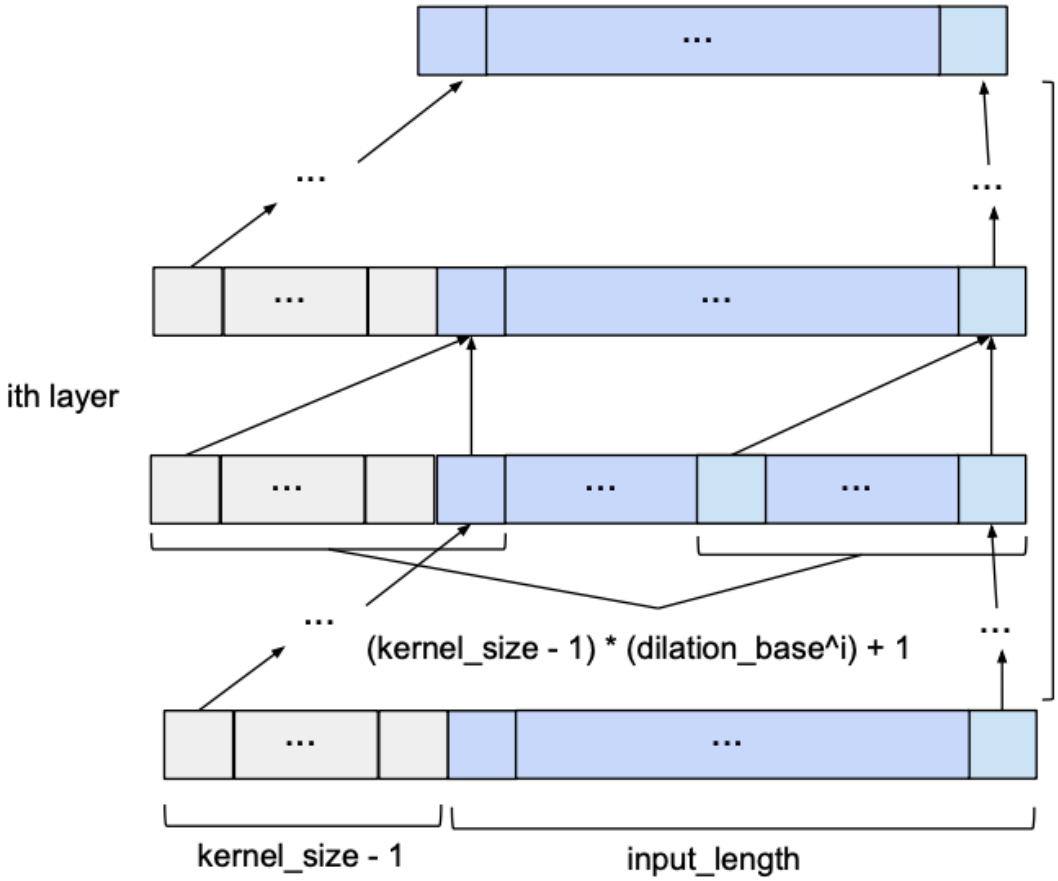
We can see that the number of layers is now logarithmic rather than linear in the length of the input. This constitutes a significant improvement, which can be achieved without sacrificing receptive field coverage.

Now the only thing left to specify is the number of zero-padding entries required at each layer. Given a dilation base *b*, a kernel size *k* and a number of *i* layers below our current layer, then the number of zero-padding entries *p* needed for the current layer are computed as follows:

$$p = b^i \cdot (k-1)$$

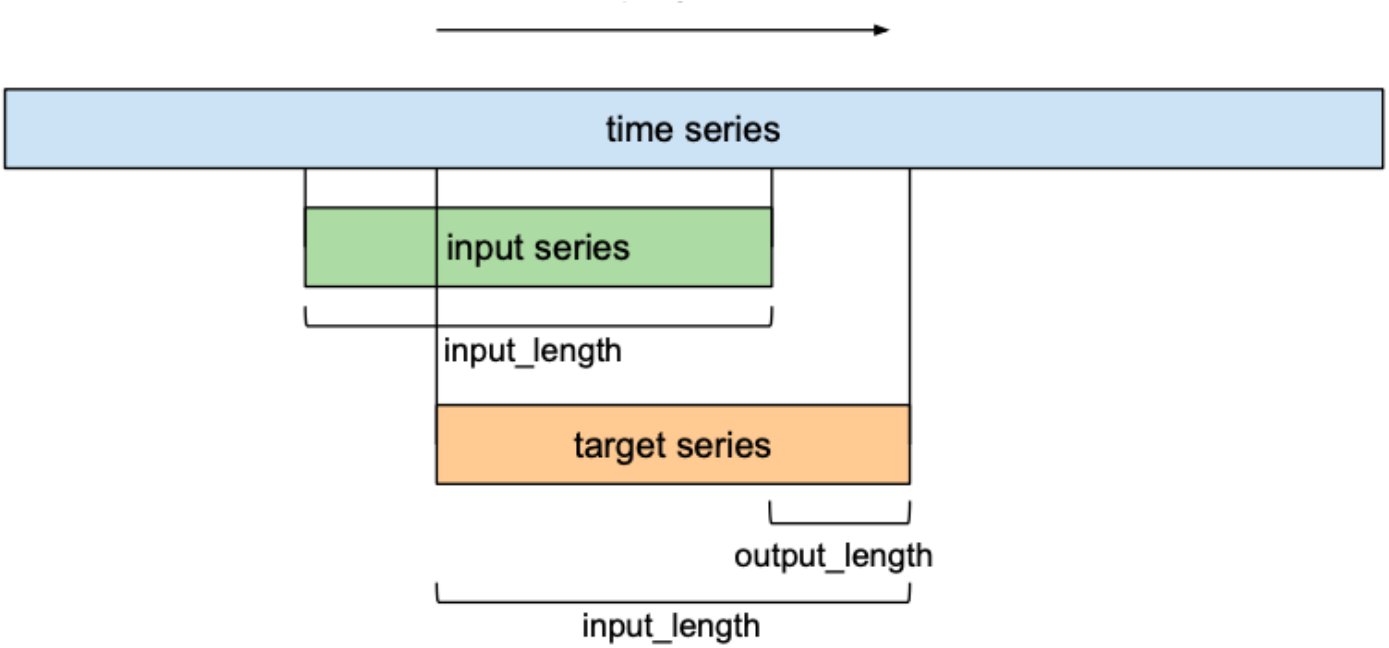
Basic TCN Overview

Given input_length, kernel_size, dilation_base and the minimum number of layers required for full history coverage, the basic TCN network would look something like this:



Forecasting

So far we have only talked about the 'input sequence' and the 'output sequence' without getting into how they relate to each other. In the context of forecasting, we want to predict the next entries of a time series into the future. To train our TCN network to do forecasting, the training set will consist of (input sequence, target sequence)-pairs of equally-sized subsequences of the given time series. A target series will be a series that is shifted forward in relation to its respective input series by a certain number output_length. This means that a target series of length input_lengthcontains the last (input_length – output_length) elements of its respective input sequence as first elements, and the output_length elements that come after the last entry of the input series as its final elements. In the context of forecasting, this means that the maximum forecasting horizon that can be predicted with such a model is equal to output_length. Using a sliding window approach, many overlapping pairs of input and target sequences can be created out of one time series.



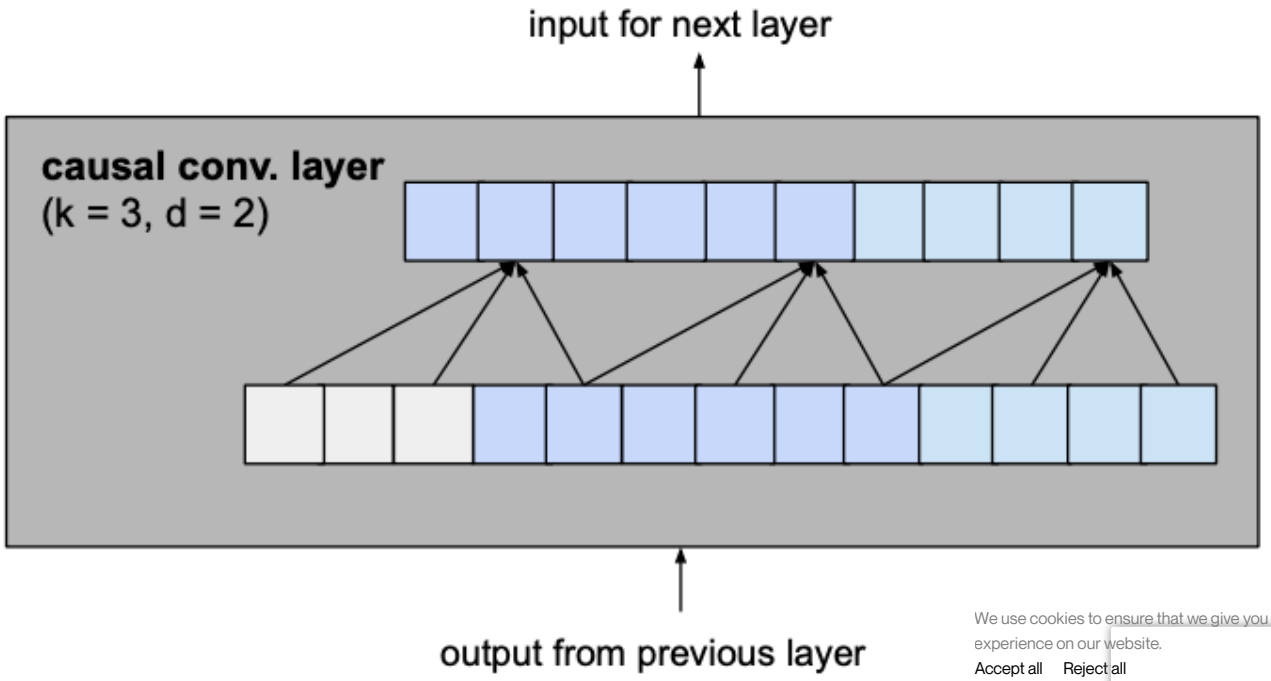
Improvements to the Model

S. Bai et al. (*) suggest a few additions to the basic TCN architecture for improved performance which will be discussed in this section, namely residual connections, regularization and activation functions.

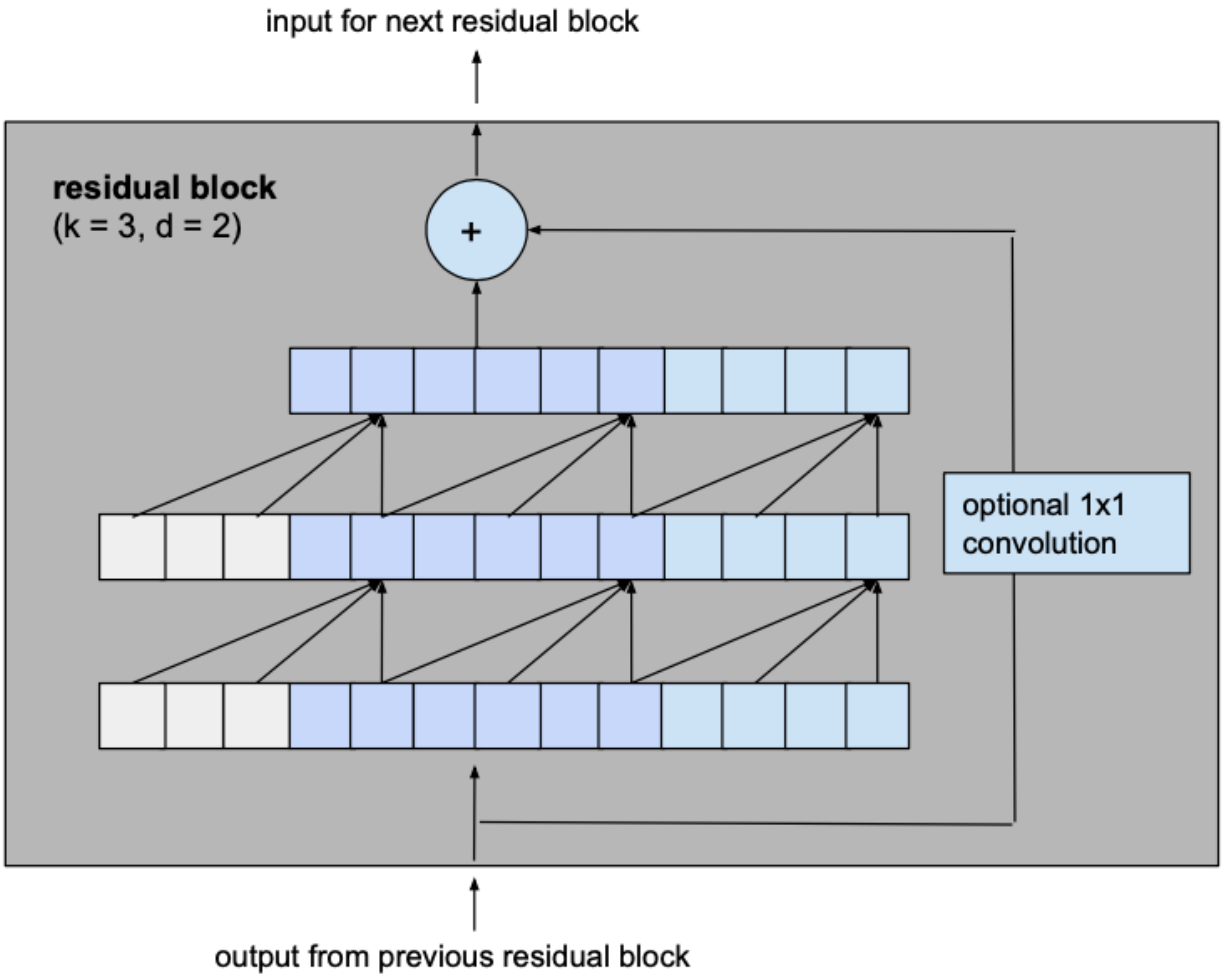
Residual Blocks

The biggest modification we make to the previously introduced basic model is to change the fundamental building block of the model from a simple 1D causal convolutional layer to a residual block which consists of 2 layers with the same dilation factor and a residual connection.

Let's consider a layer with a dilation factor d of 2 and kernel size k of 3 from the basic model to see how this translates into a residual block of the improved model.



becomes



The output of the two convolutional layers will be added to the input of the residual block to produce the input for the next block. For all inner blocks of the network, i.e. all but the first and the last one, the input and output channel width is the same, namely num_filters. Since the first convolutional layer of the first residual block and the second convolutional layer of the last residual block may have different input and output channel widths, the width of the residual tensor might have to be adjusted, which is done using a 1×1 convolution.

This change affects the calculus for the minimum number of required layers for full coverage. Now we have to think about how many residual blocks are necessary to achieve a full receptive field coverage. Adding a residual block to a TCN adds twice as much receptive field width than when adding a basic causal layer, since it includes 2 such layers. So the total size of the receptive field r of a TCN with dilation base b, kernel size k with k ≥ b and number of residual blocks n can be computed as

$$r = 1 + \sum_{i=0}^{n-1} 2 \cdot (k-1) \cdot b^i = 1 + 2 \cdot (k-1) \cdot \frac{b^n - 1}{b-1}$$

which leads to a minimum number of residual blocks n for full history coverage of input

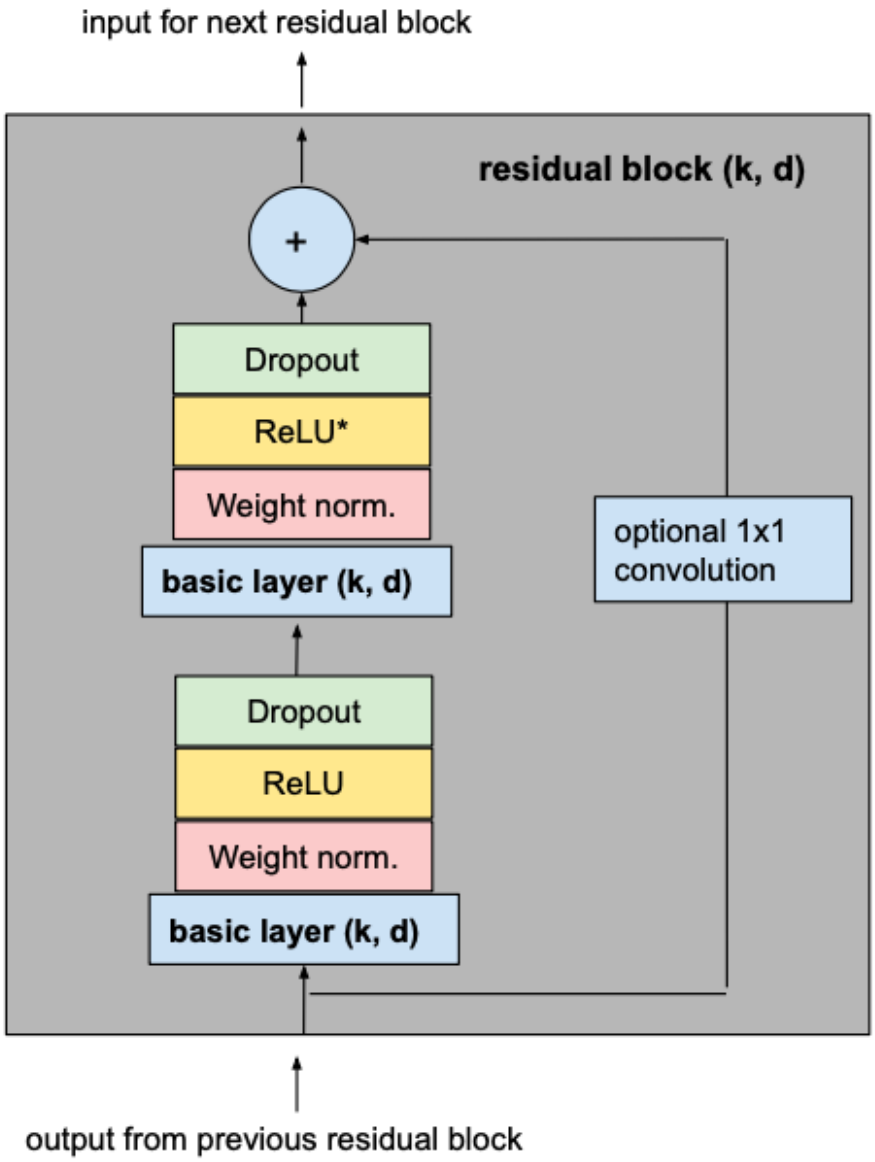
$$n = \left\lceil log_b \left(\frac{(l-1) \cdot (b-1)}{(k-1) \cdot 2} + 1 \right) \right\rceil$$

Activation, Normalization, Regularization

To make our TCN more than just an overly complex linear regression model, activation functions need to be added on top of the convolutional layers to introduce non-linearities. ReLU activations are added to the residual blocks after both convolutional layers.

To normalize the input of hidden layers (which counteracts the exploding gradient problem among other things), weight normalization is applied to every convolutional layer.

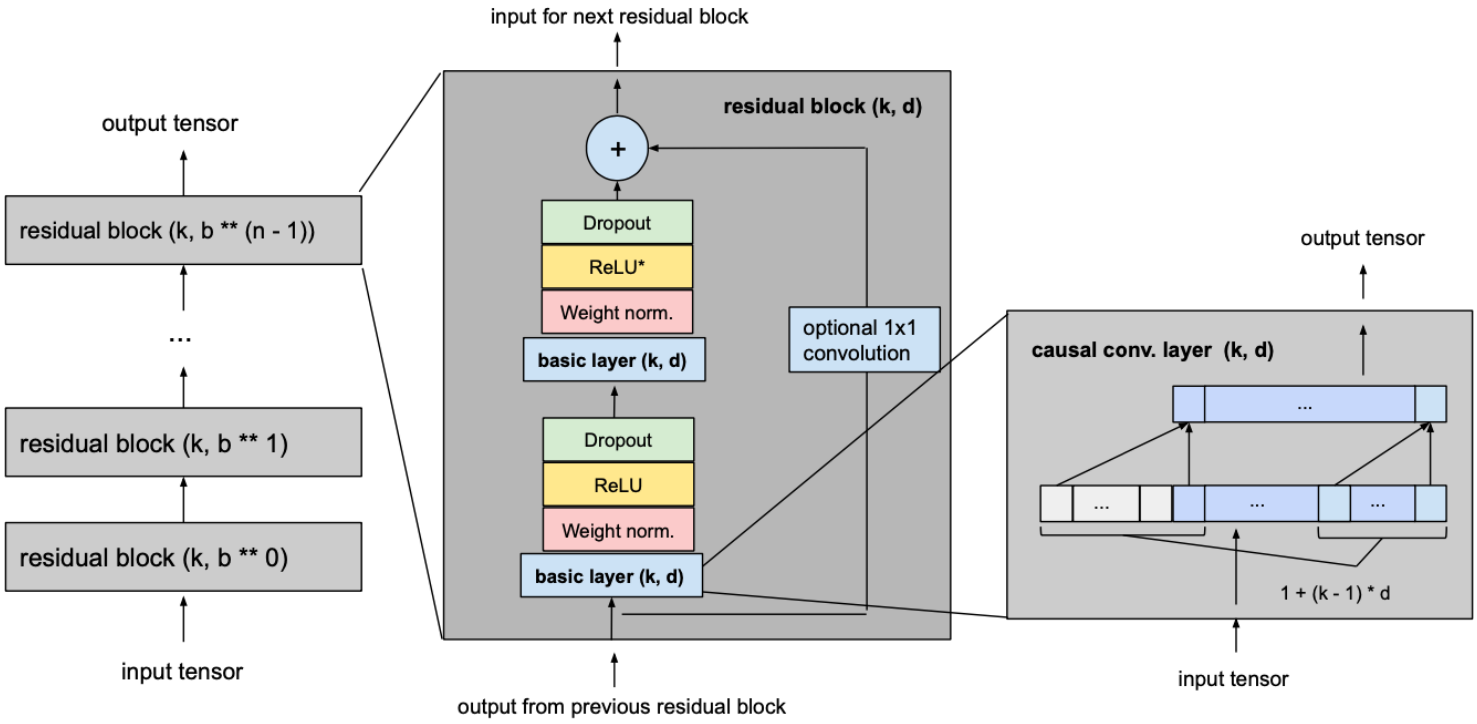
In order to prevent overfitting, regularization is introduced via dropout after every convolutional layer in every residual block. The following figure shows the final residual block.



The asterisk in the second ReLU unit indicates that it is present in every layer but the la
we want our final output to be able to take on negative values as well (this differs from tl
outlined in the paper).

Final Model

The following picture shows our final TCN model with *l* equal to input_length, *k* equal to kernel_size, *b* equal to dilation_base, $k \geq b$ and with a minimum number of residual blocks for full history coverage *n*,where *n* can be computed from the other values as explained above.

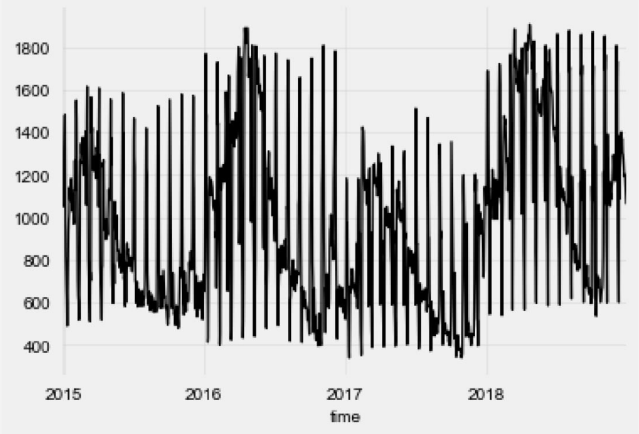


Example

Let's take a look at an example of how we can use the TCN architecture to forecast a time series using the *Darts* library.

First, we need a time series to train and evaluate our model on. For this, we use a Kaggle dataset containing hourly energy production data from Spain. More specifically, we choose to predict the 'run-of-river hydroelectricity' production. Furthermore, to make the problem less computation-intensive, we average the energy production over each day to get a daily time series.

```
from darts import TimeSeries
from darts.dataprocessing.transformers import
MissingValuesFiller
import pandas as pd

df = pd.read_csv('energy_dataset.csv', delimiter=",")
df['time'] = pd.to_datetime(df['time'], utc=True)
df['time']= df.time.dt.tz_localize(None)

df_day_avg =
df.groupby(df['time'].astype(str).str.split(" ").str[0]).mean().reset_index()

value_filler = MissingValuesFiller()
series =
value_filler.transform(TimeSeries.from_dataframe(df_day_avg, 'time', ['generation hydro run-of-ri

series.plot()
```



We can see that, in addition to a yearly seasonality, there are regular 'spikes' in energy production that come in monthly intervals. Since the TCN model supports multiple input channels, we can add additional time series components to the current time series that encode the current day of the month. This can help our TCN model to converge faster.

```
series = series.add_datetime_attribute('day', one_hot=True)
```

Now we split our data into train and validation components and perform standardization.

```
from darts.dataprocessing.transformers import Scaler

train, val =
series.split_after(pd.Timestamp('20170901'))

scaler = Scaler()
train_transformed = scaler.fit_transform(train)
val_transformed = scaler.transform(val)
series_transformed = scaler.transform(series)
```

Now it's time to create and train our TCN model. Note that all bold variable names that have appeared in the above description of the architecture can be used as arguments for the constructor of the *Darts* TCN implementation. The output_length parameter is set to 7 since we want to perform weekly forecasts. When training the model, we specify as target_series only the first component of our training series, since we do not want to predict the helper time series we added earlier. We tried out a few different hyperparameter combinations, but most of the values were chosen quite arbitrarily.

```
from darts.models import TCNModel

model = TCNModel(
    input_size=train.width,
    n_epochs=20,
    input_length=365,
    output_length=7,
    dropout=0,
    dilation_base=2,
    weight_norm=True,
    kernel_size=7,
    num_filters=4,
    random_state=0
)

model.fit(
    training_series=train_transformed,
    target_series=train_transformed['0'],
    val_training_series=val_transformed,
    val_target_series=val_transformed['0'],
verbose=True
)
```

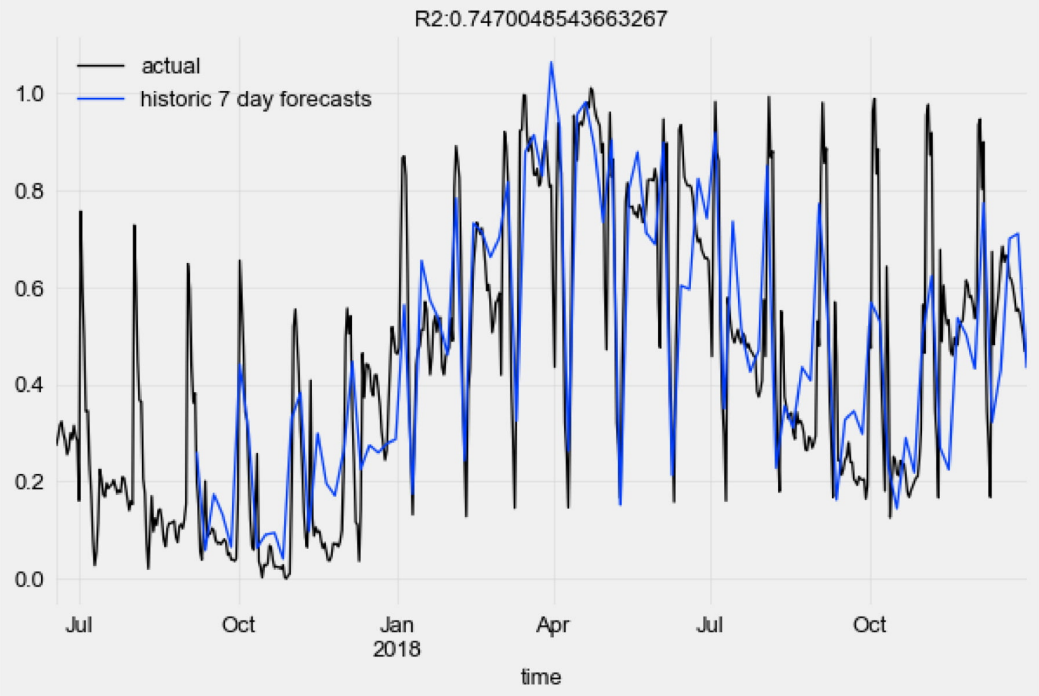
To evaluate our model, we want to test its performance using a 7-day forecasting horiz

different points in time in the validation set. For this, we use the historic backtesting functionality from *Darts*. Note that the model is provided new input data for every prediciton, but it is never retrained. To save time, we set the stride to 5.

```
pred_series = model.backtest(
    series_transformed,
    target_series=series_transformed['0'],
    start=pd.Timestamp('20170901'),
    forecast_horizon=7,
    stride=5,
    retrain=False,
    verbose=True,
    use_full_output_length=True
)
```

Let's visualize the historic forecast predictions of our TCN model against the ground truth data points and compute the R2 score.

```
from darts.metrics import r2_score
import matplotlib.pyplot as plt

series_transformed[900:]['0'].plot(label='actual')
pred_series.plot(label=('historic 7 day forecasts'))
r2_score_value = r2_score(series_transformed['0'], pred_series)

plt.title('R2:' + str(r2_score_value))
plt.legend()
```



For more details and additional examples, please check out our TCN examples notebook on GitHub.

Conclusion

Deep learning in sequence modeling is, for the most part, still widely associated with re
network architectures. But research has shown that these types of models can be outp
many tasks by a TCN, both in terms of predictive performance and efficiency. In this art
explored how this promising model can be understood in terms of simple building blocl

convolutional layers, dilations and residual connections, and how they all fit together. Furthermore, we successfully applied the current *Darts* implementation of the TCN architecture to predict a real-world time series.

Thanks to Julien Herzen.

Sequence Classification Using 1-D Convolutions

Open in MATLAB Online

Copy Command

This example shows how to classify sequence data using a 1-D convolutional neural network.

To train a deep neural network to classify sequence data, you can use a 1-D convolutional neural network. A 1-D convolutional layer learns features by applying sliding convolutional filters to 1-D input. Using 1-D convolutional layers can be faster than using recurrent layers because convolutional layers can process the input with a single operation. By contrast, recurrent layers must iterate over the time steps of the input. However, depending on the network architecture and filter sizes, 1-D convolutional layers might not perform as well as recurrent layers, which can learn long-term dependencies between time steps.

Load Sequence Data

Load the example data from `WaveformData.mat`. The data is a `numObservations`-by-1 cell array of sequences, where `numObservations` is the number of sequences. Each sequence is a `numTimeSteps`-by-`numChannels` numeric array, where `numTimeSteps` is the number of time steps of the sequence and `numChannels` is the number of channels of the sequence.

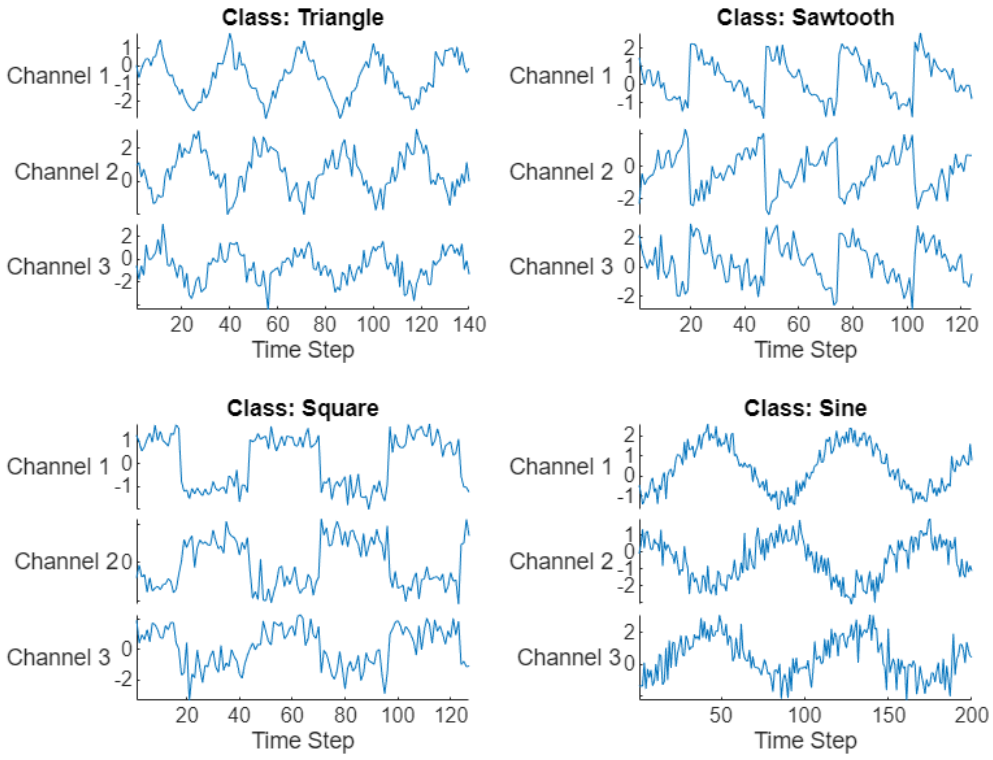
```
load WaveformData
```

Get

Visualize some of the sequences in a plot.

```
numChannels = size(data{1},2);

idx = [3 4 5 12];
figure
tiledlayout(2,2)
for i = 1:4
    nexttile
    stackedplot(data{idx(i)},DisplayLabels="Channel "+string(1:numChannels))

    xlabel("Time Step")
    title("Class: " + string(labels(idx(i))))
end
```

Get



Set aside data for validation and testing. Partition the data into a training set containing 80% of the data, a validation set containing 10% of the data, and a test set containing the remaining 10% of the data. To partition the data, use the `trainingPartitions` function, attached to this example as a supporting file. To access this file, open the example as a live script.

```
numObservations = numel(data);
[idxTrain,idxValidation,idxTest] = trainingPartitions(numObservations, [0.8 0.1 0.1]);
XTrain = data(idxTrain);
TTrain = labels(idxTrain);

XValidation = data(idxValidation);
TValidation = labels(idxValidation);

XTest = data(idxTest);
TTest = labels(idxTest);
```

Define 1-D Convolutional Network Architecture

Define the 1-D convolutional neural network architecture.

- Specify the input size as the number of channels of the input data.
- Specify two blocks of 1-D convolution, ReLU, and layer normalization layers, where the convolutional layer has a filter size of 5. Specify 32 and 64 filters for the first and second convolutional layers, respectively. For both convolutional layers, left-pad the inputs such that the outputs have the same length (causal padding).
- To reduce the output of the convolutional layers to a single vector, use a 1-D global average pooling layer.
- To map the output to a vector of probabilities, specify a fully connected layer with an output size matching the number of classes followed by a softmax layer.

You can also build this network using the Deep Network Designer app. On the Deep Network Designer Start Page, in the **Sequence-to-Label Classification Networks (Untrained)** section, click **1-D CNN**.

```
filterSize = 5;
numFilters = 32;

classNames = categories(TTrain);
```

```
numClasses = numel(classNames);

layers = [ ...
    sequenceInputLayer(numChannels)
    convolution1dLayer(filterSize,numFilters,Padding="causal")
    reluLayer
    layerNormalizationLayer
    convolution1dLayer(filterSize,2*numFilters,Padding="causal")
    reluLayer
    layerNormalizationLayer
    globalAveragePooling1dLayer
    fullyConnectedLayer(numClasses)
    softmaxLayer];
```

Specify Training Options

Specify the training options. Choosing among the options requires empirical analysis. To explore different training option configurations by running experiments, you can use the Experiment Manager app.

- Train for 60 epochs using the Adam optimizer with a learn rate of 0.01.
- Left-pad the sequences.
- Validate the network using the validation data.
- Monitor the training progress in a plot and suppress the verbose output.

```
options = trainingOptions("adam", ...
    MaxEpochs=60, ...
    InitialLearnRate=0.01, ...
    SequencePaddingDirection="left", ...
    ValidationData={XValidation,TValidation}, ...
    Plots="training-progress", ...
    Metrics="accuracy", ...
    Verbose=false);
```

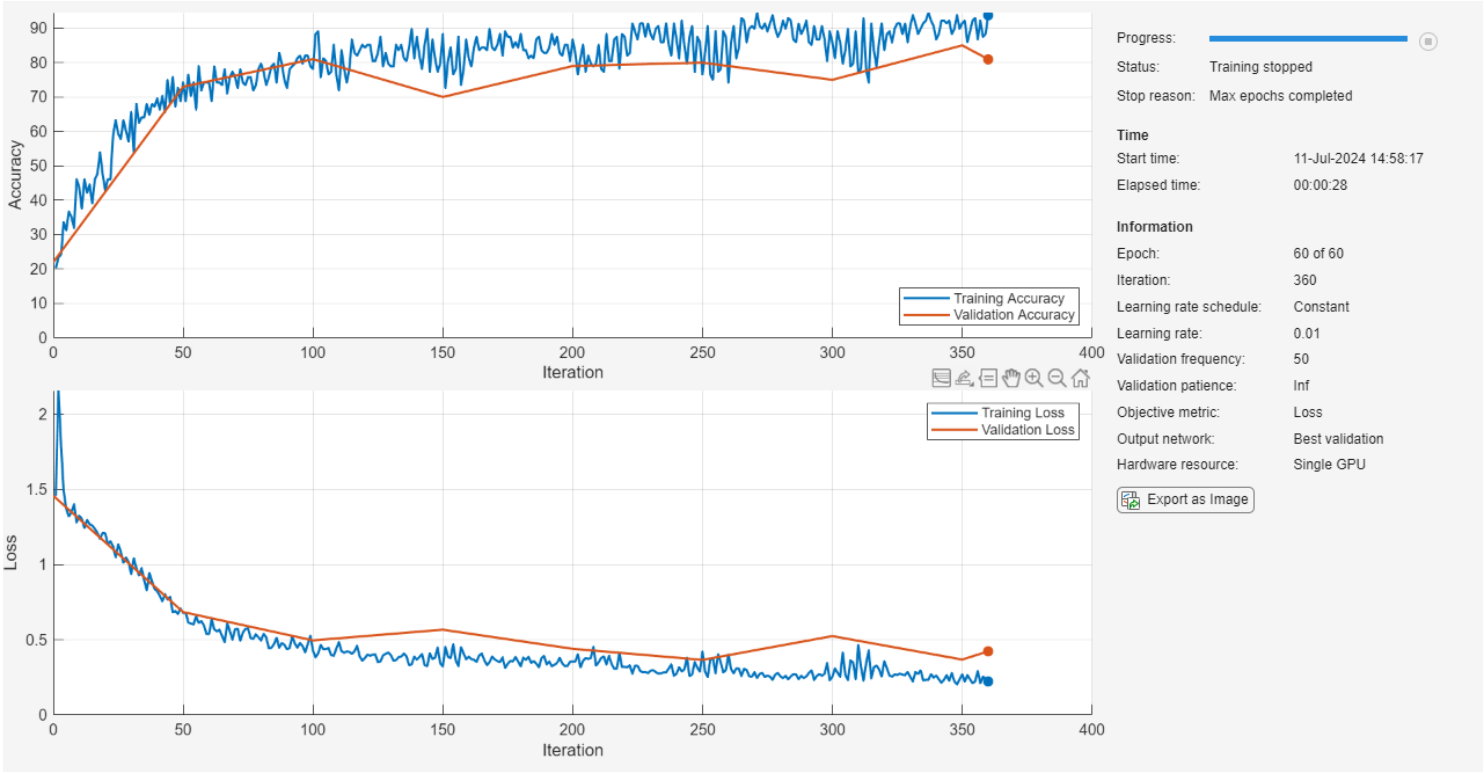
Get

Train Neural Network

Train the neural network using the `trainnet` function. For classification, use cross-entropy loss. By default, the `trainnet` function uses a GPU if one is available. Training on a GPU requires a Parallel Computing Toolbox™ license and a supported GPU device. For information on supported devices, see GPU Computing Requirements (Parallel Computing Toolbox). Otherwise, the `trainnet` function uses the CPU. To specify the execution environment, use the `ExecutionEnvironment` training option.

```
net = trainnet(XTrain,TTrain,layers,"crossentropy",options);
```

Get



Test Neural Network

Test the neural network using the `testnet` function and use the same arguments as used for training. For single-label classification, evaluate the accuracy. The accuracy is the percentage of correct predictions. By default, the `testnet` function uses a GPU if one is available. To select the execution environment manually, use the `ExecutionEnvironment` argument of the `testnet` function.

```
accuracy = testnet(net,XTest,TTest,"accuracy",SequencePaddingDirection="left")
```

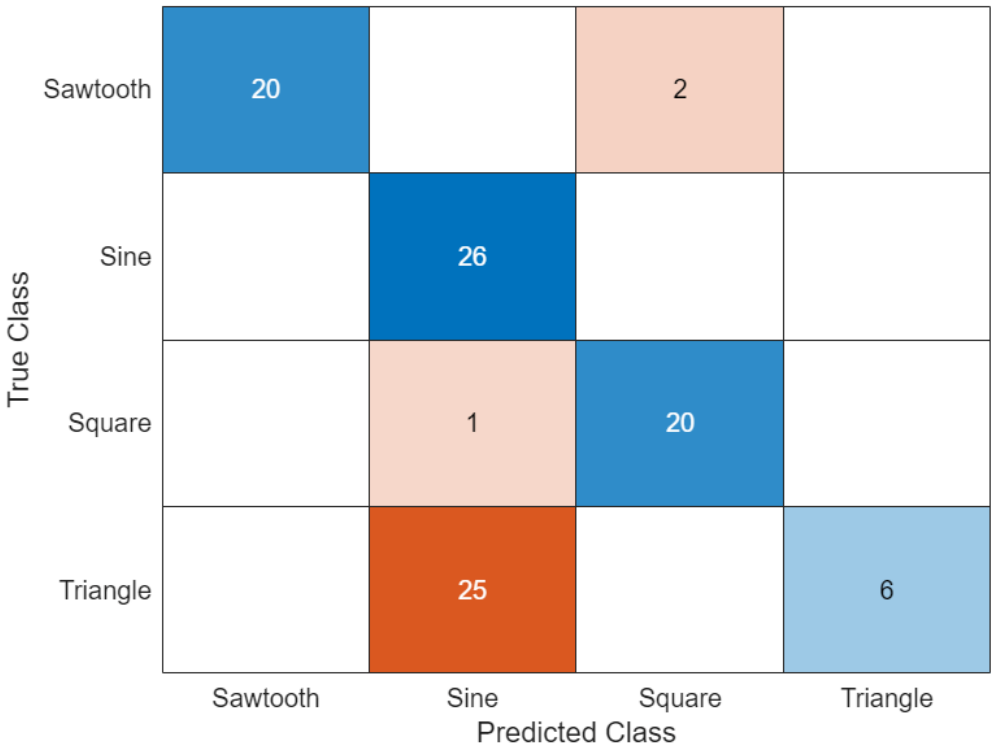
Get

```
accuracy =
72
```

Visualize the predictions in a confusion matrix. Make predictions using the `minibatchpredict` function and use the same sequence padding options as used for training. To make predictions with multiple observations, use the `minibatchpredict` function. To convert the prediction scores to labels, use the `scores2label` function. The `minibatchpredict` function automatically uses a GPU if one is available. To select the execution environment manually, use the `ExecutionEnvironment` argument of the `minibatchpredict` function.

```
scores = minibatchpredict(net,XTest,SequencePaddingDirection="left");
YTest = scores2label(scores, classNames);
figure
confusionchart(TTest,YTest)
```

Get



See Also

`convolution1dLayer` | `trainnet` | `trainingOptions` | `dlnetwork` | `sequenceInputLayer` | `maxPooling1dLayer` | `averagePooling1dLayer` | `globalMaxPooling1dLayer` | `globalAveragePooling1dLayer`

Topics

Sequence-to-Sequence Classification Using 1-D Convolutions

Sequence Classification Using Deep Learning

Train Sequence Classification Network Using Data with Imbalanced Classes

Sequence-to-Sequence Classification Using Deep Learning

Sequence-to-Sequence Regression Using Deep Learning

Sequence-to-One Regression Using Deep Learning

Time Series Forecasting Using Deep Learning

Interpret Deep Learning Time-Series Classifications Using Grad-CAM

Long Short-Term Memory Neural Networks

List of Deep Learning Layers

Policy Networks

Stable Baselines3 provides policy networks for images (CnnPolicies), other type of input features (MlpPolicies) and multiple different inputs (MultiInputPolicies).

Warning

For A2C and PPO, continuous actions are clipped during training and testing (to avoid out of bound error). SAC, DDPG and TD3 squash the action, using a `tanh()` transformation, which handles bounds more correctly.

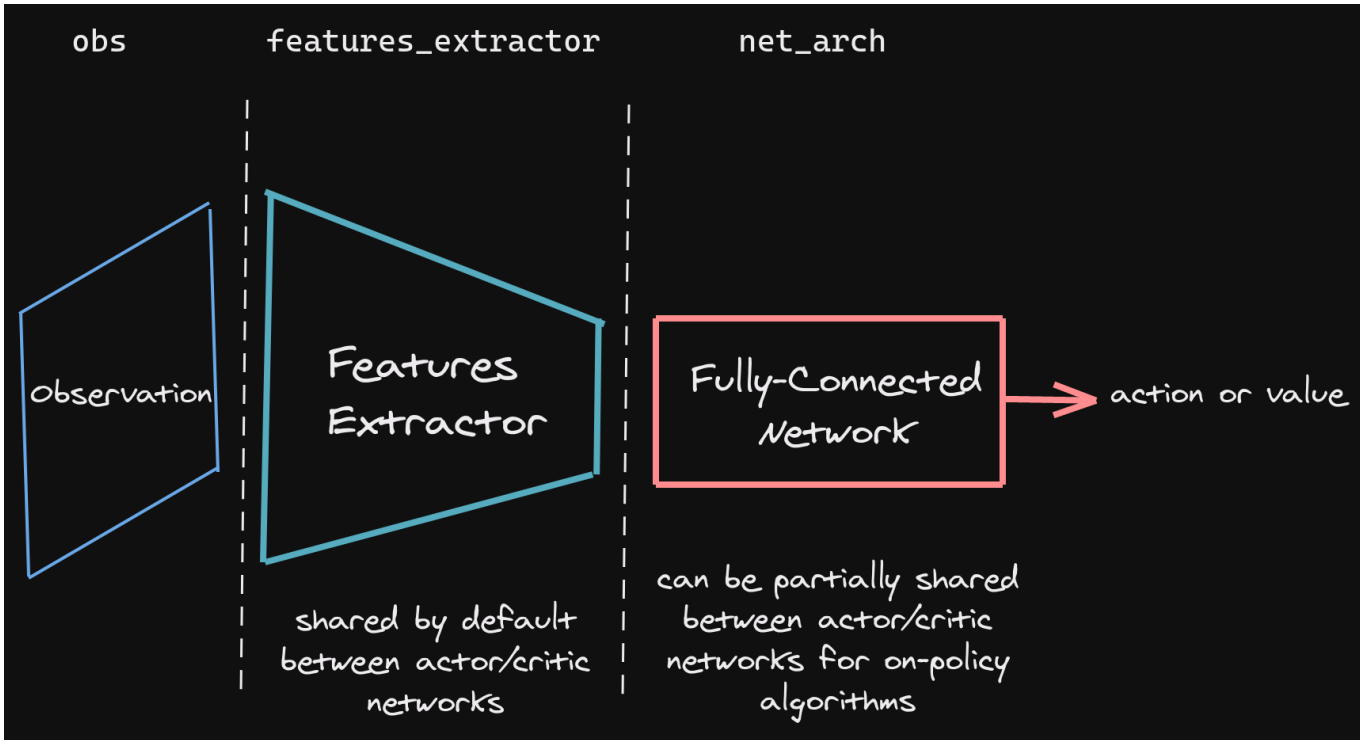
SB3 Policy

SB3 networks are separated into two main parts (see figure below):

- A features extractor (usually shared between actor and critic when applicable, to save computation) whose role is to extract features (i.e. convert to a feature vector) from high-dimensional observations, for instance, a CNN that extracts features from images. This is the `features_extractor_class` parameter. You can change the default parameters of that features extractor by passing a `features_extractor_kwargs` parameter.
- A (fully-connected) network that maps the features to actions/value. Its architecture is controlled by the `net_arch` parameter.

Note

All observations are first pre-processed (e.g. images are normalized, discrete obs are converted to one-hot vectors, ...) before being fed to the features extractor. In the case of vector observations, the features extractor is just a `Flatten` layer.

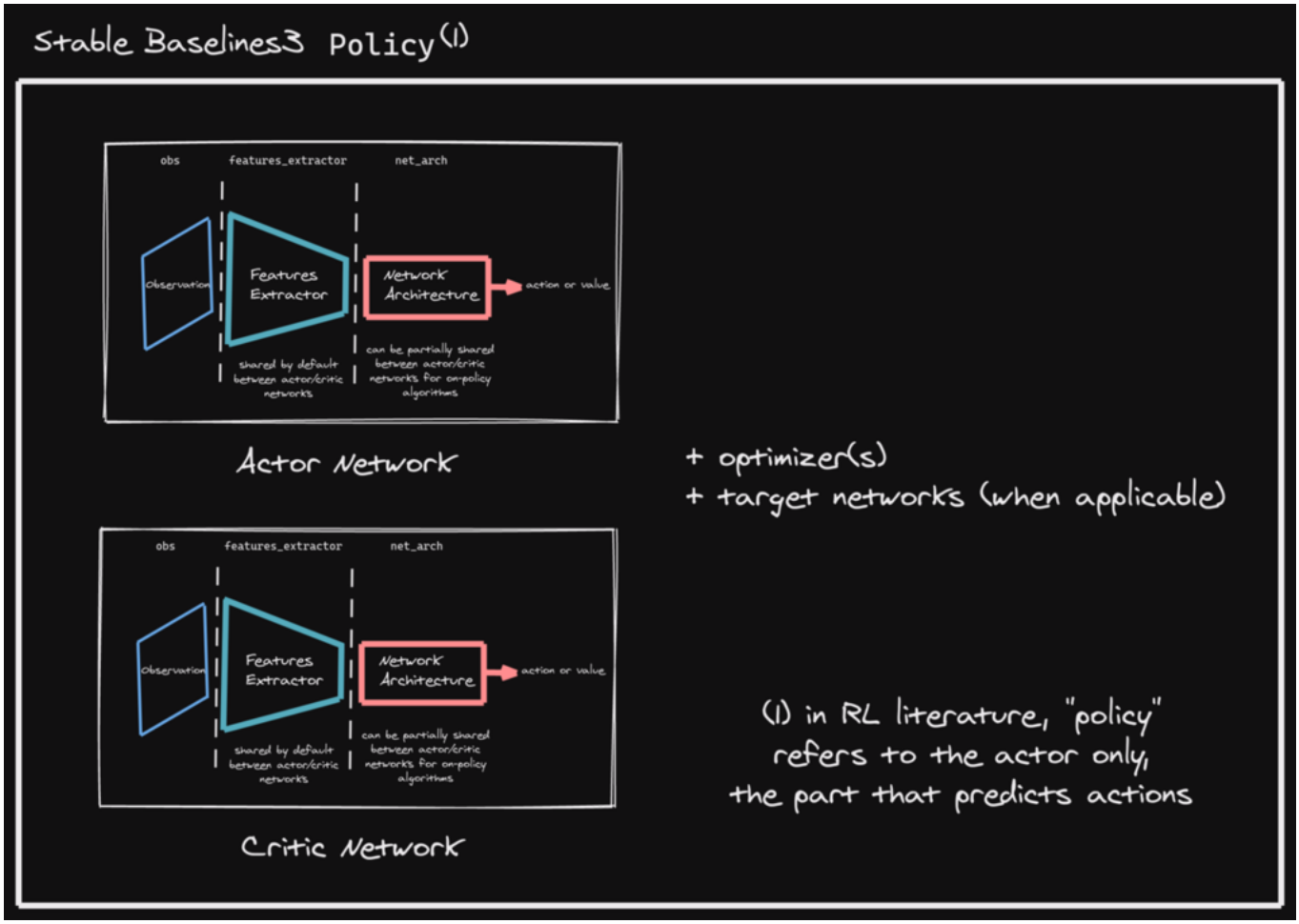


SB3 policies are usually composed of several networks (actor/critic networks + target networks when applicable) together with the associated optimizers.

Each of these network have a features extractor followed by a fully-connected network.

Note

When we refer to “policy” in Stable-Baselines3, this is usually an abuse of language compared to RL terminology. In SB3, “policy” refers to the class that handles all the networks useful for training, so not only the network used to predict actions (the “learned controller”).



Default Network Architecture

The default network architecture used by SB3 depends on the algorithm and the observation space. You can visualize the architecture by printing `model.policy` (see issue #329).

For 1D observation space, a 2 layers fully connected net is used with:

- 64 units (per layer) for PPO/A2C/DQN
- 256 units for SAC
- [400, 300] units for TD3/DDPG (values are taken from the original TD3 paper)

For image observation spaces, the “Nature CNN” (see code for more details) is used for feature extraction, and SAC/TD3 also keeps the same fully connected network after it. The other algorithms only have a linear layer after the CNN. The CNN is shared between actor and critic for A2C/PPO (on-policy algorithms) to reduce computation. Off-policy algorithms (TD3, DDPG, SAC, ...) have separate feature extractors: one for the actor and one for the critic, since the best performance is obtained with this configuration.

For mixed observations (dictionary observations), the two architectures from abov CNN for images and then two layers fully-connected network (with a smaller outp CNN).

Custom Network Architecture

One way of customising the policy network architecture is to pass arguments when creating the model, using `policy_kwargs` parameter:

Note

An extra linear layer will be added on top of the layers specified in `net_arch`, in order to have the right output dimensions and activation functions (e.g. Softmax for discrete actions).

In the following example, as CartPole's action space has a dimension of 2, the final dimensions of the `net_arch`'s layers will be:

```
        obs
        <4>
   /             \
 <32>           <32>
  |              |
 <32>           <32>
  |              |
 <2>            <1>
action         value
```

```python
import gymnasium as gym
import torch as th

from stable_baselines3 import PPO

# Custom actor (pi) and value function (vf) networks
# of two layers of size 32 each with Relu activation function
# Note: an extra linear layer will be added on top of the pi and the vf nets, respectively
policy_kwargs = dict(activation_fn=th.nn.ReLU,
                     net_arch=dict(pi=[32, 32], vf=[32, 32]))
# Create the agent
model = PPO("MlpPolicy", "CartPole-v1", policy_kwargs=policy_kwargs, verbose=1)
# Retrieve the environment
env = model.get_env()
# Train the agent
model.learn(total_timesteps=20_000)
# Save the agent
model.save("ppo_cartpole")

del model
# the policy_kwargs are automatically loaded
model = PPO.load("ppo_cartpole", env=env)
```


Custom Feature Extractor

If you want to have a custom features extractor (e.g. custom CNN when using images), you can define class that derives from `BaseFeaturesExtractor` and then pass it to the model when training.

Note

For on-policy algorithms, the features extractor is shared by default between the actor and the critic to save computation (when applicable). However, this can be changed setting `share_features_extractor=False` in the `policy_kwargs` (both for on-policy and off-policy algorithms).

```
import torch as th
import torch.nn as nn
from gymnasium import spaces

from stable_baselines3 import PPO
from stable_baselines3.common.torch_layers import BaseFeaturesExtractor


class CustomCNN(BaseFeaturesExtractor):
    """
    :param observation_space: (gym.Space)
    :param features_dim: (int) Number of features extracted.
        This corresponds to the number of unit for the last layer.
    """

    def __init__(self, observation_space: spaces.Box, features_dim: int = 256):
        super().__init__(observation_space, features_dim)
        # We assume CxHxW images (channels first)
        # Re-ordering will be done by pre-preprocessing or wrapper
        n_input_channels = observation_space.shape[0]
        self.cnn = nn.Sequential(
            nn.Conv2d(n_input_channels, 32, kernel_size=8, stride=4, padding=0),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=0),
            nn.ReLU(),
            nn.Flatten(),
        )

        # Compute shape by doing one forward pass
        with th.no_grad():
            n_flatten = self.cnn(
                th.as_tensor(observation_space.sample()[None]).float()
            ).shape[1]

        self.linear = nn.Sequential(nn.Linear(n_flatten, features_dim), nn.ReLU())

    def forward(self, observations: th.Tensor) -> th.Tensor:
        return self.linear(self.cnn(observations))

policy_kwargs = dict(
    features_extractor_class=CustomCNN,
    features_extractor_kwargs=dict(features_dim=128),
)
model = PPO("CnnPolicy", "BreakoutNoFrameskip-v4", policy_kwargs=policy_kwargs
model.learn(1000)
```

Multiple Inputs and Dictionary Observations

Stable Baselines3 supports handling of multiple inputs by using `Dict` Gym space. This can be done using `MultiInputPolicy`, which by default uses the `CombinedExtractor` features extractor to turn multiple inputs into a single vector, handled by the `net_arch` network.

By default, `CombinedExtractor` processes multiple inputs as follows:

1. If input is an image (automatically detected, see `common.preprocessing.is_image_space`), process image with Nature Atari CNN network and output a latent vector of size `256`.
2. If input is not an image, flatten it (no layers).
3. Concatenate all previous vectors into one long vector and pass it to policy.

Much like above, you can define custom features extractors. The following example assumes the environment has two keys in the observation space dictionary: “image” is a (1,H,W) image (channel first), and “vector” is a (D,) dimensional vector. We process “image” with a simple downsampling and “vector” with a single linear layer.

```python
import gymnasium as gym
import torch as th
from torch import nn

from stable_baselines3.common.torch_layers import BaseFeaturesExtractor

class CustomCombinedExtractor(BaseFeaturesExtractor):
    def __init__(self, observation_space: gym.spaces.Dict):
        # We do not know features-dim here before going over all the items,
        # so put something dummy for now. PyTorch requires calling
        # nn.Module.__init__ before adding modules
        super().__init__(observation_space, features_dim=1)

        extractors = {}

        total_concat_size = 0
        # We need to know size of the output of this extractor,
        # so go over all the spaces and compute output feature sizes
        for key, subspace in observation_space.spaces.items():
            if key == "image":
                # We will just downsample one channel of the image by 4x4 and flatten.
                # Assume the image is single-channel (subspace.shape[0] == 0)
                extractors[key] = nn.Sequential(nn.MaxPool2d(4), nn.Flatten())
                total_concat_size += subspace.shape[1] // 4 * subspace.shape[2] // 4
            elif key == "vector":
                # Run through a simple MLP
                extractors[key] = nn.Linear(subspace.shape[0], 16)
                total_concat_size += 16

        self.extractors = nn.ModuleDict(extractors)

        # Update the features dim manually
        self._features_dim = total_concat_size

    def forward(self, observations) -> th.Tensor:
        encoded_tensor_list = []

        # self.extractors contain nn.Modules that do all the processing.
        for key, extractor in self.extractors.items():
            encoded_tensor_list.append(extractor(observations[key]))
        # Return a (B, self._features_dim) PyTorch tensor, where B is batch dimension.
        return th.cat(encoded_tensor_list, dim=1)
```

On-Policy Algorithms

Custom Networks

If you need a network architecture that is different for the actor and the critic when using `PPO`, `A2C` or `TRPO`, you can pass a dictionary of the following structure: `dict(pi=[<actor network architecture>], vf=[<critic network architecture>])`.

For example, if you want a different architecture for the actor (aka `pi`) and the cri function aka `vf`) networks, then you can specify `net_arch=dict(pi=[32, 32], vf=[64, 64])`.

Otherwise, to have actor and critic that share the same network architecture, you only need to specify `net_arch=[128, 128]` (here, two hidden layers of 128 units each, this is equivalent to `net_arch=dict(pi=[128, 128], vf=[128, 128])`).

If shared layers are needed, you need to implement a custom policy network (see advanced example below).

Examples

Same architecture for actor and critic with two layers of size 128: `net_arch=[128, 128]`

```
        obs
   /            \
 <128>          <128>
  |              |
 <128>          <128>
  |              |
action         value
```

Different architectures for actor and critic: `net_arch=dict(pi=[32, 32], vf=[64, 64])`

```
        obs
   /            \
 <32>           <64>
  |              |
 <32>           <64>
  |              |
action         value
```

Advanced Example

If your task requires even more granular control over the policy/value architecture, you can redefine the policy directly:

```python
from typing import Callable, Dict, List, Optional, Tuple, Type, Union

from gymnasium import spaces
import torch as th
from torch import nn

from stable_baselines3 import PPO
from stable_baselines3.common.policies import ActorCriticPolicy


class CustomNetwork(nn.Module):
    """
    Custom network for policy and value function.
    It receives as input the features extracted by the features extractor.

    :param feature_dim: dimension of the features extracted with the features_extractor (e.g.
features from a CNN)
    :param last_layer_dim_pi: (int) number of units for the last layer of the policy network
    :param last_layer_dim_vf: (int) number of units for the last layer of the value network
    """

    def __init__(
        self,
        feature_dim: int,
        last_layer_dim_pi: int = 64,
        last_layer_dim_vf: int = 64,
    ):
        super().__init__()

        # IMPORTANT:
        # Save output dimensions, used to create the distributions
        self.latent_dim_pi = last_layer_dim_pi
        self.latent_dim_vf = last_layer_dim_vf

        # Policy network
        self.policy_net = nn.Sequential(
            nn.Linear(feature_dim, last_layer_dim_pi), nn.ReLU()
        )
        # Value network
        self.value_net = nn.Sequential(
            nn.Linear(feature_dim, last_layer_dim_vf), nn.ReLU()
        )

    def forward(self, features: th.Tensor) -> Tuple[th.Tensor, th.Tensor]:
        """
        :return: (th.Tensor, th.Tensor) latent_policy, latent_value of the specified network.
            If all layers are shared, then ``latent_policy == latent_value``
        """
        return self.forward_actor(features), self.forward_critic(features)

    def forward_actor(self, features: th.Tensor) -> th.Tensor:
        return self.policy_net(features)

    def forward_critic(self, features: th.Tensor) -> th.Tensor:
        return self.value_net(features)


class CustomActorCriticPolicy(ActorCriticPolicy):
    def __init__(
        self,
        observation_space: spaces.Space,
        action_space: spaces.Space,
```

```
        lr_schedule: Callable[[float], float],
        *args,
        **kwargs,
    ):
        # Disable orthogonal initialization
        kwargs["ortho_init"] = False
        super().__init__(
            observation_space,
            action_space,
            lr_schedule,
            # Pass remaining arguments to base class
            *args,
            **kwargs,
        )


    def _build_mlp_extractor(self) -> None:
        self.mlp_extractor = CustomNetwork(self.features_dim)


model = PPO(CustomActorCriticPolicy, "CartPole-v1", verbose=1)
model.learn(5000)
```

Off-Policy Algorithms

If you need a network architecture that is different for the actor and the critic when using `SAC`, `DDPG`, `TQC` or `TD3`, you can pass a dictionary of the following structure: `dict(pi=[<actor network architecture>], qf=[<critic network architecture>])`.

For example, if you want a different architecture for the actor (aka `pi`) and the critic (Q-function aka `qf`) networks, then you can specify `net_arch=dict(pi=[64, 64], qf=[400, 300])`.

Otherwise, to have actor and critic that share the same network architecture, you only need to specify `net_arch=[256, 256]` (here, two hidden layers of 256 units each).

Note

For advanced customization of off-policy algorithms policies, please take a look at the code. A good understanding of the algorithm used is required, see discussion in issue #425

```
from stable_baselines3 import SAC

# Custom actor architecture with two layers of 64 units each
# Custom critic architecture with two layers of 400 and 300 units
policy_kwargs = dict(net_arch=dict(pi=[64, 64], qf=[400, 300]))
# Create the agent
model = SAC("MlpPolicy", "Pendulum-v1", policy_kwargs=policy_kwargs, verbose=1
model.learn(5000)
```

Adapting 1D CNN Architectures for Reinforcement Learning in Variable-Length Quantum Gate Sequences

Key Points

- Research suggests that 1D CNN architectures from domains like quantum circuit prediction and sequence classification (e.g., text or DNA) can be effectively adapted for your RL setup, such as an A2C actor-critic network, by using the CNN as a feature extractor for the sequential input of quantum gate representations.
- A robust choice appears to be a multi-layer 1D CNN with small kernel sizes (e.g., 3) to capture local repeating patterns in gate instructions, increasing filters across layers (e.g., 256 per layer) for hierarchical feature learning, and global average pooling to handle variable sequence lengths without padding issues.
- Evidence leans toward shallow to moderate depths (2-4 conv layers) for practical performance, avoiding excessive complexity that could hinder learning in RL settings; this balances quality and speed, with adaptations showing high accuracy (e.g., $R^2 > 0.99$) in similar sequential tasks.
- Kernel sizes should be small (3-5) rather than your current 130, as larger kernels may miss fine-grained patterns in repeating gate structures; start with 150 input channels matching your IRS, and reduce channels progressively if needed for efficiency.
- Incorporate LayerNorm (as in your design) and PReLU for stability, plus dropout (e.g., 0.5) to prevent overfitting; for RL, branch the network after pooling into policy (softmax over actions) and value (linear) heads.

Recommended Architecture Adaptation

Based on proven designs for sequential data with repeating patterns, adapt a 1D CNN from quantum circuit prediction tasks. Input shape: (variable_seq_len, 150), where seq_len is the number of gates, and 150 matches your qubit + gate encoding. Use zero padding for convolutions to preserve length, and global average pooling to produce fixed-size features regardless of seq_len.

- **Conv Layers**: 4 layers, each with 256 filters, kernel size 3, stride 1, ReLU activation, zero padding.
- **Normalization and Activation**: Apply LayerNorm after each conv, followed by PReLU.
- **Pooling**: Global average pooling after the last conv to reduce to (256) features.
- **Dense Layers**: 4 layers with 512 units each, ReLU, then branch for RL: one head with softmax for policy π (N actions), another linear for value V.
- **Regularization**: Dropout 0.5 before dense layers.
- This setup has shown strong extrapolation in variable-length quantum sequences, achieving near-perfect predictions ($R^2 > 0.99$) up to 60 qubits. Adjust filters downward (e.g., to 128) if model size becomes a concern, but prioritize learning quality.

Implementation Tips

- Handle variable lengths via dynamic batching or masking in your RL framework (e.g., PyTorch); no need for fixed padding beyond conv operations.
- For optimization: Use ADAM with MSE or RL-specific losses; train on heterogeneous sequence lengths to improve generalization.
- Test kernel sizes 3-5 initially, as they align with local patterns in gate interactions; larger (e.g., 12) could work if patterns span more instructions, per DNA motif models.
- Expected balance: Moderate depth ensures reasonable inference speed (milliseconds per forward pass on GPU) while supporting high policy performance.

A multi-layer 1D CNN architecture, adapted from supervised quantum circuit prediction tasks, offers a robust and practical solution for processing your variable-length sequential input of quantum gate representations in an RL context like A2C. This design leverages small kernels to detect local repeating patterns in gate instructions, hierarchical layers for complex feature extraction, and pooling mechanisms to accommodate varying sequence lengths. It has demonstrated high accuracy and scalability in similar domains, with R^2 scores exceeding 0.99 for predictions on quantum circuits up to 60 qubits, even when trained on smaller sizes. The architecture can be integrated as a shared feature extractor for actor and critic networks, branching into policy and value outputs.

Input Representation and Handling Variable Lengths

Your input—a variable-length sequence of gate instructions, each encoded as a 150-dimensional vector (130 qubits + 20 for gates/parameters)—is treated as a 1D multivariate time series: shape (seq_len, 150), where seq_len varies with the number of gates. This mirrors time series or sequence data in other domains, such as accelerometer signals (e.g., 128 timesteps x 9 channels) or DNA bases (50 timesteps x 4 channels). To handle variability:

- Apply zero padding in conv layers to maintain spatial integrity without altering representations.
- Use global average pooling (or max-over-time pooling) after convolutions to reduce the temporal dimension to a fixed-size vector, independent of seq_len. This avoids the need for fixed padding or truncation, ensuring the model processes circuits of any length up to your max.
- In RL training (e.g., A2C), batch sequences dynamically or use masking to ignore padded parts if necessary.

This approach aligns with best practices for sequential data, where global pooling enables scalability and prevents information loss in variable inputs.

Core Convolutional Layers

Start with 4 convolutional layers to build hierarchical features, increasing complexity from local gate patterns (e.g., adjacent qubit interactions) to global circuit behaviors. Each layer uses:

- Filters: 256 (consistent across layers for simplicity; can increase to 512 in deeper variants for more capacity).
- Kernel size: 3 (small to focus on local repeating patterns, like gate adjacencies; avoids your large 130, which risks capturing irrelevant global noise).
- Stride: 1 (preserves resolution for detailed feature extraction).
- Padding: Zero (maintains output length close to input).
- Activation: ReLU (standard for non-linearity; pair with your PReLU for improved gradient flow in negative regions).

Follow each conv with LayerNorm to stabilize activations, as in your current design. This setup, drawn from quantum circuit models, extracts features effectively from angle sequences (analogous to your gate encodings) and extrapolates well to larger systems. Total conv parameters scale with channels (start at 150 input channels, no reduction needed unless efficiency demands it).

Pooling and Regularization

- After the final conv, apply global average pooling to aggregate features into a fixed 256-dimensional vector. This captures the most salient patterns across the sequence, similar to max-over-time in text models, and inherently supports variable lengths.
- Add dropout (rate 0.5) post-pooling to mitigate overfitting, especially in RL where data can be noisy from exploration.

Dense Layers and RL Adaptation

- Follow pooling with 4 dense layers of 512 units each, ReLU activation, to refine high-level features.
- For A2C/PPO: Branch after the last dense— one path to a softmax layer for policy π (over N actions), another to a linear layer for value V. This shared trunk reduces computation, as in standard RL policy nets.
- Total model size: Moderate (~1-2M parameters depending on dense scaling), balancing learning speed and precision without constraints.

Hyperparameter Recommendations

Based on proven designs:

- **Filters/Channels**: 128-256 per layer; start low and increase for capacity without bloating size.
- **Depth**: 2-4 conv layers; deeper (4) aids hierarchical learning in complex sequences like yours.
- **Kernel Sizes**: 3-5; use multiple parallel (e.g., branches with 3,4,5) if patterns vary in scale, as in TextCNN.
- **Strides/Padding**: Stride 1, zero padding for full coverage.
- **Activations/Normalization**: ReLU/PReLU with LayerNorm.
- **Optimization**: ADAM, batch size 32-50; for RL, integrate with policy gradients.

Performance and Balance

This architecture achieves high learning quality in similar tasks, with rapid convergence and $R^2 > 0.99$ on extrapolated sequences. In RL adaptations, expect stable policy improvement; for speed-precision balance, test on your data— shallower variants (2 layers) train faster but may sacrifice final accuracy.

Comparison Table of Similar Architectures

Architecture Source	Domain	Conv Layers	Filters per Layer	Kernel Sizes	Pooling	Dense Layers	Handles Var Len?	Key Notes
Quantum Circuit Prediction	Quantum sequences	4	256	3	Global Avg	4 (512 units)	Yes (global pooling)	Scalable to large N; $R^2 > 0.99$
TextCNN	Text sequences	1 (parallel branches)	100 per branch	3,4,5	Max-over-time	1-2	Yes (max pooling)	Classic for var-len; dropout 0.5
HAR (Multi-Layer)	Time series (accel)	2	64	3	Max (size 2)	2 (100+6 units)	Fixed windows, adaptable	~90% acc; multivariate channels
DNA Binding	DNA sequences	1	32	12	Max (size 4)	2 (16+1 units)	Yes (flattening)	99% acc; motif-focused kernels
Activity Recognition (GitHub)	Accel data	2	100, 160	10 (inferred)	Max (3) + Global Avg	1 (6 units)	Fixed (80 steps)	164k params; dropout 0.5

Alternative Adaptations

- **From TextCNN**: Use parallel conv branches with kernels 3-5 (100 filters each) for multi-scale pattern capture in gates; max pooling handles var len well, proven in NLP for repeating structures.
- **From HAR**: 2 conv layers (64 filters, kernel 3) for simpler multivariate seq; multi-head variant splits your 150 channels (e.g., qubits vs. gates) for specialized learning.
- **From DNA**: Single conv (32 filters, kernel 12) if patterns span ~12 instructions; quick for prototyping.
- Open-source variants (e.g., GitHub HAR) suggest maxpool 2-10 for redundancy reduction in repetitive data.

These designs prioritize similarity to your 1D repeating structure over unrelated domains, ensuring high likelihood of effective learning.

Key Citations

- https://arxiv.org/pdf/2402.04992.pdf
- https://arxiv.org/pdf/1408.5882.pdf
- https://machinelearningmastery.com/cnn-models-for-human-activity-recognition-time-series-classification/
- https://divingintogeneticsandgenomics.com/post/how-to-use-1d-convolutional-neural-network-conv1d-to-predict-dna-sequence-binding-to-protein/
- https://github.com/harryjdavies/Python1D_CNNs
- https://github.com/bharatm11/1D_CNN_Human_activity_recognition

Challenges and opportunities in the supervised learning of quantum circuit outputs

Simone Cantori[1,2*] and Sebastiano Pilati[1,2]

[1]School of Science and Technology, Physics Division, Università di Camerino, Via Madonna delle Carceri, Camerino (MC), 62032, Italy.
[2]Sezione di Perugia, INFN, Perugia, 06123, Italy.

*Corresponding author(s). E-mail(s): simone.cantori@unicam.it;
Contributing authors: sebastiano.pilati@unicam.it;

Abstract

Recently, deep neural networks have proven capable of predicting some output properties of relevant random quantum circuits, indicating a strategy to emulate quantum computers alternative to direct simulation methods such as, e.g., tensor-network methods. However, the reach of this alternative strategy is not yet clear. Here we investigate if and to what extent neural networks can learn to predict the output expectation values of circuits often employed in variational quantum algorithms, namely, circuits formed by layers of CNOT gates alternated with random single-qubit rotations. On the one hand, we find that the computational cost of supervised learning scales exponentially with the inter-layer variance of the random angles. This allows entering a regime where quantum computers can easily outperform classical neural networks. On the other hand, circuits featuring only inter-qubit angle variations are easily emulated. In fact, thanks to a suitable scalable design, neural networks accurately predict the output of larger and deeper circuits than those used for training, even reaching circuit sizes which turn out to be intractable for the most common simulation libraries, considering both state-vector and tensor-network algorithms. We provide a repository of testing data in this regime, to be used for future benchmarking of quantum devices and novel classical algorithms.

Keywords: quantum computing, random quantum circuits, supervised learning, deep neural networks

1 Introduction

In recent years, deep-learning techniques have proven fitted to tackle various critical computational tasks in quantum many-body physics, including, among others [1–4], approximating ground-state wave functions [5, 6], learning universal energy-density functionals [7–10], implementing force fields for molecular dynamics [11, 12], or performing quantum-state tomography [13]. It is therefore natural to ask whether neural networks can also emulate the behaviour of quantum computers. For example, they could learn to reproduce the complex correlations among qubits from sets of measured bitstrings [14], or they could be trained via supervised learning to predict output expectation values from circuit descriptors [15].

In many applications of supervised deep-learning to many-body problems, scalability has emerged as a critical enabling feature of the adopted network architectures [15–22]. In the context of quantum computing, scalability might allow the trained networks to emulate larger circuits than those used for training, potentially reaching intractable regimes for other classical simulation methods. If successful, this approach could provide useful benchmark data for the further development of quantum computing devices. In fact, some encouraging results have been reported in Ref. [15]. Specifically, the supervised training of deep networks was performed on small universal circuits amenable to classical simulations. The networks learned to map suitable representations of universal quantum circuits to the corresponding output expectation values, and they could be tested also on larger circuits. Yet, the actual potential of this supervised-learning approach is not clear. A performance degradation was in fact reported, e.g., for deeper circuits rather than for more qubits, or for scrambling rather than for localized circuit dynamics [23]. It is worth pointing out that assessing the potential of the supervised learning of quantum circuits is important also for the further development of quantum error mitigation schemes [24–28]. For example, in Ref. [29], error suppression was implemented by training machine-learning models on exact expectation values computed for classically tractable circuits. Chiefly, for intractable circuit sizes, training was performed on noisy data improved via conventional zero-noise extrapolation, leading to a reduction of the computational overhead, in particular at runtime.

The central goal of this article is to delineate some of the successes and the limitations of scalable supervised learning of the output properties of quantum circuits. We address this issue focusing on a general circuit structure familiar from many applications of the variational quantum eigensolver (see, e.g., [30–34]). Specifically, these circuits feature layers of CNOT gates alternated with single-qubit rotations with random angles. On the one hand, we demonstrate a dramatic failure of classical supervised learning: the computational cost exponentially increases with the variance of the inter-layer angle fluctuations. This observation allows us pinpointing a regime where quantum devices can conveniently outperform classical deep learning. On the other hand, for circuits featuring only intra-layer angle variations, supervised training is efficient in terms of required training circuits. Moreover, the scalable networks accurately extrapolate to larger and deeper circuits than those used in the training phase. An analysis of the computational cost of state-vector and tensor-network simulations is provided, considering two of the most popular and performant software

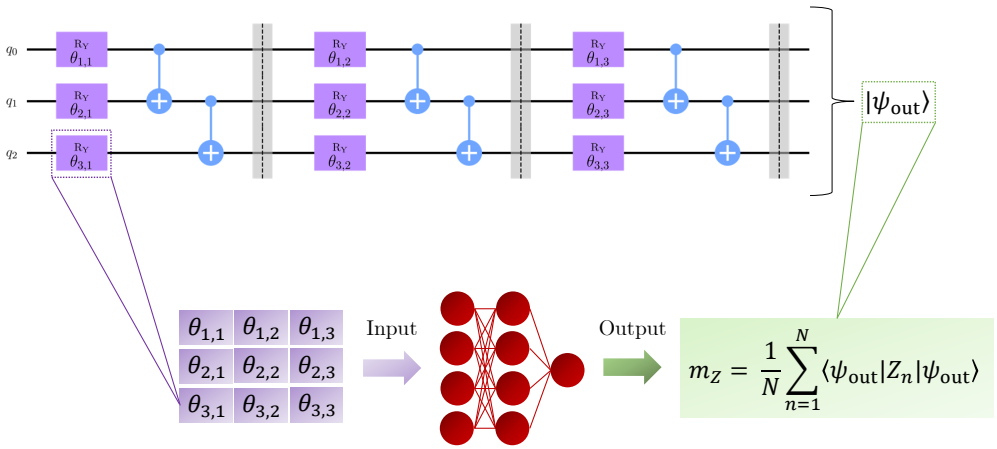


Fig. 1: Schematic representation of the supervised learning protocol of quantum circuits. The structure of the quantum circuits is shown in the upper part. Single-qubit rotations $R_y(\theta_{n,p})$ with random angles $\theta_{n,p}$ at qubit $n = 1, \ldots, N$ and layer $p = 1, \ldots, P$ are alternated with CNOT gates acting on adjacent qubits. The input state is $|\psi_{\text{in}}\rangle = |0\rangle^{\otimes N}$. The circuit descriptors $\theta_{n,p}$ (shown in the lower part) constitute the neural network input. The output is the expectation value $m_z = \frac{1}{N}\sum_{n=1}^{N}\langle\psi_{\text{out}}|Z_n|\psi_{\text{out}}\rangle$, where $|\psi_{\text{out}}\rangle$ is the output state of the quantum circuit.

executed on high-performance computers. This analysis shows that the scalable networks largely overcome the range where the current general-purpose implementations of these classical algorithms are practical. To favour future developments of quantum devices and novel classical algorithms, a dataset of benchmark data produced by one of our trained neural networks is provided at the repository of Ref. [35].

The rest of the Article is organized as follows: in Section 2, we describe the quantum circuits we consider, as well as the adopted neural-network architectures and their training protocols. Section 3 is devoted to the analysis of the prediction accuracy after supervised training, both for fixed circuit sizes and for extrapolations to larger and deeper circuits. In Section 4, we discuss the computational cost of classical simulations of the quantum circuits, considering both state-vector simulations and a tensor-network method. Section 5 summarizes our main findings and their implications on the perspective of simulating quantum computers via deep learning.

2 Methods

2.1 Quantum circuit architecture

We consider quantum circuits composed of N qubits and P layers of gates. In each layer, a parametrized single-qubit gate is applied to each qubit, and the two-qubit CNOT gate is applied to all pairs of adjacent qubits (assuming open boundary conditions). The single-qubit gates implement rotations around the y axis, corresponding

to the following matrix representation:

$$R_y(\theta) = \begin{bmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix} . \tag{1}$$

Notice that the angles applied to different qubits or in different layers may vary; we denote with $\theta_{n,p}$ the angle for the qubit with index $n = 1, \ldots, N$ at the layer $p = 1, \ldots, P$. These angles are randomly generated as follows:

$$\theta_{n,p} = u_n + u_p, \tag{2}$$

where $u_{n/p} \sim U\left(a_{N/P}, b_{N/P}\right)$, and the symbol $U\left(a, b\right)$ denotes the uniform distribution in the range $u \in [a, b]$. In the following, we consider intra-layer and inter-layer (uniform) distributions with means μ_N and μ_P, respectively; the corresponding variances are denoted as σ_N^2 and σ_P^2. The required boundaries of the uniform ranges are easily determined as $a_{N/P} = \mu_{N/P} - \sqrt{3\sigma_{N/P}^2}$ and $b_{N/P} = \mu_{N/P} + \sqrt{3\sigma_{N/P}^2}$. Specifically, three circuit configurations are considered: i) only inter-later angle fluctuations are allowed, i.e., we set $\sigma_N = 0$; ii) only intra-layer fluctuations are allowed, i.e., we set $\sigma_P = 0$; iii) both intra-layer and inter-layer fluctuations are allowed, i.e., $\sigma_N > 0$ and $\sigma_P > 0$. Examples of these three configurations are represented in Fig. 2.

To perform supervised learning, a suitable set of circuit descriptors, which we collectively indicate with the symbol Θ, is needed [36]. For what concerns configuration i), the angles only depend on the layer index p, i.e., we can write $\theta_{n,p} = \theta_p$; therefore, each circuit is unambiguously identified by the P-dimensional array $\Theta = (\theta_1, \ldots, \theta_P)$. For configuration ii), one can write $\theta_{n,p} = \theta_n$, and the minimal unambiguous circuit representation is the N-dimensional array $\Theta = (\theta_1, \ldots, \theta_N)$. Within configuration iii), a $N \times P$ matrix is needed, namely we set $\Theta = [\theta_{n,p}]$. Here, it is worth pointing out that the $N \times P$ descriptor matrix can, in principle, be adopted also for configurations i) and ii), by considering constant angles for different qubits or different layers, respectively. In fact, we employ this matrix representation in some of the extrapolation tests, performed on circuits in configurations ii), in subsection 3.2. In this case, the $N \times P$ descriptors matrix reads:

$$\Theta = \begin{bmatrix} \theta_1 & \ldots & \theta_1 \\ \theta_2 & \ldots & \theta_2 \\ \vdots & \ddots & \vdots \\ \theta_N & \ldots & \theta_N \end{bmatrix} . \tag{3}$$

2.2 Neural network architecture

Our goal is to train deep neural networks to learn the map $f_\omega\left(\Theta\right) = \tilde{y}$, where with ω we denote the weights and biases characterizing the network, and the output $\tilde{y}$ is supposed to accurately approximate the chosen target value. Specifically, the target

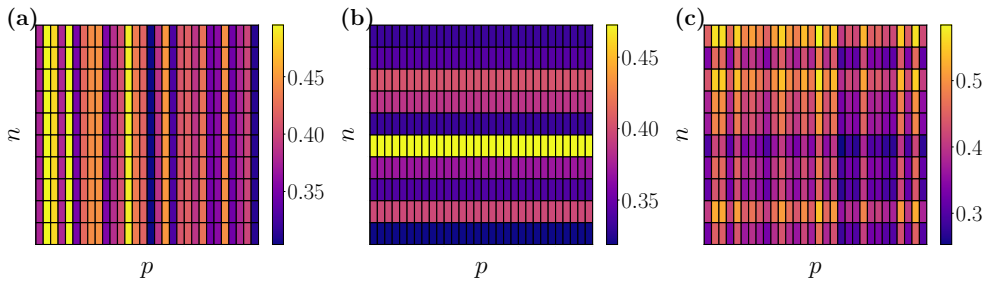


Fig. 2: Color-scale representation of the random angle $\theta_{n,p}$ as a function of the qubit index $n = 1, \ldots, N$ and the layer index $p = 1, \ldots, P$. The angles are sampled according to Eq. (2). Three circuit configurations are considered: i) Only inter-layer variations are allowed by setting $\sigma_N^2 = 0$, see panel (a). ii) Only intra-layer variations are allowed by setting $\sigma_P^2 = 0$, see panel (b). iii) Both intra-layer and inter-layer variations are allowed, see panel (c).

we consider is the average magnetization per qubit:

$$m_z = \frac{1}{N} \sum_{n=1}^{N} \langle \psi_{\text{out}} | \, Z_n \, | \psi_{\text{out}} \rangle \ , \tag{4}$$

where Z is the standard Pauli operator, and the output state can be computed as $|\psi_{\text{out}}\rangle = \hat{U} \, |\psi_{\text{in}}\rangle$, with $\hat{U}$ the unitary operator corresponding to the quantum circuit; the input state $|\psi_{\text{in}}\rangle = |0\rangle^{\otimes N}$ is a product state of spin-up eigenstates of Z_n.

Three neural-network architectures are considered: multilayer perceptrons (MLPs), convolutional neural networks (CNNs), and long short-term memory networks (LSTMs). Hereafter, these models are briefly described, while all relevant further details are provided in Appendix A. MLPs represent perhaps the most archetypal neural-network structure (they are sometimes referred to as "plain vanilla" models [37]). They are formed by a sequence of dense layers, i.e. layers featuring (bipartite) all-to-all connectivity among neurons in adjacent layers. The numbers of layers and of neurons in each layer can be adjusted, seeking for a compromise between expressivity and computational cost. This architecture is not scalable, i.e., it can be applied only to inputs of a predetermined fixed size. We adopt it only for the analysis of subsection 3.1, where the size and depth of the testing circuits coincides with the ones of the training circuits.

As a term of comparison, we confront the performance of MLPs against the ones of more sophisticated models, namely, the CNNs and LSTMs. The latter represent a type of recurrent neural network. They are commonly adopted in the analysis of temporal sequences. The long-short memory structure is designed to account for memory effect, while suppressing the vanishing gradient problems [38]. Their key feature is the LSTM cell, which includes input, forget, and output gates. These gates control the flow of information, enabling the network to selectively store, discard, and utilize information over extended sequences. Recently, LSTMs have been adopted also to

learn the unitary dynamics of quantum spin models, adiabatic quantum computers, and gate-based circuits [23, 39, 40]. We also apply them to quantum circuits, specifically for configuration i), by identifying the (discretized) time with the layer index p. The angle θ_p is used to predict the expectation value at the corresponding time index $m_z^{(p)}$, together with the information passed from the previous time history. The resulting scheme is detailed in Appendix A.

In subsection 3.2, we perform extrapolation tests, namely, we test the trained networks on larger and deeper circuits than those included in the training sets. For this, a scalable architecture is required, pointing to convolutional models. Yet, it is worth pointing out that the standard CNNs adopted in computer science (usually for image recognition [41]), which feature convolutional filters with fixed-size kernels followed by a set of dense layers, are not fully scalable. In fact, while the former layers can, in principle, be applied to variable-size inputs, dense layers require instead a fixed input size. Following Ref. [17], we include a global pooling operation after the last convolutional layer, thus obtaining a fully scalable architecture. We perform extrapolations, with circuits in configuration ii), to larger qubit numbers at fixed circuit depths, as well as extrapolations to both larger and deeper circuit. In the first case, a one-dimensional CNN operating on the N-dimensional descriptor array is appropriate. In the second case, scalability in both N and P is obtained considering a two-dimensional CNN, and the input is a $N \times P$ matrix, as discussed in subsection 2.1. It should be mentioned here that, in the literature, various alternative strategies have been discussed to perform scalable learning of quantum many-body systems [19]. These strategies include, e.g., tiling the systems [16], adopting fixed-size system representations [18], or addressing extensive observables [21, 39]. For the tests on fixed circuit sizes and depths, full scalability is instead not required. Hence, a standard CNN featuring a flatten operation to connect the last convolutional layer to the dense part is adequate. We adopt this structure for the tests in subsection 3.1.

The training of the networks described above is performed via supervised learning. A training dataset $\{(\Theta_k, y_k)\}_{k=1}^{K_{\text{train}}}$, where y_k represents the exact target value corresponding to the circuit with angles Θ_k, is generated by performing circuit simulations using the Qiskit library [42] and the qsimcirq library [43]. We mostly perform exact state-vector simulations. Tensor-network methods are also adopted to assess the cost of classical simulations; see Section 5. The network parameters ω are optimized by minimizing the mean squared error

$$\mathcal{L} = \frac{1}{K_{\text{train}}} \sum_{k=1}^{K_{\text{train}}} (y_k - \tilde{y}_k)^2 \,, \tag{5}$$

where $\tilde{y}_k = f_\omega(\Theta_k)$. The training is performed via a popular variant of stochastic gradient descent, namely, the ADAM algorithm [44]. To quantify the prediction accuracy, we consider the coefficient of determination

$$R^2 = 1 - \frac{\sum_{k=1}^{K_{\text{test}}} (y_k - \tilde{y}_k)^2}{\sum_{k=1}^{K_{\text{test}}} (y_k - \bar{y})^2} \,, \tag{6}$$

where $\bar{y}$ is the average of the target values and K_{test} is the number of random circuits in the test set. Clearly, the test sets contain only circuits which are not included in the training set. Notice that perfect predictions correspond to $R^2 = 1$, while a constant model with the exact average corresponds to $R^2 = 0$.

3 Accuracy of supervised learning

3.1 Testing on fixed circuit sizes

Hereafter we investigate if and in which conditions deep learning techniques allow predicting the magnetization per spin m_z in the output state of the quantum circuits described in subsection 2.1. In this first analysis, the tests are performed on circuits with the same qubit number and circuit depth of the ones included in the training datasets. Hence, a scalable architecture is not required. Most of the tests are performed by adopting the MLP architecture, but LSTMs and CNNs are also considered for comparison. The first test addresses circuits in configuration i) (see subsection 2.1), which only feature inter-layer angle fluctuations. The results are shown in Fig. 3. One notices that the prediction accuracy rapidly decreases with the inter-layer variance σ_P^2. The drop is somehow delayed for smaller angle means μ_P [1]. Furthermore, the three adopted neural networks (MLP, CNN, and LSTM) display very similar performances, indicating that the network architecture does not play a key role.

In general, one expects that networks trained on larger datasets display a superior performance. This indeed occurs also in the problem addressed here. To quantify this effect, we determine the number of training circuits K_{train} required to reach the accuracy threshold $R^2 \geq 0.99$, as a function of the inter-layer variance σ_P^2. The results are analysed in Fig. 4, considering the MLP architecture. Notably, we observe an exponential increase as a function of σ_P^2, which in turn implies an exponential computational cost for dataset generation and network training. Already for $\sigma_P^2 \gtrsim 0.01$, computationally prohibitive numbers of training circuits are required. This indicates the failure of supervised learning for circuits with large inter-layer angle fluctuations. Notice that this type of failure differs from the one discussed in Ref. [23], as the latter only occurs in extrapolations to larger circuit sizes, while here it occurs already for fixed circuit size.

On the other hand, for circuits with small inter-layer variance $\sigma_P \simeq 0$, the network predictions are very accurate. This is confirmed by the analysis on circuits in configuration iii) (see details in Section 2), which feature both intra-layer and inter-layer angle fluctuations; this analysis is shown in Fig. 5. One observes a slow accuracy decrease with the intra-layer variance σ_N^2, while increasing the inter-layer variance σ_P^2 causes a sharp downward shift. Again, it is worth determining the required dataset size K_{train} to reach $R^2 \geq 0.99$, as a function of σ_N^2, now fixing $\sigma_P^2 = 0$. These results are shown in Fig. 4. One notices that orders of magnitude smaller K_{train} are required, and that the rate of increase is qualitatively slower as compared to the previously analysed case, i.e. configuration i). Hence, for relevant rotation ranges, e.g., in the range $\theta_{n,p} \in [0, \pi/2]$ [45], circuits featuring only intra-layer fluctuations are amenable

[1] It occurs also for $\mu_P \simeq 0$, but in a larger σ_P range than the one shown in Fig. 3.

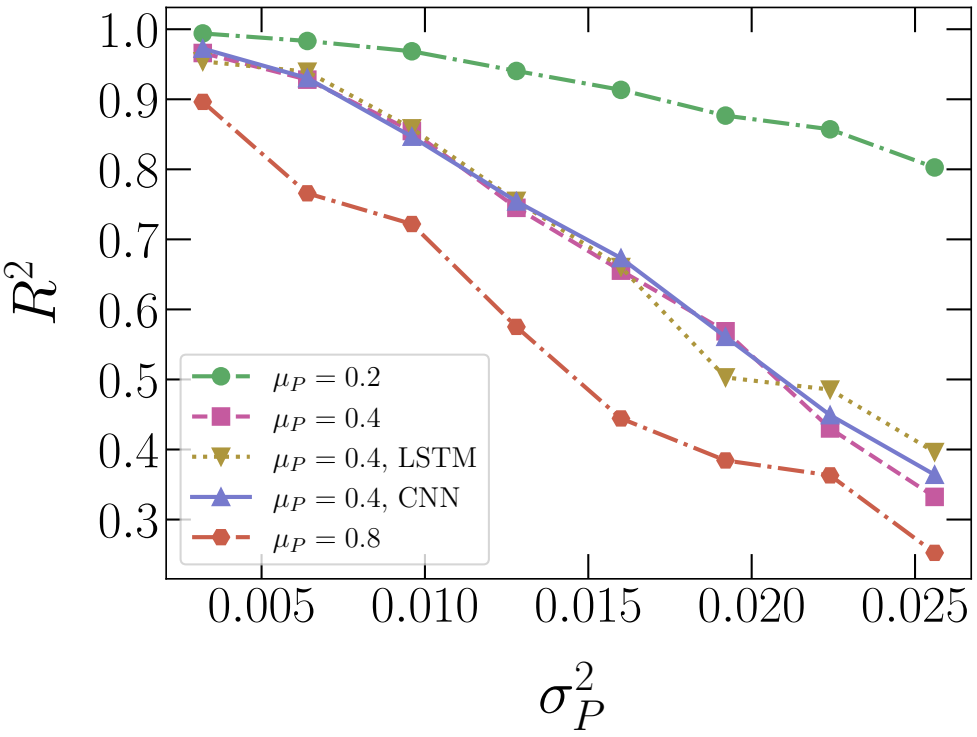


Fig. 3: Coefficient of determination R^2 as a function of the inter-layer variance σ_P^2. Different datasets correspond to different values of the mean μ_P and to different network architectures: MLP, LSTM, and CNN. The networks are trained and tested on quantum circuits of the same size: $N = 10$ and $P = 30$. The training sets include 10^4 circuits.

to supervised learning. This is in contrast with the case of inter-layer fluctuations, where the required K_{train} can be easily tuned to unfeasible sizes.

3.2 Extrapolations to larger circuits

Hereafter, we test the networks on larger and/or deeper circuits than those used in the training phase. For this, a scalable network architecture is required. We consider circuits in configuration ii), i.e., we allow only intra-layer angle variations by setting $\sigma_P = 0$. In this scenario supervised learning for fixed circuit sizes has proven successful (see subsection 3.1), and it is thus at least plausible to obtain accurate extrapolations also for larger and deeper circuits. In the first test, the extrapolation is performed only to larger qubit numbers, fixing the depth of both training and testing circuits at $P = 30$. In this setup, a one-dimensional CNN provides the required scalability, as discussed in subsection 2.2. The random angles are sampled from uniform distributions with mean $\mu_N = 0.4$ and variance $\sigma_N^2 = 0.1024$. To help the networks learning how to

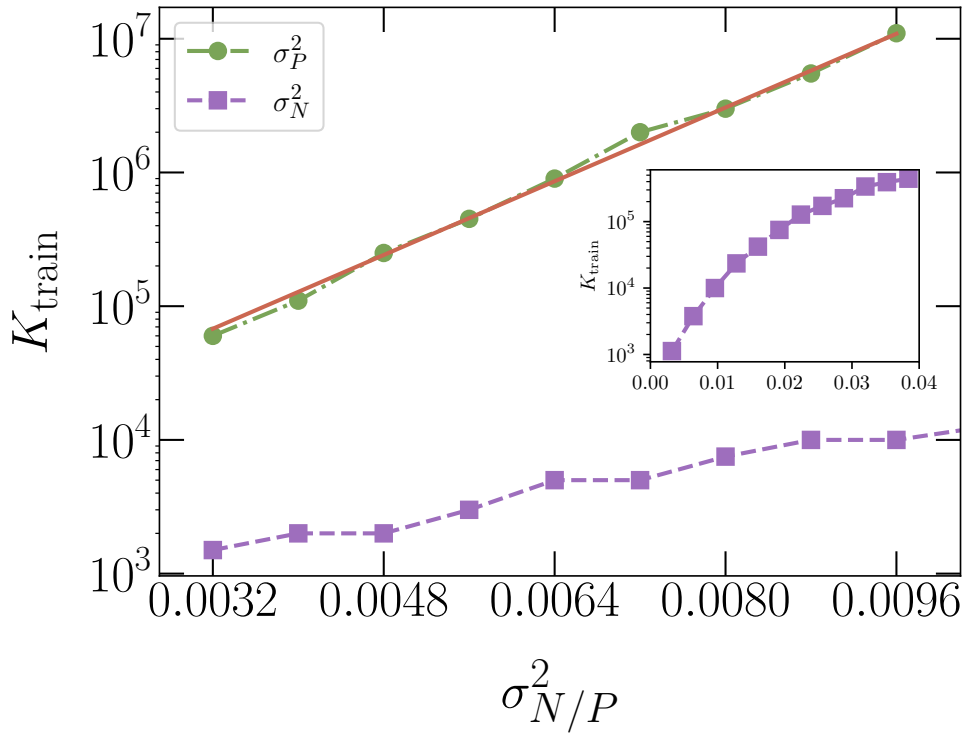


Fig. 4: Number of training circuits K_{train} required to reach the accuracy $R^2 \geq 0.99$, as a function of the inter-layer variance σ_P^2 or the intra-layer variance σ_N^2 . The MLP is trained and tested on circuits with $N = 10$ qubits and $P = 30$ layers. (Green) circles correspond to circuits in configuration i): $\sigma_N^2 = 0$ and inter-layer mean $\mu_P = 0.4$; (purple) squares correspond to configuration ii): $\sigma_P^2 = 0$ and $\mu_N = 0.4$. The continuous (red) line represents an exponential fitting function: $K_{\text{train}} = a \exp(b\sigma_P^2)$, with the parameters $a \simeq 5350$ and $b \simeq 794$ obtained from a best fit analysis. The inset displays the scaling of K_{train} in a larger range of σ_N^2; here, the error-bar is estimated considering two trainings of the MLP and it turns out to be smaller than the marker size.

scale the predictions with the circuit size, we gather heterogeneous training datasets featuring circuits with different qubit numbers. Specifically, the training qubit numbers are $N_{\text{train}} \in \{N_0, N_0 + 1, N_0 + 2\}$, and the smallest training circuits range from $N_0 = 8$ to $N_0 = 18$ qubits. Here, the training sets include 1.2×10^6 circuits for each size. In Fig. 6, the prediction accuracy is analysed as a function of the size of the testing circuits N_{test}. It is reasonable to expect that small training circuits might not provide sufficient information to allow the network accurately extrapolating to qubit numbers $N_{\text{test}} \gg N_{\text{train}}$. This is indeed the case; for example, the network trained with $N_0 = 8$ is no longer accurate in the regime $N_{\text{test}} \gtrsim 13$. On the other hand, when the training qubit numbers are sufficiently large, namely $N_{\text{train}} \gtrsim 12$, the accuracy no longer collapses for larger N_{test}. This allows one to accurately predict the output of

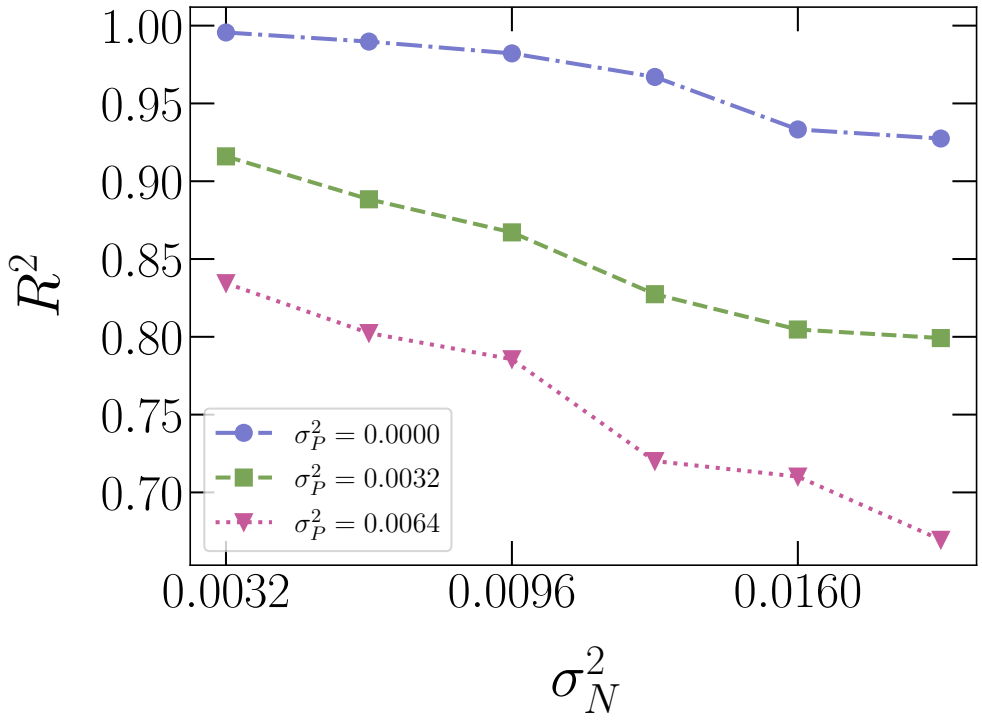


Fig. 5: Coefficient of determination R^2 as a function of σ_N^2 for different values of σ_P^2. The MLPs are trained and tested on quantum circuits with $N = 10$, $P = 30$, $\mu_N = 0.4$ and $\mu_P = 0$. Here, the MLP has input shape $N \times P$. The training sets include 10^4 quantum circuits.

circuits with, at least, as many as $N_{\text{test}} = 30$ qubits. For the depths we consider, performing state-vector simulations of significantly larger circuits on classical computers is impractical. In fact, already to reach qubit numbers $N_{\text{test}} \gtrsim 25$, high-performance computers are required. Furthermore, for the circuit model addressed here, simulations based on tensor-network methods are also ruled out, since they require impractically large bond-dimensions. This point is demonstrated in Section 4, where we also provide timings and other relevant details of our state-vector simulations in the large circuit regime.

In the absence of benchmark data from other classical simulation methods in the regime $N_{\text{test}} \gtrsim 30$, the neural-network accuracy can be assessed by comparing the predictions provided from trainings performed on different qubit numbers N_{train}. In Fig. 7, we examine the coefficient of determination R^2 of different networks, identifying the predictions of the network with the largest training circuits, namely, $N_{\text{train}} \in \{18, 19, 20\}$, as the ground-truth benchmark. One notices that the predictions rapidly converge as the training circuit-size increases. The predictions obtained from the second-largest training set barely deviate from the chosen benchmark, reaching

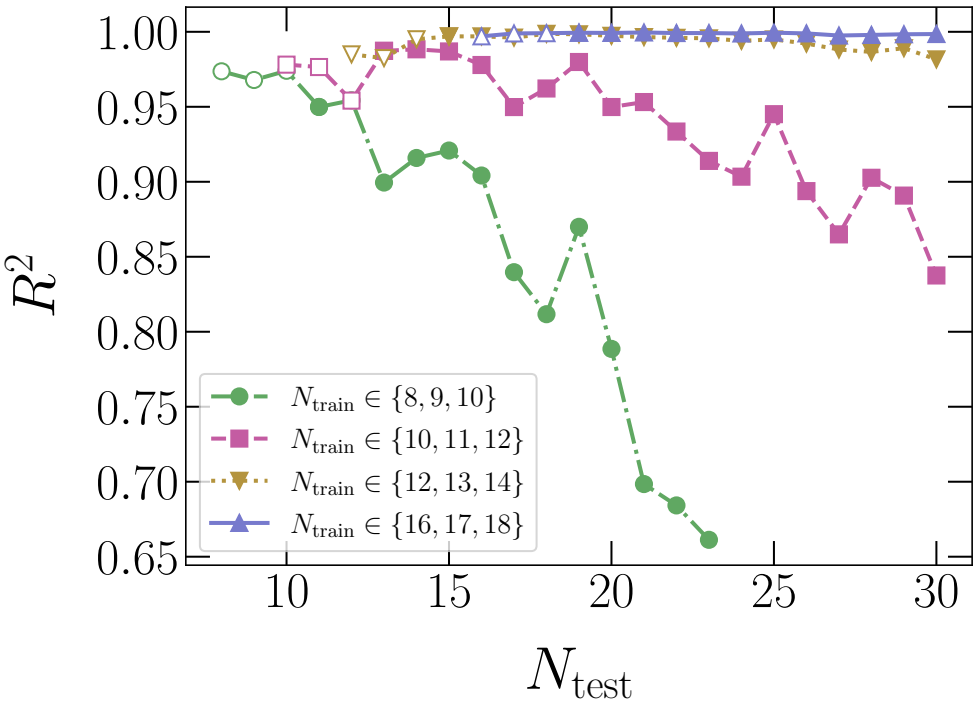


Fig. 6: R^2 as a function of the number of qubits N_{test} of the testing quantum circuits. Four 1D-CNNs are trained on different qubit numbers N_{train}, considering circuits in configuration ii) of depth $P = 30$, with $\mu_N = 0.4$ and $\sigma_N^2 = 0.1024$. The empty symbols represent the tests performed on the same circuit size as the training circuits. Filled symbols correspond to extrapolations to large circuits.

$R^2 > 0.99$ even up to $N_{\text{test}} = 60$. This accuracy threshold can be interpreted as a conservative estimate of the prediction accuracy of the benchmark network in the regime $N_{\text{test}} \simeq 60$.

Thus, we conclude that scalable supervised learning provides reliable expectation-value predictions in regime where the current implementations of two of the most popular classical simulation methods are essentially impractical. To favour future comparisons with both quantum devices and novel classical simulation platforms, the predictions for quantum circuits in the ramge $N \in [30, 60]$ are made available to the community through the repository in Ref. [35]. It is worth emphasizing that this network is trained and tested on random angles with mean $\mu_N = 0.4$ and (intra-layer) variance $\sigma_N^2 = 0.1024$. Tests performed on different distributions are not guaranteed to be successful. For example, performing the test on the circuit with no rotations, corresponding to the descriptors $\theta_n = 0$ for $n = 1, \ldots, N$, the network prediction turns out to largely deviate from the expected result $m_z = 1$. Yet, this problem can be easily solved, e.g., by augmenting the training dataset with 10^3 circuits generated with mean $\mu_N = 0$ and $\sigma_N^2 = 10^{-4}$. This simple trick leads to an accurate prediction

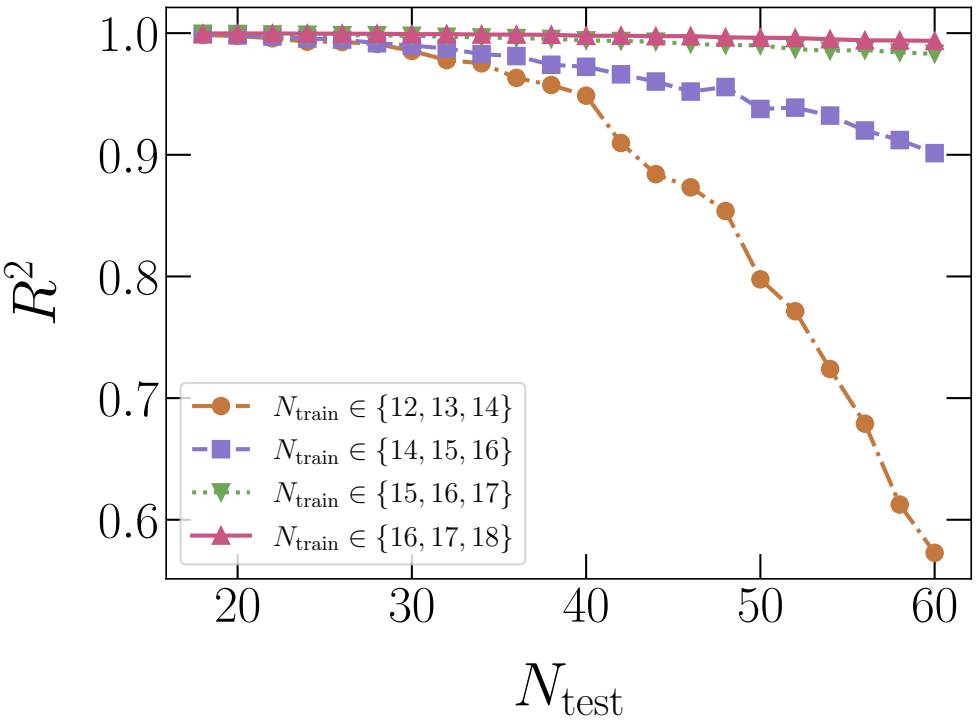


Fig. 7: R^2 as a function of the number of qubits N_{test} of the testing circuits. Four 1D-CNN are trained on heterogeneous circuits featuring three qubit numbers N_{train} (see keys), and their predictions are compared to the ones from the training sizes $N_{\text{train}} \in \{18, 19, 20\}$. The circuits are in configuration ii), with angles as in Fig. 6.

of the CNN trained with quantum circuits with $N_{\text{train}} \in \{16, 17, 18\}$ qubits for the above test circuit for $N_{\text{test}} = 100$, namely, $\tilde{y} = 0.99074167 \cong m_z$.

Next, we analyse extrapolations in two directions, namely, to circuits featuring both more qubits and more layers than those included in the training set. In this case, full scalability is obtained adopting a two-dimensional CNN, as discussed in subsection 2.2. The extrapolation accuracy is analysed in Fig. 8. The network is trained on circuits with qubit numbers $N_{\text{train}} \in \{12, 13, 14\}$ and depths $P_{\text{train}} \in \{28, 30\}$, featuring 1.2×10^6 instances of each size and depth. Notably, also in this test the prediction accuracy appears not to degrade with the size and the depth of the testing circuits, indicating that scalability in both directions is indeed feasible. To further visualize the networks accuracy and the (non-trivial) output distribution, in Fig. 9 we show a scatter plot of predictions $\tilde{y}$ for circuits with $N_{\text{test}} = 20$ qubits and $P = 40$ layers, as a function of the ground-truth expectation values m_z. The good agreement is clear. Some small deviations only occur for the largest magnetizations in the dataset. This can be attributed to the sparsity of training instances in that regime.

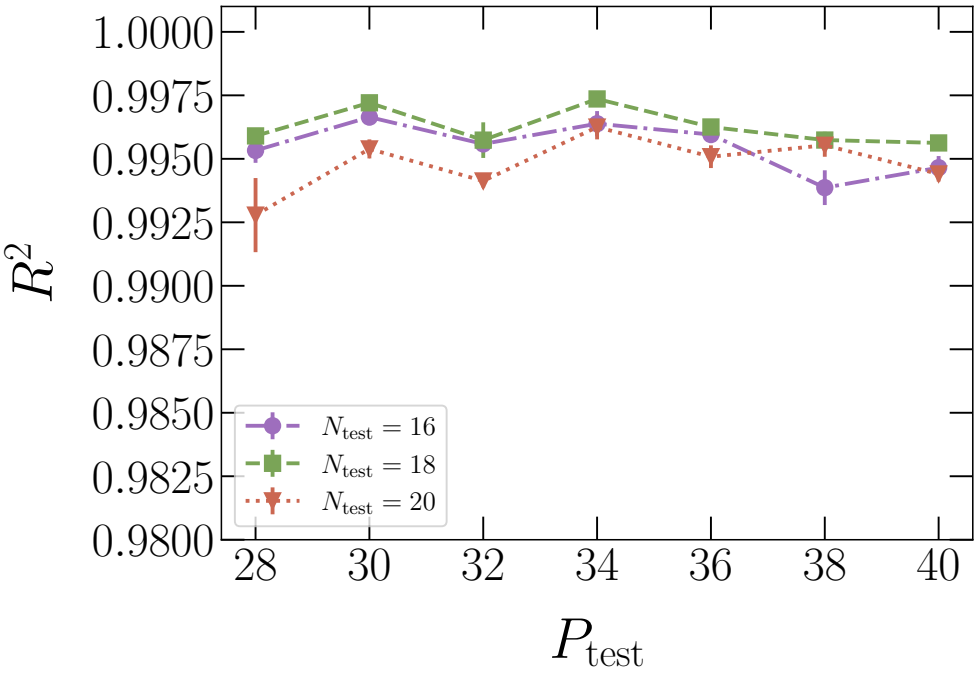


Fig. 8: R^2 as a function of the number of layers of gates P_{test} of the quantum circuits in the test set, for different number of qubits N_{test}. The CNN is trained on circuits in configuration ii) with $N_{\text{train}} \in \{12, 13, 14\}$, $P_{\text{train}} \in \{28, 30\}$, $\mu_N = 0.4$, and $\sigma_N^2 = 0.1024$. The test set includes 10^3 circuits and it is divided in four subsets with 250 instances. The error-bars represent the estimated standard deviation of the average over the four subsets.

4 Computational cost of classical circuit simulations

The computational cost of accurate classical simulations of general quantum circuits is expected to scale exponentially for sufficiently large sizes and depths. Hereafter, we demonstrate that, for the random circuit we address, this exponential scaling sets in already for $N \gtrsim 20$, making simulations of $N \simeq 60$ circuits essentially impossible. First, we consider so-called state-vector methods, which provide numerically exact predictions of output properties. We adopt two of the most popular circuit simulation software, namely, the Qiskit [42] and qsimcirq [43] libraries. It is worth mentioning that a detailed performance assessment of essentially all leading software platforms for circuit simulations has recently been reported [46]. Considering variegate circuit types, that study assessed that Qiskit and qsimcirq are, indeed, the most efficient libraries in the large N regime across variegate circuit types and hardware platforms. Notice that also Ref. [46] presents exponential scalings for $N \gtrsim 20$, while timings on smaller circuits are often influenced by programming overheads. In Fig. 10, we show the computation time for two different hardware, referred to as Hardware1 and

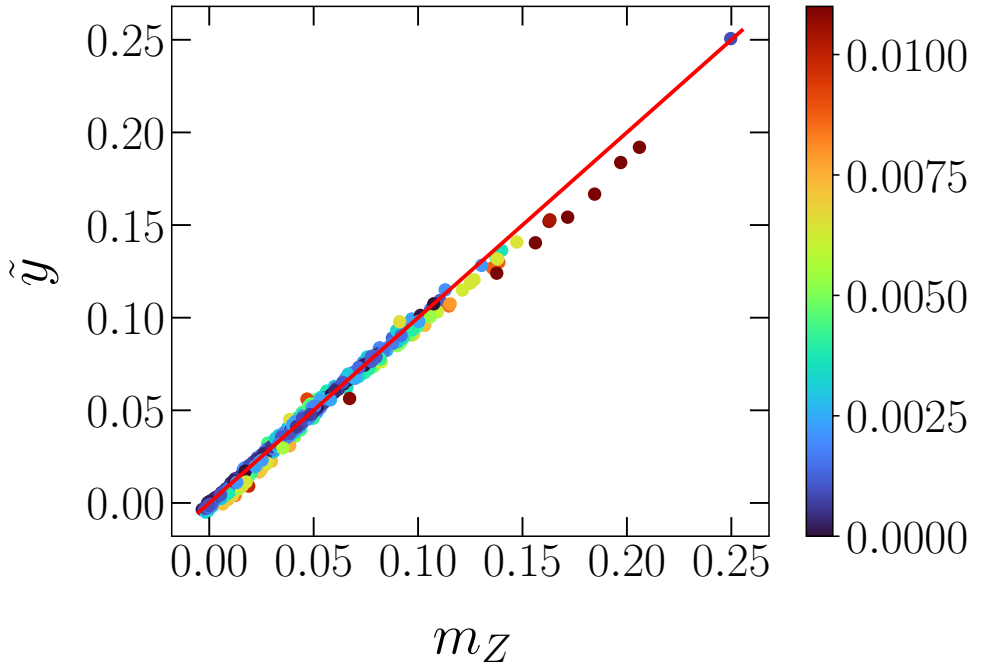


Fig. 9: Magnetization per spin $\tilde{y}$ predicted by the CNN as a function of the exact results m_z. The quantum circuits have $N = 20$ qubits and $P = 40$ layers of gates. The CNN has been trained on quantum circuits with $N_{\rm train} \in \{12, 13, 14\}$ and $P_{\rm train} \in \{28, 30\}$. The color scale represents $|\tilde{y} - m_z|$. The (red) line represents the bisector $\tilde{y} = m_z$. The rotation angles are as in Fig. 8.

Hardware2. Notice that qsimcirq operates using single precision, while Qiskit is set to operate in double precision. By fully utilizing Qiskit's capabilities, we are able to maximize the performance of the available hardware by exploiting all CPU cores. In particular, Hardware1 features the following CPU: Intel(R) Xeon(R) Gold 6154 CPU, 72 threads (36 cores with hyperthreading), with 192 Gb of RAM memory; Hardware2 is available from the Galileo100 cluster at CINECA, whereby each node features 2 Intel CascadeLake 8260 CPUs, with 24 cores each and a total of 384GB RAM memory. Importantly, the largest qubit number we are able to address $N = 34$ requires order of $T = 5 \times 10^3$s. Exploiting accelerators such as GPUs is expected to provide a quantitative speed-up [46], but sizes such as $N \simeq 60$ are evidently out of reach due to the observed exponential scaling with N.

Given the above timings based on state-vector algorithms, tensor-network methods offer a promising alternative. In particular, for circuits that do not generate largely entangled states, the bond dimension needed to eliminate any bias is known to be small. Shallow circuits acting on product input states are not expected to create strongly entangled states. It is thus worth inspecting whether the circuit depths

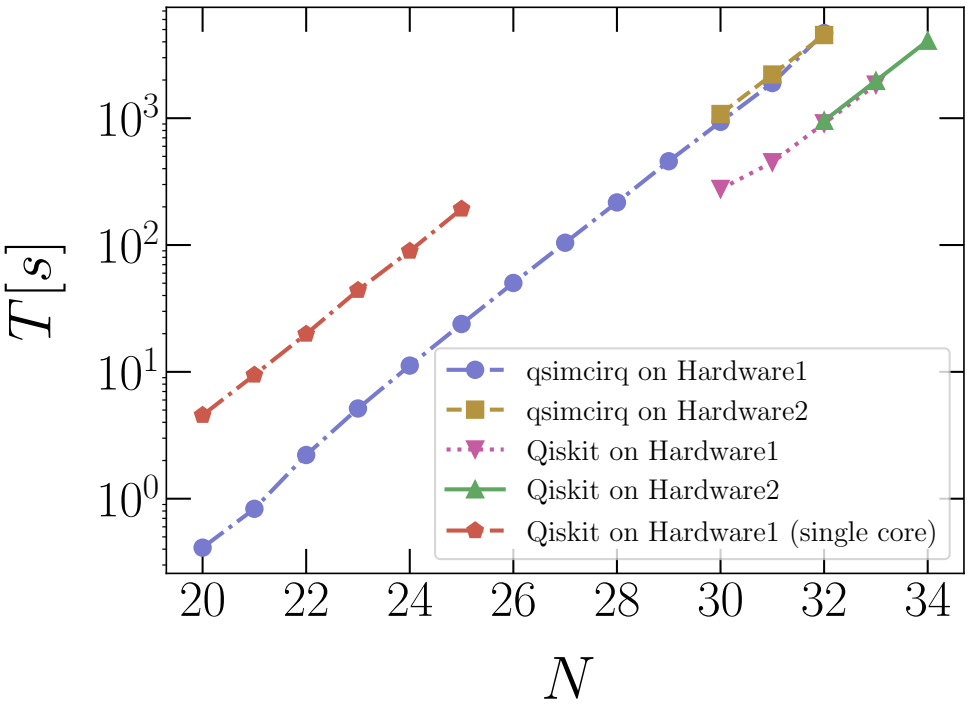


Fig. 10: Time T (in seconds) required to simulate one random circuit with state-vector methods, as a function of the qubit number N. The simulations are performed with the Qiskit and qsimcirq libraries on two different hardware. The circuits have N qubits, $P = 30$ layers of gates, $\mu_N = 0.4$, $\sigma_N^2 = 0.1024$, and $\sigma_P^2 = 0$.

addressed here are amenable to tensor-network simulations. For this purpose, we determine the bond dimension required to predict the output expectation value m_z with an accuracy corresponding to $R^2 \geq 0.99$, as a function of qubit number and circuit depth. The results are shown in Fig 11, addressing a circuit featuring only intra-layer angle fluctuations, namely, the circuit configuration for which supervised deep-learning is efficient. One notices that already for depths $P \gtrsim 30$ the bond dimension approaches the worst-case scenario, namely, the exponential scaling $\chi \propto 2^{N/2}$. The small deviations from this scaling can be attributed to the allowed precision tolerance and to the finite circuit depth. The exponential scaling of the bond dimension is expected to lead to an equivalent scaling of the total computation time of the tensor-network method. This is indeed demonstrated by the analysis reported in Fig. 12, which is based on Qiskit matrix-product-state routine executed on Hardware1. Extrapolating the exponential trend observed for $N \gtrsim 14$ leads, again, to unfeasible timings for $N \simeq 60$ qubits.

All the timings reported above corroborate the findings discussed in Section 3.2, which indicate that, for the circuits in configuration ii), the scalable CNNs can emulate

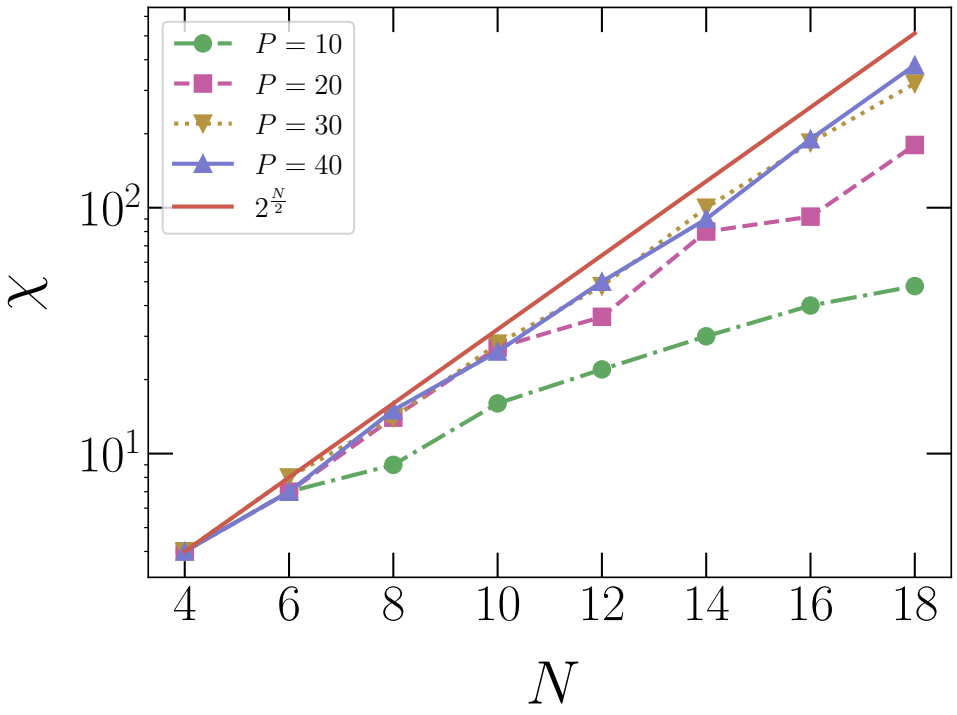


Fig. 11: Bond dimension χ required to simulate quantum circuits with the accuracy $R^2 \geq 0.99$ using Qiskit's matrix product state (MPS) method, as a function of the qubit number N. To calculate R^2 we compare the MPS prediction with the ground-truth result m_z obtained via a state-vector simulation. The results are averaged over 10 random circuits, featuring P layers of gates, $\mu = 0.4$, $\sigma_N^2 = 0.1024$, and $\sigma_P^2 = 0$. The (red) line represents the bond dimension needed to perform an ideal simulation: $\chi \propto 2^{\frac{N}{2}}$.

quantum computers beyond the reach of the current implementations of the most popular classical simulation libraries.

5 Conclusions

If and to what extent deep neural networks can emulate quantum circuits is an open question, with critical implications for the further development of quantum computing. In fact, it was recently proposed that autoregressive networks, e.g., large language models, might be able to learn the complex correlations occurring in bitstrings measured from the output state of universal quantum computers [14]. Some numerical evidence of this type of unsupervised training, which uses datasets including only measured bistrings, was indeed reported, but employing quantum-annealing devices

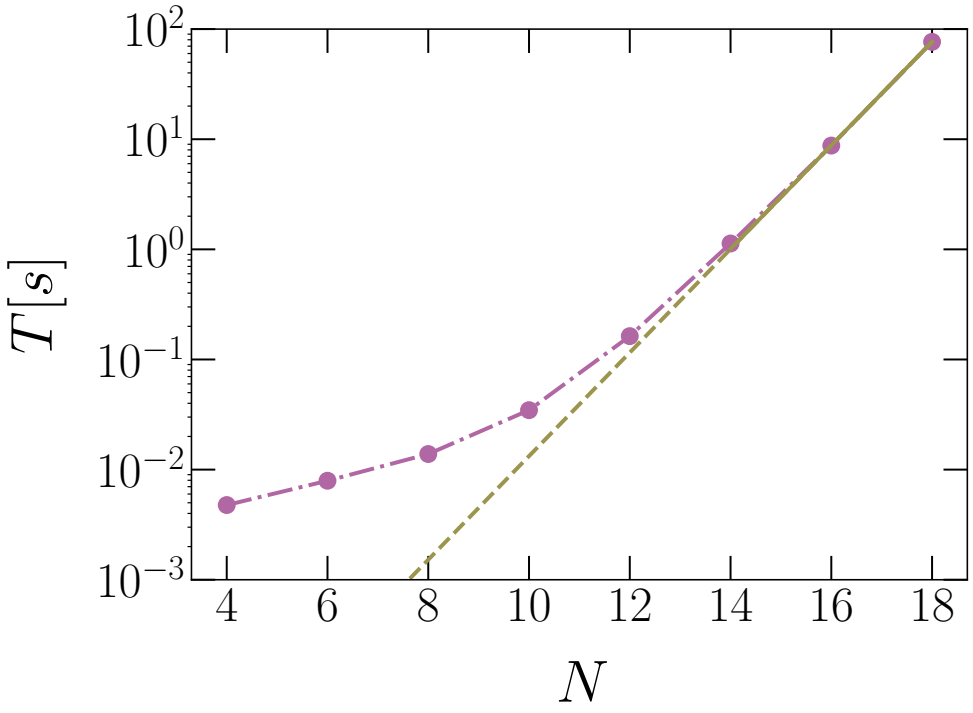


Fig. 12: Time T (in seconds) required to simulate, using Qiskit MPS method, one quantum circuit as a function of the number of qubits N. The circuits have $P = 30$ layers of gates, $\mu_N = 0.4$, $\sigma_N^2 = 0.1024$, and $\sigma_P^2 = 0$. The continuous (dark yellow) line represents an exponential fitting function: $T = a\exp(bN)$, with the parameters $a \simeq 2.6 \times 10^{-7}$, $b \simeq 1.08$ obtained from a best fit analysis in the large size regime $N \geq 14$. These results are obtained exploiting one core of Hardware1. The bond dimension is scaled to reach the accuracy $R^2 \geq 0.99$ for each N.

tailored to sample low-energy configurations of spin glasses [47]. In the context of universal gate-based devices, deep neural networks were trained via supervised learning – hence, using datasets featuring both circuit descriptors and the corresponding target values – to predict some output expectation values [15, 23]. However, the limitations of this supervised approach are still unclear, and it is the purpose of this Article to quantify them in some relevant cases. In fact, by focusing on parametrized circuits familiar from applications of variational quantum algorithms, we identified a regime where supervised learning entails an exponential cost, as well as a successful regime where scalable networks outperform the most common general-purpose classical simulation algorithms.

In detail, the circuits we addressed feature layers of single-qubits rotations with random angles, alternated with layers of CNOT gates acting on all pairs of adjacent qubits on a chain. The variance of the (random) angle fluctuations turns out to be the critical factor that determines the performance of supervised learning. Indeed, the

required amount of training data scales exponentially with the inter-layer variance; this allows us identifying the regime where supervised-trained neural networks fail to efficiently emulate quantum circuits. It is worth mentioning that the heuristic circuits commonly used in the variational quantum eigensolver do indeed feature (also) inter-layer angle variations. Notice also that different neural networks, namely the multi-layer perceptron, convolutional networks, and long-short term memory networks, reach comparable performances, indicating that the amount of training data is the critical resource that determines the prediction accuracy. On the other hand, circuits featuring only intra-layer angle fluctuations are accurately emulated at a feasible computational cost. In fact, thanks to the scalable architecture, the trained networks scale up to larger qubit numbers and circuit depths than those used in the training phase. Interestingly, the extrapolation accuracy rapidly improves with the size of the training circuits. In fact, by comparing predictions from trainings performed on different qubit numbers, we obtained clear indications that these extrapolations maintain an accuracy level of $R^2 \geq 0.99$ at least up to $N_{\text{test}} \simeq 60$, and possibly even further. Notice that, as shown by the computational-cost analysis reported in Section 4, exactly simulating these circuit sizes is currently out of reach for two of the most popular and efficient state-vector simulation software. In fact, in the regime $N \gtrsim 20$, the computational cost displays the expected asymptotic exponential scaling with N. On the other hand, approximate simulations based on tensor-network method are also out of game, since for the circuit depths we consider the required bond dimension turns out to scale close to exponentially.

It is plausible that, in the near or midterm future, further algorithmic and/or hardware developments, in particular adopting some more specialized technique, might reach sizes comparable to the one addressed in this Article. For example, it was recently shown that, adopting the belief-propagation approximation for tensor contraction [48], a tensor network method is able to accurately simulate the 127 qubits of IBM's Eagle Kicked Ising experiment [45]. To favour future comparisons with such novel simulation tools, as well as with actual quantum devices, we provide in the repository of Ref. [35] dataset suitable for benchmarking purposes. In detail, these datasets include sets of circuits descriptors associated to the corresponding output expectation value m_z, as predicted by one of our scalable neural networks, reaching the circuit size $N = 60$.

A full understanding of what makes a quantum circuit intractable for supervised learning will require further investigations. It is worth mentioning here that probabilistic deep-learning simulation algorithms have also been proposed [49], but their reach is also still not well understood. The studies on supervised training are important also to set target goals to claim quantum computational advantage and to further improve error-mitigation schemes based on deep learning. In principle, supervised training could be performed also using a combination of exact expectation values from classical simulations of small circuits, and noisy outputs of (classically intractable) large-scale quantum devices. We leave these endeavours to future investigations.

Acknowledgements. We are thankful to Luca Brodoloni, Emanuele Costa, Andrea Mari, Giuseppe Scriva, and David Vitali for fruitful discussions. We acknowledge support from the Italian Ministry of University and Research under the PRIN2022

project “Hybrid algorithms for quantum simulators” – 2022H77XB7, and partial support under the PRIN-PNRR 2022 MUR project ”UEFA” – P2022NMBAJ. Support from the PNRR MUR project PE0000023-NQSTI is also acknowledged. S.P. acknowledges support from the CINECA awards IsCa6_NEMCAQS and IsCb2_NEMCASRA, for the availability of high-performance computing resources and support. We also acknowledge the EuroHPC Joint Undertaking for awarding this project access to the EuroHPC supercomputer LUMI, hosted by CSC (Finland) and the LUMI consortium through a EuroHPC Regular Access call.

Declarations

- Some benchmark data are made available through the repository at Ref. [35]. All other data and codes are available from the authors upon reasonable request.

Appendix A Machine learning models

A.1 Multilayer perceptron

The multilayer perceptron (MLP) adopted in this study is composed of five layers. The first four layers include 512 neurons each. These layers employ the rectified linear unit (in jargon, ReLU) activation functions to add non-linearity. The output layer consists of a single neuron with linear activation.

A.2 Long short-term memory network

The long short-term memory (LSTM) network employed in this investigation is structured with three LSTM layers, each comprising 256 neurons. These layers are configured to produce an output for each time step, which is identified with the layer index p. The final layer is a dense layer that generates a value for each time step, containing the sequential information encoded by the preceding LSTM layers. In this case, the input to the p-th time-step is the angle describing the corresponding layer of gates, while the output is the global magnetization $m_z^{(p)}$ after the action of that layer of gates. This procedure is described in Fig. A1. The basic architecture of an LSTM unit consists of three gates: the input gate (i_t), the forget gate (f_t), and the output gate (o_t), along with a memory cell (c_t) that stores information over time. The hidden state at time t is denoted as h_t. The input gate controls the flow of information that should be stored in the memory cell. It determines which parts of the incoming data should be remembered and updated in the cell state. The input gate is activated by the sigmoid function, which provides values between 0 and 1:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)\,. \tag{A1}$$

Here, W_i and U_i are weight matrices, b_i is a bias vector, σ is the sigmoid activation function, x_t is the input at time t, and h_{t-1} is the hidden state from the previous time step. The forget gate decides which information from the previous cell state should

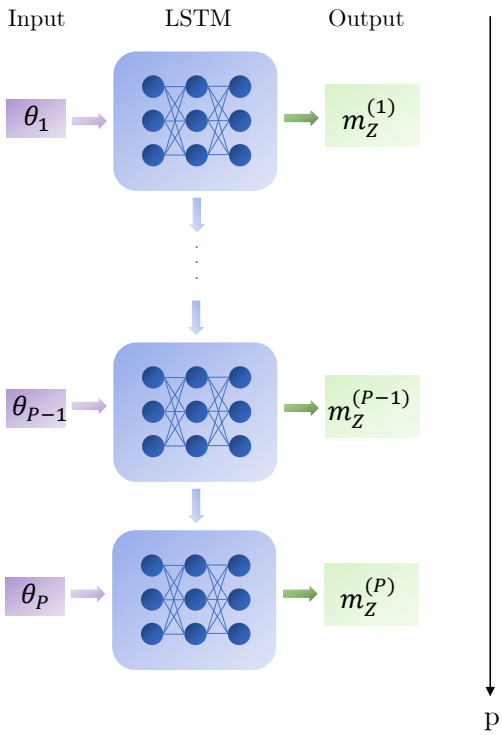


Fig. A1: Schematic representation of the application of the LSTM network to the simulation of quantum circuit featuring P layers. The rotation angle at layer $p = 1, \ldots, P$ is denoted with θ_p, while $m_Z^{(p)}$ denote the magnetization per spin after the corresponding layer.

be discarded. It helps in removing irrelevant information from the memory cell. The forget gate layer is

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)\,, \tag{A2}$$

where W_f and U_f are the weight matrices, while b_f is the bias vector. The output gate determines what information from the current memory cell state should be the output. It controls the flow of information from the memory cell to the hidden state. The output gate layer can be written as

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)\,, \tag{A3}$$

where W_o and U_o are the weight matrices and b_o is the bias vector. The memory cell is updated by combining the information from the input gate and the cell input activation (g_t), and forgetting the irrelevant information determined by the forget gate. The cell input activation vector and the state of the cell are described by the following equations:

$$g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g)\,, \tag{A4}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \,. \tag{A5}$$

Here, tanh is the hyperbolic tangent activation function, W_g and U_g are weight matrices, b_g is the bias vector, $\odot$ denotes element-wise multiplication, and c_{t-1} is the previous cell state. The hidden state is updated by applying the output gate to the hyperbolic tangent of the current cell state:

$$h_t = o_t \odot \tanh(c_t) \,. \tag{A6}$$

A.3 Convolutional neural networks with flatten layer

We use this artificial neural network as a comparison with MLP and LSTM. It has four convolutional layers with 32, 64, 128 and 128 filters each. They employ the ReLU activation function and a kernel size of three, with zero padding. A flatten layer connects the convolutional part with the dense part. The latter is composed by four layers, each containing 256 neurons and the ReLU activation function. The output layer includes one neuron with linear activation function.

A.4 Convolutional neural networks with global pooling layer

The 1D Convolutional neural network (1D-CNN) with global pooling includes four convolutional layers. The initial one features 256 filters, employs the ReLU activation function and utilizes a kernel size of three with zero padding. This layer is designed to process input sequences of variable length, each containing a single channel. Successive convolutional layers maintain the same configuration. Following the convolutional layers, a global average pooling layer is incorporated to capture global information across the data and to maintain the scalability property of the convolutional architecture. The network ends with four dense layers, each containing 512 neurons and activated by the ReLU function. The last dense layer consists of a single neuron, serving as the output layer for the network. The 2D convolutional neural network (2D-CNN) has an equivalent architecture but with 128 filters for each convolutional layer.

References

[1] G. Carleo, I. Cirac, K. Cranmer, L. Daudet, M. Schuld, N. Tishby, L. Vogt-Maranto, L. Zdeborová, Machine learning and the physical sciences. Rev. Mod. Phys. **91**, 045002 (2019). https://doi.org/10.1103/RevModPhys.91.045002. URL https://link.aps.org/doi/10.1103/RevModPhys.91.045002

[2] J. Carrasquilla, Machine learning for quantum matter. Adv. Phys.: X **5**(1), 1797528 (2020). https://doi.org/10.1080/23746149.2020.1797528. URL https://doi.org/10.1080/23746149.2020.1797528. https://doi.org/10.1080/23746149.2020.1797528

[3] K.T. Schütt, S. Chmiela, O.A. Von Lilienfeld, A. Tkatchenko, K. Tsuda, K.R. Müller, Machine learning meets quantum physics. Lect. Notes Phys. (2020). https://doi.org/https://doi.org/10.1007/978-3-030-40245-7

Convolutional Neural Networks for Sentence Classification

Yoon Kim
New York University
yhk255@nyu.edu

Abstract

We report on a series of experiments with convolutional neural networks (CNN) trained on top of pre-trained word vectors for sentence-level classification tasks. We show that a simple CNN with little hyperparameter tuning and static vectors achieves excellent results on multiple benchmarks. Learning task-specific vectors through fine-tuning offers further gains in performance. We additionally propose a simple modification to the architecture to allow for the use of both task-specific and static vectors. The CNN models discussed herein improve upon the state of the art on 4 out of 7 tasks, which include sentiment analysis and question classification.

1 Introduction

Deep learning models have achieved remarkable results in computer vision (Krizhevsky et al., 2012) and speech recognition (Graves et al., 2013) in recent years. Within natural language processing, much of the work with deep learning methods has involved learning word vector representations through neural language models (Bengio et al., 2003; Yih et al., 2011; Mikolov et al., 2013) and performing composition over the learned word vectors for classification (Collobert et al., 2011). Word vectors, wherein words are projected from a sparse, 1-of-V encoding (here V is the vocabulary size) onto a lower dimensional vector space via a hidden layer, are essentially feature extractors that encode semantic features of words in their dimensions. In such dense representations, semantically close words are likewise close—in euclidean or cosine distance—in the lower dimensional vector space.

Convolutional neural networks (CNN) utilize layers with convolving filters that are applied to local features (LeCun et al., 1998). Originally invented for computer vision, CNN models have subsequently been shown to be effective for NLP and have achieved excellent results in semantic parsing (Yih et al., 2014), search query retrieval (Shen et al., 2014), sentence modeling (Kalchbrenner et al., 2014), and other traditional NLP tasks (Collobert et al., 2011).

In the present work, we train a simple CNN with one layer of convolution on top of word vectors obtained from an unsupervised neural language model. These vectors were trained by Mikolov et al. (2013) on 100 billion words of Google News, and are publicly available.[1] We initially keep the word vectors static and learn only the other parameters of the model. Despite little tuning of hyperparameters, this simple model achieves excellent results on multiple benchmarks, suggesting that the pre-trained vectors are 'universal' feature extractors that can be utilized for various classification tasks. Learning task-specific vectors through fine-tuning results in further improvements. We finally describe a simple modification to the architecture to allow for the use of both pre-trained and task-specific vectors by having multiple channels.

Our work is philosophically similar to Razavian et al. (2014) which showed that for image classification, feature extractors obtained from a pre-trained deep learning model perform well on a variety of tasks—including tasks that are very different from the original task for which the feature extractors were trained.

2 Model

The model architecture, shown in figure 1, is a slight variant of the CNN architecture of Collobert et al. (2011). Let $\mathbf{x}_i \in \mathbb{R}^k$ be the k-dimensional word vector corresponding to the i-th word in the sentence. A sentence of length n (padded where

[1] https://code.google.com/p/word2vec/

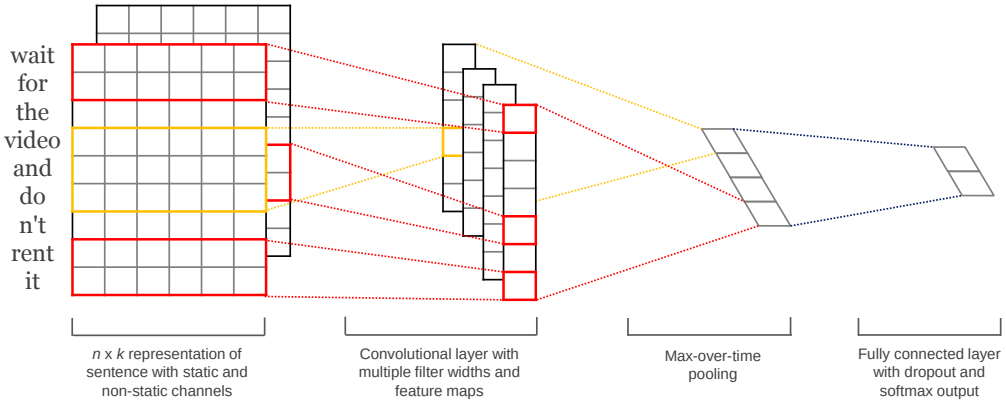


Figure 1: Model architecture with two channels for an example sentence.

necessary) is represented as

$$\mathbf{x}_{1:n} = \mathbf{x}_1 \oplus \mathbf{x}_2 \oplus \ldots \oplus \mathbf{x}_n, \quad (1)$$

where $\oplus$ is the concatenation operator. In general, let $\mathbf{x}_{i:i+j}$ refer to the concatenation of words $\mathbf{x}_i, \mathbf{x}_{i+1}, \ldots, \mathbf{x}_{i+j}$. A convolution operation involves a *filter* $\mathbf{w} \in \mathbb{R}^{hk}$, which is applied to a window of h words to produce a new feature. For example, a feature c_i is generated from a window of words $\mathbf{x}_{i:i+h-1}$ by

$$c_i = f(\mathbf{w} \cdot \mathbf{x}_{i:i+h-1} + b). \quad (2)$$

Here $b \in \mathbb{R}$ is a bias term and f is a non-linear function such as the hyperbolic tangent. This filter is applied to each possible window of words in the sentence $\{\mathbf{x}_{1:h}, \mathbf{x}_{2:h+1}, \ldots, \mathbf{x}_{n-h+1:n}\}$ to produce a *feature map*

$$\mathbf{c} = [c_1, c_2, \ldots, c_{n-h+1}], \quad (3)$$

with $\mathbf{c} \in \mathbb{R}^{n-h+1}$. We then apply a max-over-time pooling operation (Collobert et al., 2011) over the feature map and take the maximum value $\hat{c} = \max\{\mathbf{c}\}$ as the feature corresponding to this particular filter. The idea is to capture the most important feature—one with the highest value—for each feature map. This pooling scheme naturally deals with variable sentence lengths.

We have described the process by which *one* feature is extracted from *one* filter. The model uses multiple filters (with varying window sizes) to obtain multiple features. These features form the penultimate layer and are passed to a fully connected softmax layer whose output is the probability distribution over labels.

In one of the model variants, we experiment with having two 'channels' of word vectors—one that is kept static throughout training and one that is fine-tuned via backpropagation (section 3.2).[2] In the multichannel architecture, illustrated in figure 1, each filter is applied to both channels and the results are added to calculate c_i in equation (2). The model is otherwise equivalent to the single channel architecture.

2.1 Regularization

For regularization we employ dropout on the penultimate layer with a constraint on l_2-norms of the weight vectors (Hinton et al., 2012). Dropout prevents co-adaptation of hidden units by randomly dropping out—i.e., setting to zero—a proportion p of the hidden units during foward-backpropagation. That is, given the penultimate layer $\mathbf{z} = [\hat{c}_1, \ldots, \hat{c}_m]$ (note that here we have m filters), instead of using

$$y = \mathbf{w} \cdot \mathbf{z} + b \quad (4)$$

for output unit y in forward propagation, dropout uses

$$y = \mathbf{w} \cdot (\mathbf{z} \circ \mathbf{r}) + b, \quad (5)$$

where $\circ$ is the element-wise multiplication operator and $\mathbf{r} \in \mathbb{R}^m$ is a 'masking' vector of Bernoulli random variables with probability p of being 1. Gradients are backpropagated only through the unmasked units. At test time, the learned weight vectors are scaled by p such that $\hat{\mathbf{w}} = p\mathbf{w}$, and $\hat{\mathbf{w}}$ is used (without dropout) to score unseen sentences. We additionally constrain l_2-norms of the weight vectors by rescaling $\mathbf{w}$ to have $||\mathbf{w}||_2 = s$ whenever $||\mathbf{w}||_2 > s$ after a gradient descent step.

[2] We employ language from computer vision where a color image has red, green, and blue channels.

Data	c	l	N	$\|V\|$	$\|V_{pre}\|$	*Test*
MR	2	20	10662	18765	16448	CV
SST-1	5	18	11855	17836	16262	2210
SST-2	2	19	9613	16185	14838	1821
Subj	2	23	10000	21323	17913	CV
TREC	6	10	5952	9592	9125	500
CR	2	19	3775	5340	5046	CV
MPQA	2	3	10606	6246	6083	CV

Table 1: Summary statistics for the datasets after tokenization. c: Number of target classes. l: Average sentence length. N: Dataset size. $|V|$: Vocabulary size. $|V_{pre}|$: Number of words present in the set of pre-trained word vectors. *Test*: Test set size (CV means there was no standard train/test split and thus 10-fold CV was used).

3 Datasets and Experimental Setup

We test our model on various benchmarks. Summary statistics of the datasets are in table 1.

- **MR**: Movie reviews with one sentence per review. Classification involves detecting positive/negative reviews (Pang and Lee, 2005).[3]
- **SST-1**: Stanford Sentiment Treebank—an extension of MR but with train/dev/test splits provided and fine-grained labels (very positive, positive, neutral, negative, very negative), re-labeled by Socher et al. (2013).[4]
- **SST-2**: Same as SST-1 but with neutral reviews removed and binary labels.
- **Subj**: Subjectivity dataset where the task is to classify a sentence as being subjective or objective (Pang and Lee, 2004).
- **TREC**: TREC question dataset—task involves classifying a question into 6 question types (whether the question is about person, location, numeric information, etc.) (Li and Roth, 2002).[5]
- **CR**: Customer reviews of various products (cameras, MP3s etc.). Task is to predict positive/negative reviews (Hu and Liu, 2004).[6]
- **MPQA**: Opinion polarity detection subtask of the MPQA dataset (Wiebe et al., 2005).[7]

3.1 Hyperparameters and Training

For all datasets we use: rectified linear units, filter windows (h) of 3, 4, 5 with 100 feature maps each, dropout rate (p) of 0.5, l_2 constraint (s) of 3, and mini-batch size of 50. These values were chosen via a grid search on the SST-2 dev set.

We do not otherwise perform any dataset-specific tuning other than early stopping on dev sets. For datasets without a standard dev set we randomly select 10% of the training data as the dev set. Training is done through stochastic gradient descent over shuffled mini-batches with the Adadelta update rule (Zeiler, 2012).

3.2 Pre-trained Word Vectors

Initializing word vectors with those obtained from an unsupervised neural language model is a popular method to improve performance in the absence of a large supervised training set (Collobert et al., 2011; Socher et al., 2011; Iyyer et al., 2014). We use the publicly available `word2vec` vectors that were trained on 100 billion words from Google News. The vectors have dimensionality of 300 and were trained using the continuous bag-of-words architecture (Mikolov et al., 2013). Words not present in the set of pre-trained words are initialized randomly.

3.3 Model Variations

We experiment with several variants of the model.

- **CNN-rand**: Our baseline model where all words are randomly initialized and then modified during training.
- **CNN-static**: A model with pre-trained vectors from `word2vec`. All words—including the unknown ones that are randomly initialized—are kept static and only the other parameters of the model are learned.
- **CNN-non-static**: Same as above but the pre-trained vectors are fine-tuned for each task.
- **CNN-multichannel**: A model with two sets of word vectors. Each set of vectors is treated as a 'channel' and each filter is applied

[3] https://www.cs.cornell.edu/people/pabo/movie-review-data/

[4] http://nlp.stanford.edu/sentiment/ Data is actually provided at the phrase-level and hence we train the model on both phrases and sentences but only score on sentences at test time, as in Socher et al. (2013), Kalchbrenner et al. (2014), and Le and Mikolov (2014). Thus the training set is an order of magnitude larger than listed in table 1.

[5] http://cogcomp.cs.illinois.edu/Data/QA/QC/

[6] http://www.cs.uic.edu/~liub/FBS/sentiment-analysis.html

[7] http://www.cs.pitt.edu/mpqa/

Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-rand	76.1	45.0	82.7	89.6	91.2	79.8	83.4
CNN-static	81.0	45.5	86.8	93.0	92.8	84.7	**89.6**
CNN-non-static	**81.5**	48.0	87.2	93.4	93.6	84.3	89.5
CNN-multichannel	81.1	47.4	**88.1**	93.2	92.2	**85.0**	89.4
RAE (Socher et al., 2011)	77.7	43.2	82.4	–	–	–	86.4
MV-RNN (Socher et al., 2012)	79.0	44.4	82.9	–	–	–	–
RNTN (Socher et al., 2013)	–	45.7	85.4	–	–	–	–
DCNN (Kalchbrenner et al., 2014)	–	48.5	86.8	–	93.0	–	–
Paragraph-Vec (Le and Mikolov, 2014)	–	**48.7**	87.8	–	–	–	–
CCAE (Hermann and Blunsom, 2013)	77.8	–	–	–	–	–	87.2
Sent-Parser (Dong et al., 2014)	79.5	–	–	–	–	–	86.3
NBSVM (Wang and Manning, 2012)	79.4	–	–	93.2	–	81.8	86.3
MNB (Wang and Manning, 2012)	79.0	–	–	**93.6**	–	80.0	86.3
G-Dropout (Wang and Manning, 2013)	79.0	–	–	93.4	–	82.1	86.1
F-Dropout (Wang and Manning, 2013)	79.1	–	–	**93.6**	–	81.9	86.3
Tree-CRF (Nakagawa et al., 2010)	77.3	–	–	–	–	81.4	86.1
CRF-PR (Yang and Cardie, 2014)	–	–	–	–	–	82.7	–
SVM_S (Silva et al., 2011)	–	–	–	–	**95.0**	–	–

Table 2: Results of our CNN models against other methods. **RAE**: Recursive Autoencoders with pre-trained word vectors from Wikipedia (Socher et al., 2011). **MV-RNN**: Matrix-Vector Recursive Neural Network with parse trees (Socher et al., 2012). **RNTN**: Recursive Neural Tensor Network with tensor-based feature function and parse trees (Socher et al., 2013). **DCNN**: Dynamic Convolutional Neural Network with k-max pooling (Kalchbrenner et al., 2014). **Paragraph-Vec**: Logistic regression on top of paragraph vectors (Le and Mikolov, 2014). **CCAE**: Combinatorial Category Autoencoders with combinatorial category grammar operators (Hermann and Blunsom, 2013). **Sent-Parser**: Sentiment analysis-specific parser (Dong et al., 2014). **NBSVM, MNB**: Naive Bayes SVM and Multinomial Naive Bayes with uni-bigrams from Wang and Manning (2012). **G-Dropout, F-Dropout**: Gaussian Dropout and Fast Dropout from Wang and Manning (2013). **Tree-CRF**: Dependency tree with Conditional Random Fields (Nakagawa et al., 2010). **CRF-PR**: Conditional Random Fields with Posterior Regularization (Yang and Cardie, 2014). **SVM_S**: SVM with uni-bi-trigrams, wh word, head word, POS, parser, hypernyms, and 60 hand-coded rules as features from Silva et al. (2011).

to both channels, but gradients are back-propagated only through one of the channels. Hence the model is able to fine-tune one set of vectors while keeping the other static. Both channels are initialized with `word2vec`.

In order to disentangle the effect of the above variations versus other random factors, we eliminate other sources of randomness—CV-fold assignment, initialization of unknown word vectors, initialization of CNN parameters—by keeping them uniform within each dataset.

4 Results and Discussion

Results of our models against other methods are listed in table 2. Our baseline model with all randomly initialized words (CNN-rand) does not perform well on its own. While we had expected performance gains through the use of pre-trained vectors, we were surprised at the magnitude of the gains. Even a simple model with static vectors (CNN-static) performs remarkably well, giving competitive results against the more sophisticated deep learning models that utilize complex pooling schemes (Kalchbrenner et al., 2014) or require parse trees to be computed beforehand (Socher et al., 2013). These results suggest that the pre-trained vectors are good, 'universal' feature extractors and can be utilized across datasets. Fine-tuning the pre-trained vectors for each task gives still further improvements (CNN-non-static).

4.1 Multichannel vs. Single Channel Models

We had initially hoped that the multichannel architecture would prevent overfitting (by ensuring that the learned vectors do not deviate too far from the original values) and thus work better than the single channel model, especially on smaller datasets. The results, however, are mixed, and further work on regularizing the fine-tuning process is warranted. For instance, instead of using an additional channel for the non-static portion, one could maintain a single channel but employ extra dimensions that are allowed to be modified during training.

	Most Similar Words for	
	Static Channel	Non-static Channel
bad	*good*	*terrible*
	terrible	*horrible*
	horrible	*lousy*
	lousy	*stupid*
good	*great*	*nice*
	bad	*decent*
	terrific	*solid*
	decent	*terrific*
n't	*os*	*not*
	ca	*never*
	ireland	*nothing*
	wo	*neither*
!	*2,500*	*2,500*
	entire	*lush*
	jez	*beautiful*
	changer	*terrific*
,	*decasia*	*but*
	abysmally	*dragon*
	demise	*a*
	valiant	*and*

Table 3: Top 4 neighboring words—based on cosine similarity—for vectors in the static channel (left) and fine-tuned vectors in the non-static channel (right) from the multichannel model on the SST-2 dataset after training.

4.2 Static vs. Non-static Representations

As is the case with the single channel non-static model, the multichannel model is able to fine-tune the non-static channel to make it more specific to the task-at-hand. For example, *good* is most similar to *bad* in `word2vec`, presumably because they are (almost) syntactically equivalent. But for vectors in the non-static channel that were fine-tuned on the SST-2 dataset, this is no longer the case (table 3). Similarly, *good* is arguably closer to *nice* than it is to *great* for expressing sentiment, and this is indeed reflected in the learned vectors.

For (randomly initialized) tokens not in the set of pre-trained vectors, fine-tuning allows them to learn more meaningful representations: the network learns that exclamation marks are associated with effusive expressions and that commas are conjunctive (table 3).

4.3 Further Observations

We report on some further experiments and observations:

- Kalchbrenner et al. (2014) report much worse results with a CNN that has essentially the same architecture as our single channel model. For example, their Max-TDNN (Time Delay Neural Network) with randomly initialized words obtains 37.4% on the SST-1 dataset, compared to 45.0% for our model. We attribute such discrepancy to our CNN having much more capacity (multiple filter widths and feature maps).
- Dropout proved to be such a good regularizer that it was fine to use a larger than necessary network and simply let dropout regularize it. Dropout consistently added 2%–4% relative performance.
- When randomly initializing words not in `word2vec`, we obtained slight improvements by sampling each dimension from $U[-a, a]$ where a was chosen such that the randomly initialized vectors have the same variance as the pre-trained ones. It would be interesting to see if employing more sophisticated methods to mirror the distribution of pre-trained vectors in the initialization process gives further improvements.
- We briefly experimented with another set of publicly available word vectors trained by Collobert et al. (2011) on Wikipedia,[8] and found that `word2vec` gave far superior performance. It is not clear whether this is due to Mikolov et al. (2013)'s architecture or the 100 billion word Google News dataset.
- Adadelta (Zeiler, 2012) gave similar results to Adagrad (Duchi et al., 2011) but required fewer epochs.

5 Conclusion

In the present work we have described a series of experiments with convolutional neural networks built on top of `word2vec`. Despite little tuning of hyperparameters, a simple CNN with one layer of convolution performs remarkably well. Our results add to the well-established evidence that unsupervised pre-training of word vectors is an important ingredient in deep learning for NLP.

Acknowledgments

We would like to thank Yann LeCun and the anonymous reviewers for their helpful feedback and suggestions.

[8] http://ronan.collobert.com/senna/

1D Convolutional Neural Network Models for Human Activity Recognition

by **Jason Brownlee** on August 28, 2020 in **Deep Learning for Time Series** 236

Human activity recognition is the problem of classifying sequences of accelerometer data recorded by specialized harnesses or smart phones into known well-defined movements.

Classical approaches to the problem involve hand crafting features from the time series data based on fixed-sized windows and training machine learning models, such as ensembles of decision trees. The difficulty is that this feature engineering requires deep expertise in the field.

Recently, deep learning methods such as recurrent neural networks and one-dimensional convolutional neural networks, or CNNs, have been shown to provide state-of-the-art results on challenging activity recognition tasks with little or no data feature engineering, instead using feature learning on raw data.

In this tutorial, you will discover how to develop one-dimensional convolutional neural networks for time series classification on the problem of human activity recognition.

After completing this tutorial, you will know:

- How to load and prepare the data for a standard human activity recognition dataset and develop a single 1D CNN model that achieves excellent performance on the raw data.
- How to further tune the performance of the model, including data transformation, filter maps, and kernel sizes.
- How to develop a sophisticated multi-headed one-dimensional convolutional neural network model that provides an ensemble-like result.

Kick-start your project with my new book Deep Learning for Time Series Forecasting, including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.

How to Develop 1D Convolutional Neural Network Models for Human Activity Recognition
Photo by Wolfgang Staudt, some rights reserved.

Tutorial Overview

This tutorial is divided into four parts; they are:

1. Activity Recognition Using Smartphones Dataset
2. Develop 1D Convolutional Neural Network
3. Tuned 1D Convolutional Neural Network
4. Multi-Headed 1D Convolutional Neural Network

Activity Recognition Using Smartphones Dataset

Human Activity Recognition, or HAR for short, is the problem of predicting what a person is doing based on a trace of their movement using sensors.

A standard human activity recognition dataset is the 'Activity Recognition Using Smart Phones Dataset' made available in 2012.

It was prepared and made available by Davide Anguita, et al. from the University of Genova, Italy and is described in full in their 2013 paper "A Public Domain Dataset for Human Activity Recognition Using Smartphones." The dataset was modeled with machine learning algorithms in their 2012 paper titled "Human Activity Recognition on Smartphones using a Multiclass Hardware-Friendly Support Vector Machine."

The dataset was made available and can be downloaded for free from the UCI Machine Learning Repository:

- Human Activity Recognition Using Smartphones Data Set, UCI Machine Learning Repository

The data was collected from 30 subjects aged between 19 and 48 years old performing one of six standard activities while wearing a waist-mounted smartphone that recorded the movement data. Video was recorded of each subject performing the activities and the movement data was labeled manually from these videos.

Below is an example video of a subject performing the activities while their movement data is being recorded.

The six activities performed were as follows:

1. Walking
2. Walking Upstairs
3. Walking Downstairs
4. Sitting
5. Standing
6. Laying

The movement data recorded was the x, y, and z accelerometer data (linear acceleration) and gyroscopic data (angular velocity) from the smart phone, specifically a Samsung Galaxy S II. Observations were recorded at 50 Hz (i.e. 50 data points per second). Each subject performed the sequence of activities twice, once with the device on their left-hand-side and once with the device on their right-hand side.

The raw data is not available. Instead, a pre-processed version of the dataset was made available. The pre-processing steps included:

- Pre-processing accelerometer and gyroscope using noise filters.
- Splitting data into fixed windows of 2.56 seconds (128 data points) with 50% overlap.
- Splitting of accelerometer data into gravitational (total) and body motion components.

Feature engineering was applied to the window data, and a copy of the data with these engineered features was made available.

A number of time and frequency features commonly used in the field of human activity recognition were extracted from each window. The result was a 561 element vector of features.

The dataset was split into train (70%) and test (30%) sets based on data for subjects, e.g. 21 subjects for train and nine for test.

Experiment results with a support vector machine intended for use on a smartphone (e.g. fixed-point arithmetic) resulted in a predictive accuracy of 89% on the test dataset, achieving similar results as an unmodified SVM implementation.

The dataset is freely available and can be downloaded from the UCI Machine Learning repository.

The data is provided as a single zip file that is about 58 megabytes in size. The direct link for this download is below:

- UCI HAR Dataset.zip

Download the dataset and unzip all files into a new directory in your current working directory named "HARDataset".

Develop 1D Convolutional Neural Network

In this section, we will develop a one-dimensional convolutional neural network model (1D CNN) for the human activity recognition dataset.

Convolutional neural network models were developed for image classification problems, where the model learns an internal representation of a two-dimensional input, in a process referred to as feature learning.

This same process can be harnessed on one-dimensional sequences of data, such as in the case of acceleration and gyroscopic data for human activity recognition. The model learns to extract features from sequences of observations and how to map the internal features to different activity types.

The benefit of using CNNs for sequence classification is that they can learn from the raw time series data directly, and in turn do not require domain expertise to manually engineer input features. The model can learn an internal representation of the time series data and ideally achieve comparable performance to models fit on a version of the dataset with engineered features.

This section is divided into 4 parts; they are:

1. Load Data
2. Fit and Evaluate Model
3. Summarize Results
4. Complete Example

Load Data

The first step is to load the raw dataset into memory.

There are three main signal types in the raw data: total acceleration, body acceleration, and body gyroscope. Each has three axes of data. This means that there are a total of nine variables for each time step.

Further, each series of data has been partitioned into overlapping windows of 2.65 seconds of data, or 128 time steps. These windows of data correspond to the windows of engineered features (rows) in the previous section.

This means that one row of data has (128 * 9), or 1,152, elements. This is a little less than double the size of the 561 element vectors in the previous section and it is likely that there is some redundant data.

The signals are stored in the */Inertial Signals/* directory under the train and test subdirectories. Each axis of each signal is stored in a separate file, meaning that each of the train and test datasets have nine input files to load and one output file to load. We can batch the loading of these files into groups given the consistent directory structures and file naming conventions.

The input data is in CSV format where columns are separated by whitespace. Each of these files can be loaded as a NumPy array. The *load_file()* function below loads a dataset given the file path to the file and returns the loaded data as a NumPy array.

```
# load a single file as a numpy array
def load_file(filepath):
    dataframe = read_csv(filepath, header=None, delim_whitespace=True)
    return dataframe.values
```

We can then load all data for a given group (train or test) into a single three-dimensional NumPy array, where the dimensions of the array are [*samples, time steps, features*].

To make this clearer, there are 128 time steps and nine features, where the number of samples is the number of rows in any given raw signal data file.

The *load_group()* function below implements this behavior. The dstack() NumPy function allows us to stack each of the loaded 3D arrays into a single 3D array where the variables are separated on the third dimension (features).

```
# load a list of files into a 3D array of [samples, timesteps, features]
def load_group(filenames, prefix=''):
    loaded = list()
    for name in filenames:
        data = load_file(prefix + name)
        loaded.append(data)
    # stack group so that features are the 3rd dimension
    loaded = dstack(loaded)
    return loaded
```

We can use this function to load all input signal data for a given group, such as train or test.

The *load_dataset_group()* function below loads all input signal data and the output data for a single group using the consistent naming conventions between the train and test directories.

```
# load a dataset group, such as train or test
def load_dataset_group(group, prefix=''):
    filepath = prefix + group + '/Inertial Signals/'
    # load all 9 files as a single array
    filenames = list()
    # total acceleration
    filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt', 'total_acc_z_'+group+'.txt']
    # body acceleration
    filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt', 'body_acc_z_'+group+'.txt']
    # body gyroscope
    filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt', 'body_gyro_z_'+group+'.txt']
    # load input data
    X = load_group(filenames, filepath)
    # load class output
    y = load_file(prefix + group + '/y_'+group+'.txt')
    return X, y
```

Finally, we can load each of the train and test datasets.

The output data is defined as an integer for the class number. We must one hot encode these class integers so that the data is suitable for fitting a neural network multi-class classification model. We can do this by calling the to_categorical() Keras function.

The *load_dataset()* function below implements this behavior and returns the train and test *X* and *y* elements ready for fitting and evaluating the defined models.

```
# load the dataset, returns train and test X and y elements
def load_dataset(prefix=''):
    # load all train
    trainX, trainy = load_dataset_group('train', prefix + 'HARDataset/')
    print(trainX.shape, trainy.shape)
    # load all test
    testX, testy = load_dataset_group('test', prefix + 'HARDataset/')
    print(testX.shape, testy.shape)
    # zero-offset class values
    trainy = trainy - 1
    testy = testy - 1
    # one hot encode y
    trainy = to_categorical(trainy)
    testy = to_categorical(testy)
    print(trainX.shape, trainy.shape, testX.shape, testy.shape)
    return trainX, trainy, testX, testy
```

Fit and Evaluate Model

Now that we have the data loaded into memory ready for modeling, we can define, fit, and evaluate a 1D CNN model.

We can define a function named *evaluate_model()* that takes the train and test dataset, fits a model on the training dataset, evaluates it on the test dataset, and returns an estimate of the models performance.

First, we must define the CNN model using the Keras deep learning library. The model requires a three-dimensional input with [*samples, time steps, features*].

This is exactly how we have loaded the data, where one sample is one window of the time series data, each window has 128 time steps, and a time step has nine variables or features.

The output for the model will be a six-element vector containing the probability of a given window belonging to each of the six activity types.

These input and output dimensions are required when fitting the model, and we can extract them from the provided training dataset.

```
n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
```

The model is defined as a Sequential Keras model, for simplicity.

We will define the model as having two 1D CNN layers, followed by a dropout layer for regularization, then a pooling layer. It is common to define CNN layers in groups of two in order to give the model a good chance of learning features from the input data. CNNs learn very quickly, so the dropout layer is intended to help slow down the learning process and hopefully result in a better final model. The pooling layer reduces the learned features to 1/4 their size, consolidating them to only the most essential elements.

After the CNN and pooling, the learned features are flattened to one long vector and pass through a fully connected layer before the output layer used to make a prediction. The fully connected layer ideally provides a buffer between the learned features and the output with the intent of interpreting the learned features before making a prediction.

For this model, we will use a standard configuration of 64 parallel feature maps and a kernel size of 3. The feature maps are the number of times the input is processed or interpreted, whereas the kernel size is the number of input time steps considered as the input sequence is read or processed onto the feature maps.

The efficient Adam version of stochastic gradient descent will be used to optimize the network, and the categorical cross entropy loss function will be used given that we are learning a multi-class classification problem.

The definition of the model is listed below.

```
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(n_timesteps,n_features)))
```

```
model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
model.add(Dropout(0.5))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(100, activation='relu'))
model.add(Dense(n_outputs, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

The model is fit for a fixed number of epochs, in this case 10, and a batch size of 32 samples will be used, where 32 windows of data will be exposed to the model before the weights of the model are updated.

Once the model is fit, it is evaluated on the test dataset and the accuracy of the fit model on the test dataset is returned.

The complete *evaluate_model()* function is listed below.

```
# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy):
    verbose, epochs, batch_size = 0, 10, 32
    n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
    model = Sequential()
    model.add(Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(n_timesteps,n_features)))
    model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
    model.add(Dropout(0.5))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Flatten())
    model.add(Dense(100, activation='relu'))
    model.add(Dense(n_outputs, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # fit network
    model.fit(trainX, trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
    # evaluate model
    _, accuracy = model.evaluate(testX, testy, batch_size=batch_size, verbose=0)
    return accuracy
```

There is nothing special about the network structure or chosen hyperparameters; they are just a starting point for this problem.

Summarize Results

We cannot judge the skill of the model from a single evaluation.

The reason for this is that neural networks are stochastic, meaning that a different specific model will result when training the same model configuration on the same data.

This is a feature of the network in that it gives the model its adaptive ability, but requires a slightly more complicated evaluation of the model.

We will repeat the evaluation of the model multiple times, then summarize the performance of the model across each of those runs. For example, we can call *evaluate_model()* a total of 10 times. This will result in a population of model evaluation scores that must be summarized.

```
# repeat experiment
scores = list()
for r in range(repeats):
    score = evaluate_model(trainX, trainy, testX, testy)
    score = score * 100.0
    print('>#%d: %.3f' % (r+1, score))
    scores.append(score)
```

We can summarize the sample of scores by calculating and reporting the mean and standard deviation of the performance. The mean gives the average accuracy of the model on the dataset, whereas the standard deviation gives the average variance of the accuracy from the mean.

The function *summarize_results()* below summarizes the results of a run.

```
# summarize scores
def summarize_results(scores):
    print(scores)
    m, s = mean(scores), std(scores)
    print('Accuracy: %.3f%% (+/-%.3f)' % (m, s))
```

We can bundle up the repeated evaluation, gathering of results, and summarization of results into a main function for the experiment, called *run_experiment()*, listed below.

By default, the model is evaluated 10 times before the performance of the model is reported.

```
# run an experiment
def run_experiment(repeats=10):
    # load data
    trainX, trainy, testX, testy = load_dataset()
    # repeat experiment
    scores = list()
    for r in range(repeats):
        score = evaluate_model(trainX, trainy, testX, testy)
        score = score * 100.0
        print('>#%d: %.3f' % (r+1, score))
        scores.append(score)
    # summarize results
    summarize_results(scores)
```

Complete Example

Now that we have all of the pieces, we can tie them together into a worked example.

The complete code listing is provided below.

```python
# cnn model
from numpy import mean
from numpy import std
from numpy import dstack
from pandas import read_csv
from matplotlib import pyplot
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Flatten
from keras.layers import Dropout
from keras.layers.convolutional import Conv1D
from keras.layers.convolutional import MaxPooling1D
from keras.utils import to_categorical

# load a single file as a numpy array
def load_file(filepath):
	dataframe = read_csv(filepath, header=None, delim_whitespace=True)
	return dataframe.values

# load a list of files and return as a 3d numpy array
def load_group(filenames, prefix=''):
	loaded = list()
	for name in filenames:
		data = load_file(prefix + name)
		loaded.append(data)
	# stack group so that features are the 3rd dimension
	loaded = dstack(loaded)
	return loaded

# load a dataset group, such as train or test
def load_dataset_group(group, prefix=''):
	filepath = prefix + group + '/Inertial Signals/'
	# load all 9 files as a single array
	filenames = list()
	# total acceleration
	filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt', 'total_acc_z_'+group+'.txt']
	# body acceleration
	filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt', 'body_acc_z_'+group+'.txt']
	# body gyroscope
	filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt', 'body_gyro_z_'+group+'.txt']
	# load input data
	X = load_group(filenames, filepath)
	# load class output
	y = load_file(prefix + group + '/y_'+group+'.txt')
	return X, y

# load the dataset, returns train and test X and y elements
def load_dataset(prefix=''):
	# load all train
	trainX, trainy = load_dataset_group('train', prefix + 'HARDataset/')
	print(trainX.shape, trainy.shape)
	# load all test
	testX, testy = load_dataset_group('test', prefix + 'HARDataset/')
	print(testX.shape, testy.shape)
	# zero-offset class values
	trainy = trainy - 1
	testy = testy - 1
	# one hot encode y
	trainy = to_categorical(trainy)
	testy = to_categorical(testy)
	print(trainX.shape, trainy.shape, testX.shape, testy.shape)
	return trainX, trainy, testX, testy

# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy):
	verbose, epochs, batch_size = 0, 10, 32
	n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
	model = Sequential()
	model.add(Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(n_timesteps,n_features)))
	model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
	model.add(Dropout(0.5))
	model.add(MaxPooling1D(pool_size=2))
	model.add(Flatten())
	model.add(Dense(100, activation='relu'))
	model.add(Dense(n_outputs, activation='softmax'))
	model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
	# fit network
	model.fit(trainX, trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
	# evaluate model
	_, accuracy = model.evaluate(testX, testy, batch_size=batch_size, verbose=0)
	return accuracy

# summarize scores
def summarize_results(scores):
	print(scores)
	m, s = mean(scores), std(scores)
	print('Accuracy: %.3f%% (+/-%.3f)' % (m, s))

# run an experiment
def run_experiment(repeats=10):
	# load data
	trainX, trainy, testX, testy = load_dataset()
	# repeat experiment
	scores = list()
	for r in range(repeats):
		score = evaluate_model(trainX, trainy, testX, testy)
		score = score * 100.0
		print('>#%d: %.3f' % (r+1, score))
		scores.append(score)
	# summarize results
	summarize_results(scores)

# run the experiment
run_experiment()
```

Running the example first prints the shape of the loaded dataset, then the shape of the train and test sets and the input and output elements. This confirms the number of

samples, time steps, and variables, as well as the number of classes.

Next, models are created and evaluated and a debug message is printed for each.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Finally, the sample of scores is printed followed by the mean and standard deviation. We can see that the model performed well achieving a classification accuracy of about 90.9% trained on the raw dataset, with a standard deviation of about 1.3.

This is a good result, considering that the original paper published a result of 89%, trained on the dataset with heavy domain-specific feature engineering, not the raw dataset.

```
(7352, 128, 9) (7352, 1)
(2947, 128, 9) (2947, 1)
(7352, 128, 9) (7352, 6) (2947, 128, 9) (2947, 6)

>#1: 91.347
>#2: 91.551
>#3: 90.804
>#4: 90.058
>#5: 89.752
>#6: 90.940
>#7: 91.347
>#8: 87.547
>#9: 92.637
>#10: 91.890

[91.34713267729894, 91.55072955548015, 90.80420766881574, 90.05768578215134, 89.75229046487954, 90.93993892093654, 91.34713267729894, 87.54665761791652, 92.636

Accuracy: 90.787% (+/-1.341)
```

Now that we have seen how to load the data and fit a 1D CNN model, we can investigate whether we can further lift the skill of the model with some hyperparameter tuning.

Tuned 1D Convolutional Neural Network

In this section, we will tune the model in an effort to further improve performance on the problem.

We will look at three main areas:

1. Data Preparation
2. Number of Filters
3. Size of Kernel

Data Preparation

In the previous section, we did not perform any data preparation. We used the data as-is.

Each of the main sets of data (body acceleration, body gyroscopic, and total acceleration) have been scaled to the range -1, 1. It is not clear if the data was scaled per-subject or across all subjects.

One possible transform that may result in an improvement is to standardize the observations prior to fitting a model.

Standardization refers to shifting the distribution of each variable such that it has a mean of zero and a standard deviation of 1. It really only makes sense if the distribution of each variable is Gaussian.

We can quickly check the distribution of each variable by plotting a histogram of each variable in the training dataset.

A minor difficulty in this is that the data has been split into windows of 128 time steps, with a 50% overlap. Therefore, in order to get a fair idea of the data distribution, we must first remove the duplicated observations (the overlap), then remove the windowing of the data.

We can do this using NumPy, first slicing the array and only keeping the second half of each window, then flattening the windows into a long vector for each variable. This is quick and dirty and does mean that we lose the data in the first half of the first window.

```
# remove overlap
cut = int(trainX.shape[1] / 2)
longX = trainX[:, -cut:, :]
# flatten windows
longX = longX.reshape((longX.shape[0] * longX.shape[1], longX.shape[2]))
```

The complete example of loading the data, flattening it, and plotting a histogram for each of the nine variables is listed below.

```
# plot distributions
from numpy import dstack
from pandas import read_csv
from keras.utils import to_categorical
from matplotlib import pyplot

# load a single file as a numpy array
def load_file(filepath):
    dataframe = read_csv(filepath, header=None, delim_whitespace=True)
    return dataframe.values

```

```
# load a list of files and return as a 3d numpy array
def load_group(filenames, prefix=''):
    loaded = list()
    for name in filenames:
        data = load_file(prefix + name)
        loaded.append(data)
    # stack group so that features are the 3rd dimension
    loaded = dstack(loaded)
    return loaded

# load a dataset group, such as train or test
def load_dataset_group(group, prefix=''):
    filepath = prefix + group + '/Inertial Signals/'
    # load all 9 files as a single array
    filenames = list()
    # total acceleration
    filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt', 'total_acc_z_'+group+'.txt']
    # body acceleration
    filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt', 'body_acc_z_'+group+'.txt']
    # body gyroscope
    filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt', 'body_gyro_z_'+group+'.txt']
    # load input data
    X = load_group(filenames, filepath)
    # load class output
    y = load_file(prefix + group + '/y_'+group+'.txt')
    return X, y

# load the dataset, returns train and test X and y elements
def load_dataset(prefix=''):
    # load all train
    trainX, trainy = load_dataset_group('train', prefix + 'HARDataset/')
    print(trainX.shape, trainy.shape)
    # load all test
    testX, testy = load_dataset_group('test', prefix + 'HARDataset/')
    print(testX.shape, testy.shape)
    # zero-offset class values
    trainy = trainy - 1
    testy = testy - 1
    # one hot encode y
    trainy = to_categorical(trainy)
    testy = to_categorical(testy)
    print(trainX.shape, trainy.shape, testX.shape, testy.shape)
    return trainX, trainy, testX, testy

# plot a histogram of each variable in the dataset
def plot_variable_distributions(trainX):
    # remove overlap
    cut = int(trainX.shape[1] / 2)
    longX = trainX[:, -cut:, :]
    # flatten windows
    longX = longX.reshape((longX.shape[0] * longX.shape[1], longX.shape[2]))
    print(longX.shape)
    pyplot.figure()
    xaxis = None
    for i in range(longX.shape[1]):
        ax = pyplot.subplot(longX.shape[1], 1, i+1, sharex=xaxis)
        ax.set_xlim(-1, 1)
        if i == 0:
            xaxis = ax
        pyplot.hist(longX[:, i], bins=100)
    pyplot.show()

# load data
trainX, trainy, testX, testy = load_dataset()
# plot histograms
plot_variable_distributions(trainX)
```

Running the example creates a figure with nine histogram plots, one for each variable in the training dataset.

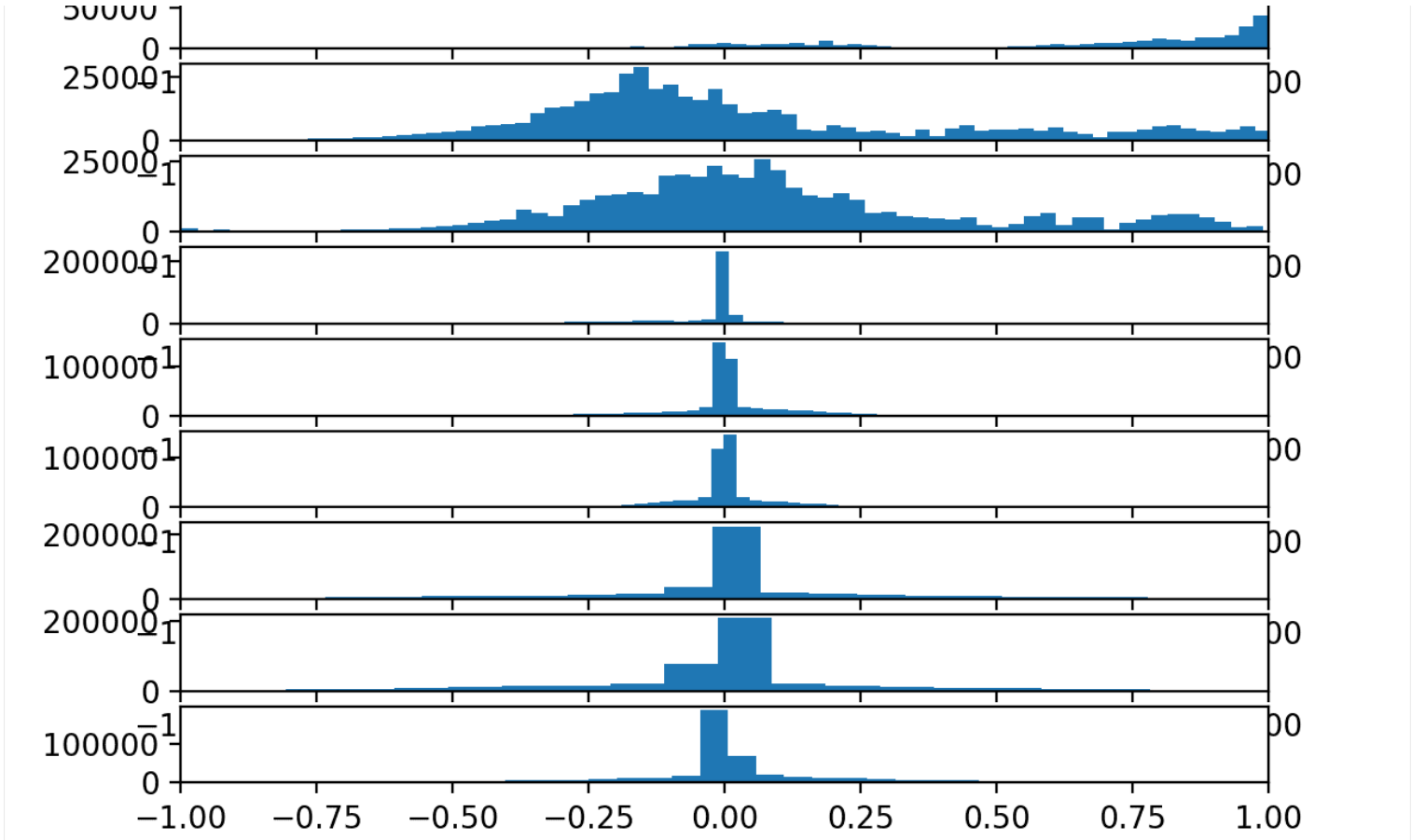
The order of the plots matches the order in which the data was loaded, specifically:

1. Total Acceleration x
2. Total Acceleration y
3. Total Acceleration z
4. Body Acceleration x
5. Body Acceleration y
6. Body Acceleration z
7. Body Gyroscope x
8. Body Gyroscope y
9. Body Gyroscope z

We can see that each variable has a Gaussian-like distribution, except perhaps the first variable (Total Acceleration x).

The distributions of total acceleration data is flatter than the body data, which is more pointed.

We could explore using a power transform on the data to make the distributions more Gaussian, although this is left as an exercise.



Histograms of each variable in the training data set

The data is sufficiently Gaussian-like to explore whether a standardization transform will help the model extract salient signal from the raw observations.

The function below named *scale_data()* can be used to standardize the data prior to fitting and evaluating the model. The StandardScaler scikit-learn class will be used to perform the transform. It is first fit on the training data (e.g. to find the mean and standard deviation for each variable), then applied to the train and test sets.

The standardization is optional, so we can apply the process and compare the results to the same code path without the standardization in a controlled experiment.

```
# standardize data
def scale_data(trainX, testX, standardize):
    # remove overlap
    cut = int(trainX.shape[1] / 2)
    longX = trainX[:, -cut:, :]
    # flatten windows
    longX = longX.reshape((longX.shape[0] * longX.shape[1], longX.shape[2]))
    # flatten train and test
    flatTrainX = trainX.reshape((trainX.shape[0] * trainX.shape[1], trainX.shape[2]))
    flatTestX = testX.reshape((testX.shape[0] * testX.shape[1], testX.shape[2]))
    # standardize
    if standardize:
        s = StandardScaler()
        # fit on training data
        s.fit(longX)
        # apply to training and test data
        longX = s.transform(longX)
        flatTrainX = s.transform(flatTrainX)
        flatTestX = s.transform(flatTestX)
    # reshape
    flatTrainX = flatTrainX.reshape((trainX.shape))
    flatTestX = flatTestX.reshape((testX.shape))
    return flatTrainX, flatTestX
```

We can update the *evaluate_model()* function to take a parameter, then use this parameter to decide whether or not to perform the standardization.

```
# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy, param):
    verbose, epochs, batch_size = 0, 10, 32
    n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
    # scale data
    trainX, testX = scale_data(trainX, testX, param)
    model = Sequential()
    model.add(Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(n_timesteps,n_features)))
    model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
    model.add(Dropout(0.5))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Flatten())
    model.add(Dense(100, activation='relu'))
    model.add(Dense(n_outputs, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
    # fit network
    model.fit(trainX, trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
    # evaluate model
    _, accuracy = model.evaluate(testX, testy, batch_size=batch_size, verbose=0)
    return accuracy
```

We can also update the *run_experiment()* to repeat the experiment 10 times for each parameter; in this case, only two parameters will be evaluated [*False, True*] for no standardization and standardization respectively.

```
# run an experiment
def run_experiment(params, repeats=10):
    # load data
    trainX, trainy, testX, testy = load_dataset()
    # test each parameter
    all_scores = list()
    for p in params:
        # repeat experiment
        scores = list()
        for r in range(repeats):
            score = evaluate_model(trainX, trainy, testX, testy, p)
            score = score * 100.0
            print('>p=%d #%d: %.3f' % (p, r+1, score))
            scores.append(score)
        all_scores.append(scores)
    # summarize results
    summarize_results(all_scores, params)
```

This will result in two samples of results that can be compared.

We will update the *summarize_results()* function to summarize the sample of results for each configuration parameter and to create a boxplot to compare each sample of results.

```
# summarize scores
def summarize_results(scores, params):
    print(scores, params)
    # summarize mean and standard deviation
    for i in range(len(scores)):
        m, s = mean(scores[i]), std(scores[i])
        print('Param=%d: %.3f%% (+/-%.3f)' % (params[i], m, s))
    # boxplot of scores
    pyplot.boxplot(scores, labels=params)
    pyplot.savefig('exp_cnn_standardize.png')
```

These updates will allow us to directly compare the results of a model fit as before and a model fit on the dataset after it has been standardized.

It is also a generic change that will allow us to evaluate and compare the results of other sets of parameters in the following sections.

The complete code listing is provided below.

```
# cnn model with standardization
from numpy import mean
from numpy import std
from numpy import dstack
from pandas import read_csv
from matplotlib import pyplot
from sklearn.preprocessing import StandardScaler
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Flatten
from keras.layers import Dropout
from keras.layers.convolutional import Conv1D
from keras.layers.convolutional import MaxPooling1D
from keras.utils import to_categorical

# load a single file as a numpy array
def load_file(filepath):
    dataframe = read_csv(filepath, header=None, delim_whitespace=True)
    return dataframe.values

# load a list of files and return as a 3d numpy array
def load_group(filenames, prefix=''):
    loaded = list()
    for name in filenames:
        data = load_file(prefix + name)
        loaded.append(data)
    # stack group so that features are the 3rd dimension
    loaded = dstack(loaded)
    return loaded

# load a dataset group, such as train or test
def load_dataset_group(group, prefix=''):
    filepath = prefix + group + '/Inertial Signals/'
    # load all 9 files as a single array
    filenames = list()
    # total acceleration
    filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt', 'total_acc_z_'+group+'.txt']
    # body acceleration
    filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt', 'body_acc_z_'+group+'.txt']
    # body gyroscope
    filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt', 'body_gyro_z_'+group+'.txt']
    # load input data
    X = load_group(filenames, filepath)
    # load class output
    y = load_file(prefix + group + '/y_'+group+'.txt')
    return X, y

# load the dataset, returns train and test X and y elements
def load_dataset(prefix=''):
    # load all train
```

```
    trainX, trainy = load_dataset_group('train', prefix + 'HARDataset/')
    print(trainX.shape, trainy.shape)
    # load all test
    testX, testy = load_dataset_group('test', prefix + 'HARDataset/')
    print(testX.shape, testy.shape)
    # zero-offset class values
    trainy = trainy - 1
    testy = testy - 1
    # one hot encode y
    trainy = to_categorical(trainy)
    testy = to_categorical(testy)
    print(trainX.shape, trainy.shape, testX.shape, testy.shape)
    return trainX, trainy, testX, testy

# standardize data
def scale_data(trainX, testX, standardize):
    # remove overlap
    cut = int(trainX.shape[1] / 2)
    longX = trainX[:, -cut:, :]
    # flatten windows
    longX = longX.reshape((longX.shape[0] * longX.shape[1], longX.shape[2]))
    # flatten train and test
    flatTrainX = trainX.reshape((trainX.shape[0] * trainX.shape[1], trainX.shape[2]))
    flatTestX = testX.reshape((testX.shape[0] * testX.shape[1], testX.shape[2]))
    # standardize
    if standardize:
        s = StandardScaler()
        # fit on training data
        s.fit(longX)
        # apply to training and test data
        longX = s.transform(longX)
        flatTrainX = s.transform(flatTrainX)
        flatTestX = s.transform(flatTestX)
    # reshape
    flatTrainX = flatTrainX.reshape((trainX.shape))
    flatTestX = flatTestX.reshape((testX.shape))
    return flatTrainX, flatTestX

# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy, param):
    verbose, epochs, batch_size = 0, 10, 32
    n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
    # scale data
    trainX, testX = scale_data(trainX, testX, param)
    model = Sequential()
    model.add(Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(n_timesteps,n_features)))
    model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
    model.add(Dropout(0.5))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Flatten())
    model.add(Dense(100, activation='relu'))
    model.add(Dense(n_outputs, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # fit network
    model.fit(trainX, trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
    # evaluate model
    _, accuracy = model.evaluate(testX, testy, batch_size=batch_size, verbose=0)
    return accuracy

# summarize scores
def summarize_results(scores, params):
    print(scores, params)
    # summarize mean and standard deviation
    for i in range(len(scores)):
        m, s = mean(scores[i]), std(scores[i])
        print('Param=%s: %.3f%% (+/-%.3f)' % (params[i], m, s))
    # boxplot of scores
    pyplot.boxplot(scores, labels=params)
    pyplot.savefig('exp_cnn_standardize.png')

# run an experiment
def run_experiment(params, repeats=10):
    # load data
    trainX, trainy, testX, testy = load_dataset()
    # test each parameter
    all_scores = list()
    for p in params:
        # repeat experiment
        scores = list()
        for r in range(repeats):
            score = evaluate_model(trainX, trainy, testX, testy, p)
            score = score * 100.0
            print('>p=%s #%d: %.3f' % (p, r+1, score))
            scores.append(score)
        all_scores.append(scores)
    # summarize results
    summarize_results(all_scores, params)

# run the experiment
n_params = [False, True]
run_experiment(n_params)
```

Running the example may take a few minutes, depending on your hardware.

The performance is printed for each evaluated model. At the end of the run, the performance of each of the tested configurations is summarized showing the mean and the standard deviation.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

We can see that it does look like standardizing the dataset prior to modeling does result in a small lift in performance from about 90.4% accuracy (close to what we saw in the

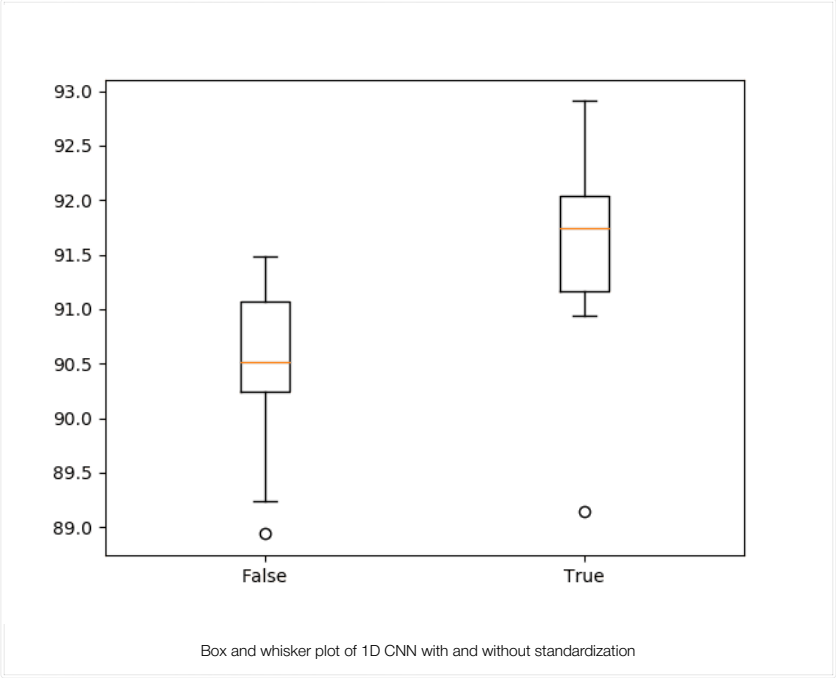
previous section) to about 91.5% accuracy.

```
(7352, 128, 9) (7352, 1)
(2947, 128, 9) (2947, 1)
(7352, 128, 9) (7352, 6) (2947, 128, 9) (2947, 6)

>p=False #1: 91.483
>p=False #2: 91.245
>p=False #3: 90.838
>p=False #4: 89.243
>p=False #5: 90.193
>p=False #6: 90.465
>p=False #7: 90.397
>p=False #8: 90.567
>p=False #9: 88.938
>p=False #10: 91.144
>p=True #1: 92.908
>p=True #2: 90.940
>p=True #3: 92.297
>p=True #4: 91.822
>p=True #5: 92.094
>p=True #6: 91.313
>p=True #7: 91.653
>p=True #8: 89.141
>p=True #9: 91.110
>p=True #10: 91.890

[[91.48286392941975, 91.24533423820834, 90.83814048184594, 89.24329826942655, 90.19341703427214, 90.46487953851374, 90.39701391245333, 90.56667797760434, 88.93

Param=False: 90.451% (+/-0.785)
Param=True: 91.517% (+/-0.965)
```

A box and whisker plot of the results is also created.

This allows the two samples of results to be compared in a nonparametric way, showing the median and the middle 50% of each sample.

We can see that the distribution of results with standardization is quite different from the distribution of results without standardization. This is likely a real effect.



Box and whisker plot of 1D CNN with and without standardization

Number of Filters

Now that we have an experimental framework, we can explore varying other hyperparameters of the model.

An important hyperparameter for the CNN is the number of filter maps. We can experiment with a range of different values, from less to many more than the 64 used in the first model that we developed.

Specifically, we will try the following numbers of feature maps:

```
n_params = [8, 16, 32, 64, 128, 256]
```

We can use the same code from the previous section and update the *evaluate_model()* function to use the provided parameter as the number of filters in the Conv1D layers. We can also update the *summarize_results()* function to save the boxplot as *exp_cnn_filters.png*.

The complete code example is listed below.

```
# cnn model with filters
from numpy import mean
from numpy import std
from numpy import dstack
from pandas import read_csv
```

```
from matplotlib import pyplot
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Flatten
from keras.layers import Dropout
from keras.layers.convolutional import Conv1D
from keras.layers.convolutional import MaxPooling1D
from keras.utils import to_categorical

# load a single file as a numpy array
def load_file(filepath):
	dataframe = read_csv(filepath, header=None, delim_whitespace=True)
	return dataframe.values

# load a list of files and return as a 3d numpy array
def load_group(filenames, prefix=''):
	loaded = list()
	for name in filenames:
		data = load_file(prefix + name)
		loaded.append(data)
	# stack group so that features are the 3rd dimension
	loaded = dstack(loaded)
	return loaded

# load a dataset group, such as train or test
def load_dataset_group(group, prefix=''):
	filepath = prefix + group + '/Inertial Signals/'
	# load all 9 files as a single array
	filenames = list()
	# total acceleration
	filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt', 'total_acc_z_'+group+'.txt']
	# body acceleration
	filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt', 'body_acc_z_'+group+'.txt']
	# body gyroscope
	filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt', 'body_gyro_z_'+group+'.txt']
	# load input data
	X = load_group(filenames, filepath)
	# load class output
	y = load_file(prefix + group + '/y_'+group+'.txt')
	return X, y

# load the dataset, returns train and test X and y elements
def load_dataset(prefix=''):
	# load all train
	trainX, trainy = load_dataset_group('train', prefix + 'HARDataset/')
	print(trainX.shape, trainy.shape)
	# load all test
	testX, testy = load_dataset_group('test', prefix + 'HARDataset/')
	print(testX.shape, testy.shape)
	# zero-offset class values
	trainy = trainy - 1
	testy = testy - 1
	# one hot encode y
	trainy = to_categorical(trainy)
	testy = to_categorical(testy)
	print(trainX.shape, trainy.shape, testX.shape, testy.shape)
	return trainX, trainy, testX, testy

# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy, n_filters):
	verbose, epochs, batch_size = 0, 10, 32
	n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
	model = Sequential()
	model.add(Conv1D(filters=n_filters, kernel_size=3, activation='relu', input_shape=(n_timesteps,n_features)))
	model.add(Conv1D(filters=n_filters, kernel_size=3, activation='relu'))
	model.add(Dropout(0.5))
	model.add(MaxPooling1D(pool_size=2))
	model.add(Flatten())
	model.add(Dense(100, activation='relu'))
	model.add(Dense(n_outputs, activation='softmax'))
	model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
	# fit network
	model.fit(trainX, trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
	# evaluate model
	_, accuracy = model.evaluate(testX, testy, batch_size=batch_size, verbose=0)
	return accuracy

# summarize scores
def summarize_results(scores, params):
	print(scores, params)
	# summarize mean and standard deviation
	for i in range(len(scores)):
		m, s = mean(scores[i]), std(scores[i])
		print('Param=%d: %.3f%% (+/-%.3f)' % (params[i], m, s))
	# boxplot of scores
	pyplot.boxplot(scores, labels=params)
	pyplot.savefig('exp_cnn_filters.png')

# run an experiment
def run_experiment(params, repeats=10):
	# load data
	trainX, trainy, testX, testy = load_dataset()
	# test each parameter
	all_scores = list()
	for p in params:
		# repeat experiment
		scores = list()
		for r in range(repeats):
			score = evaluate_model(trainX, trainy, testX, testy, p)
			score = score * 100.0
			print('>p=%d #%d: %.3f' % (p, r+1, score))
			scores.append(score)
		all_scores.append(scores)
	# summarize results
	summarize_results(all_scores, params)

```

```
# run the experiment
n_params = [8, 16, 32, 64, 128, 256]
run_experiment(n_params)
```

Running the example repeats the experiment for each of the specified number of filters.

At the end of the run, a summary of the results with each number of filters is presented.

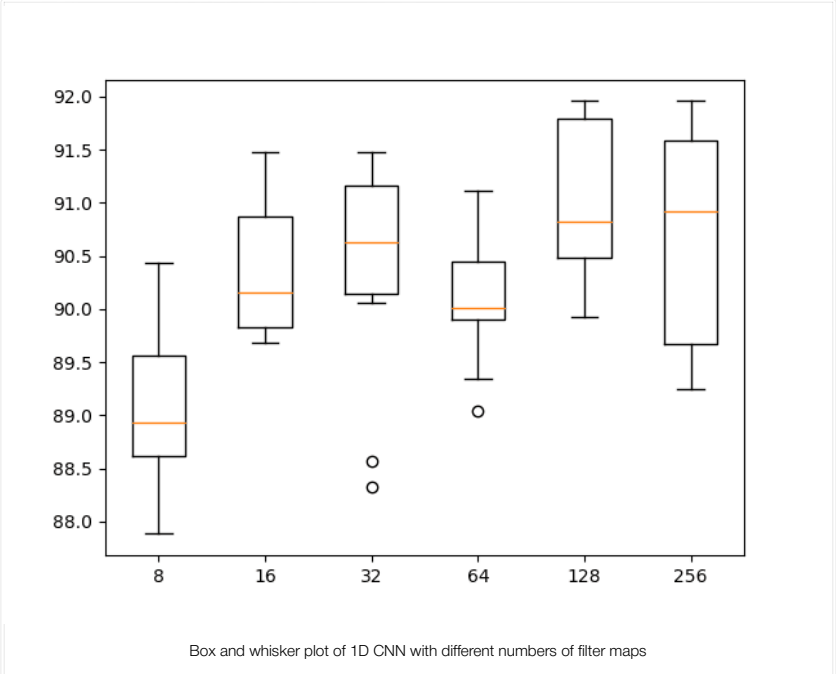
Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

We can see perhaps a trend of increasing average performance with the increase in the number of filter maps. The variance stays pretty constant, and perhaps 128 feature maps might be a good configuration for the network.

```
...
Param=8: 89.148% (+/-0.790)
Param=16: 90.383% (+/-0.613)
Param=32: 90.356% (+/-1.039)
Param=64: 90.098% (+/-0.615)
Param=128: 91.032% (+/-0.702)
Param=256: 90.706% (+/-0.997)
```

A box and whisker plot of the results is also created, allowing the distribution of results with each number of filters to be compared.

From the plot, we can see the trend upward in terms of median classification accuracy (orange line on the box) with the increase in the number of feature maps. We do see a dip at 64 feature maps (the default or baseline in our experiments), which is surprising, and perhaps a plateau in accuracy across 32, 128, and 256 filter maps. Perhaps 32 would be a more stable configuration.



Box and whisker plot of 1D CNN with different numbers of filter maps

Size of Kernel

The size of the kernel is another important hyperparameter of the 1D CNN to tune.

The kernel size controls the number of time steps consider in each "*read*" of the input sequence, that is then projected onto the feature map (via the convolutional process).

A large kernel size means a less rigorous reading of the data, but may result in a more generalized snapshot of the input.

We can use the same experimental set-up and test a suite of different kernel sizes in addition to the default of three time steps. The full list of values is as follows:

```
n_params = [2, 3, 5, 7, 11]
```

The complete code listing is provided below:

```
# cnn model vary kernel size
from numpy import mean
from numpy import std
from numpy import dstack
from pandas import read_csv
from matplotlib import pyplot
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Flatten
from keras.layers import Dropout
from keras.layers.convolutional import Conv1D
from keras.layers.convolutional import MaxPooling1D
from keras.utils import to_categorical

```

```
# load a single file as a numpy array
def load_file(filepath):
    dataframe = read_csv(filepath, header=None, delim_whitespace=True)
    return dataframe.values

# load a list of files and return as a 3d numpy array
def load_group(filenames, prefix=''):
    loaded = list()
    for name in filenames:
        data = load_file(prefix + name)
        loaded.append(data)
    # stack group so that features are the 3rd dimension
    loaded = dstack(loaded)
    return loaded

# load a dataset group, such as train or test
def load_dataset_group(group, prefix=''):
    filepath = prefix + group + '/Inertial Signals/'
    # load all 9 files as a single array
    filenames = list()
    # total acceleration
    filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt', 'total_acc_z_'+group+'.txt']
    # body acceleration
    filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt', 'body_acc_z_'+group+'.txt']
    # body gyroscope
    filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt', 'body_gyro_z_'+group+'.txt']
    # load input data
    X = load_group(filenames, filepath)
    # load class output
    y = load_file(prefix + group + '/y_'+group+'.txt')
    return X, y

# load the dataset, returns train and test X and y elements
def load_dataset(prefix=''):
    # load all train
    trainX, trainy = load_dataset_group('train', prefix + 'HARDataset/')
    print(trainX.shape, trainy.shape)
    # load all test
    testX, testy = load_dataset_group('test', prefix + 'HARDataset/')
    print(testX.shape, testy.shape)
    # zero-offset class values
    trainy = trainy - 1
    testy = testy - 1
    # one hot encode y
    trainy = to_categorical(trainy)
    testy = to_categorical(testy)
    print(trainX.shape, trainy.shape, testX.shape, testy.shape)
    return trainX, trainy, testX, testy

# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy, n_kernel):
    verbose, epochs, batch_size = 0, 15, 32
    n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
    model = Sequential()
    model.add(Conv1D(filters=64, kernel_size=n_kernel, activation='relu', input_shape=(n_timesteps,n_features)))
    model.add(Conv1D(filters=64, kernel_size=n_kernel, activation='relu'))
    model.add(Dropout(0.5))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Flatten())
    model.add(Dense(100, activation='relu'))
    model.add(Dense(n_outputs, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # fit network
    model.fit(trainX, trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
    # evaluate model
    _, accuracy = model.evaluate(testX, testy, batch_size=batch_size, verbose=0)
    return accuracy

# summarize scores
def summarize_results(scores, params):
    print(scores, params)
    # summarize mean and standard deviation
    for i in range(len(scores)):
        m, s = mean(scores[i]), std(scores[i])
        print('Param=%d: %.3f%% (+/-%.3f)' % (params[i], m, s))
    # boxplot of scores
    pyplot.boxplot(scores, labels=params)
    pyplot.savefig('exp_cnn_kernel.png')

# run an experiment
def run_experiment(params, repeats=10):
    # load data
    trainX, trainy, testX, testy = load_dataset()
    # test each parameter
    all_scores = list()
    for p in params:
        # repeat experiment
        scores = list()
        for r in range(repeats):
            score = evaluate_model(trainX, trainy, testX, testy, p)
            score = score * 100.0
            print('>p=%d #%d: %.3f' % (p, r+1, score))
            scores.append(score)
        all_scores.append(scores)
    # summarize results
    summarize_results(all_scores, params)

# run the experiment
n_params = [2, 3, 5, 7, 11]
run_experiment(n_params)
```

Running the example tests each kernel size in turn.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few

times and compare the average outcome.

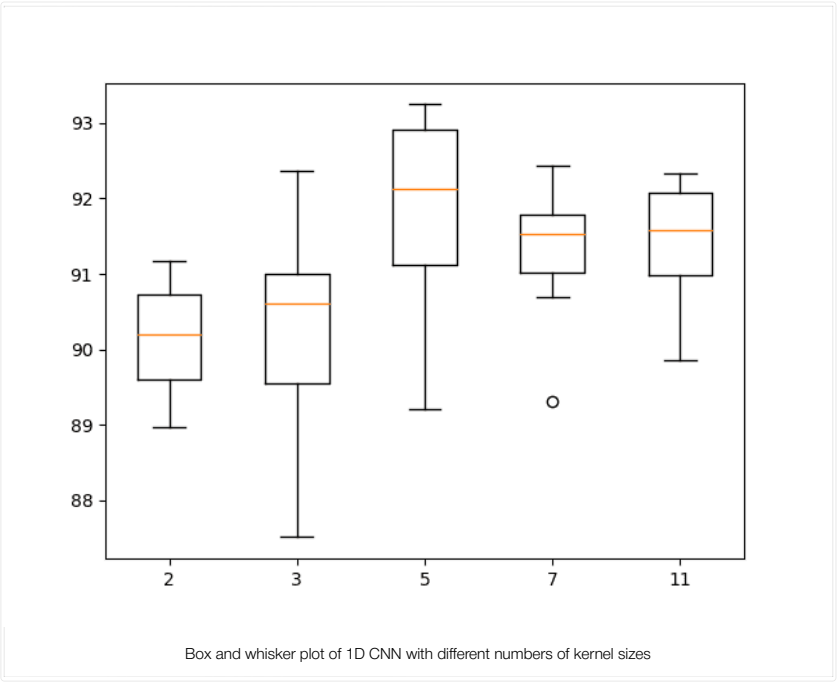
The results are summarized at the end of the run. We can see a general increase in model performance with the increase in kernel size.

The results suggest a kernel size of 5 might be good with a mean skill of about 91.8%, but perhaps a size of 7 or 11 may also be just as good with a smaller standard deviation.

```
...
Param=2: 90.176% (+/-0.724)
Param=3: 90.275% (+/-1.277)
Param=5: 91.853% (+/-1.249)
Param=7: 91.347% (+/-0.852)
Param=11: 91.456% (+/-0.743)
```

A box and whisker plot of the results is also created.

The results suggest that a larger kernel size does appear to result in better accuracy and that perhaps a kernel size of 7 provides a good balance between good performance and low variance.



Box and whisker plot of 1D CNN with different numbers of kernel sizes

This is just the beginning of tuning the model, although we have focused on perhaps the more important elements. It might be interesting to explore combinations of some of the above findings to see if performance can be lifted even further.

It may also be interesting to increase the number of repeats from 10 to 30 or more to see if it results in more stable findings.

Multi-Headed Convolutional Neural Network

Another popular approach with CNNs is to have a multi-headed model, where each head of the model reads the input time steps using a different sized kernel.

For example, a three-headed model may have three different kernel sizes of 3, 5, 11, allowing the model to read and interpret the sequence data at three different resolutions. The interpretations from all three heads are then concatenated within the model and interpreted by a fully-connected layer before a prediction is made.

We can implement a multi-headed 1D CNN using the Keras functional API. For a gentle introduction to this API, see the post:

- How to Use the Keras Functional API for Deep Learning

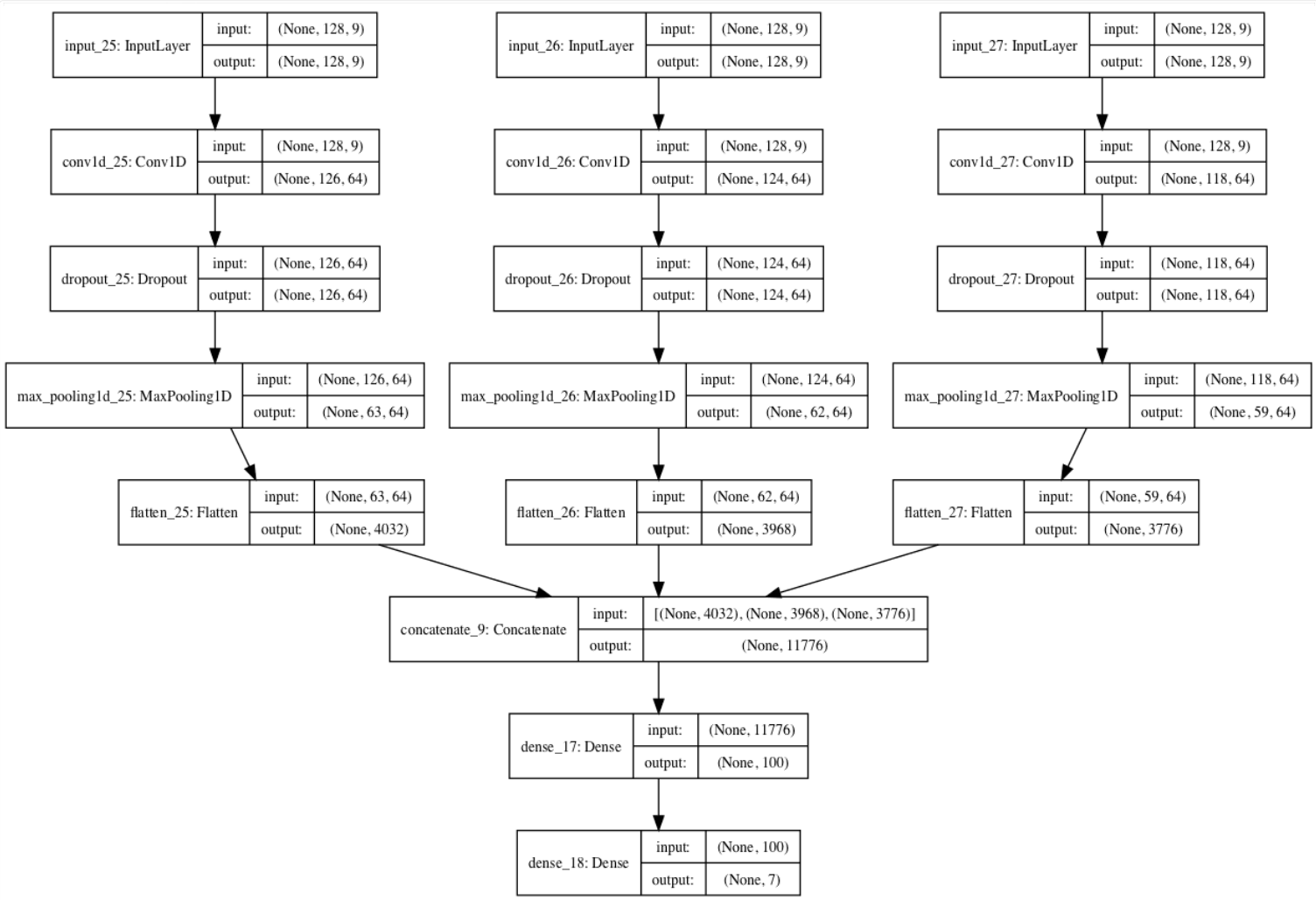
The updated version of the *evaluate_model()* function is listed below that creates a three-headed CNN model.

We can see that each head of the model is the same structure, although the kernel size is varied. The three heads then feed into a single merge layer before being interpreted prior to making a prediction.

```
# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy):
    verbose, epochs, batch_size = 0, 10, 32
    n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
    # head 1
    inputs1 = Input(shape=(n_timesteps,n_features))
    conv1 = Conv1D(filters=64, kernel_size=3, activation='relu')(inputs1)
    drop1 = Dropout(0.5)(conv1)
    pool1 = MaxPooling1D(pool_size=2)(drop1)
    flat1 = Flatten()(pool1)
    # head 2
    inputs2 = Input(shape=(n_timesteps,n_features))
    conv2 = Conv1D(filters=64, kernel_size=5, activation='relu')(inputs2)
```

```
    drop2 = Dropout(0.5)(conv2)
    pool2 = MaxPooling1D(pool_size=2)(drop2)
    flat2 = Flatten()(pool2)
    # head 3
    inputs3 = Input(shape=(n_timesteps,n_features))
    conv3 = Conv1D(filters=64, kernel_size=11, activation='relu')(inputs3)
    drop3 = Dropout(0.5)(conv3)
    pool3 = MaxPooling1D(pool_size=2)(drop3)
    flat3 = Flatten()(pool3)
    # merge
    merged = concatenate([flat1, flat2, flat3])
    # interpretation
    dense1 = Dense(100, activation='relu')(merged)
    outputs = Dense(n_outputs, activation='softmax')(dense1)
    model = Model(inputs=[inputs1, inputs2, inputs3], outputs=outputs)
    # save a plot of the model
    plot_model(model, show_shapes=True, to_file='multichannel.png')
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # fit network
    model.fit([trainX,trainX,trainX], trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
    # evaluate model
    _, accuracy = model.evaluate([testX,testX,testX], testy, batch_size=batch_size, verbose=0)
    return accuracy
```

When the model is created, a plot of the network architecture is created; provided below, it gives a clear idea of how the constructed model fits together.



Plot of the Multi-Headed 1D Convolutional Neural Network

Other aspects of the model could be varied across the heads, such as the number of filters or even the preparation of the data itself.

The complete code example with the multi-headed 1D CNN is listed below.

```
# multi-headed cnn model
from numpy import mean
from numpy import std
from numpy import dstack
from pandas import read_csv
from matplotlib import pyplot
from keras.utils import to_categorical
from keras.utils.vis_utils import plot_model
from keras.models import Model
from keras.layers import Input
from keras.layers import Dense
from keras.layers import Flatten
from keras.layers import Dropout
from keras.layers.convolutional import Conv1D
from keras.layers.convolutional import MaxPooling1D
from keras.layers.merge import concatenate
```

```

# load a single file as a numpy array
def load_file(filepath):
    dataframe = read_csv(filepath, header=None, delim_whitespace=True)
    return dataframe.values

# load a list of files and return as a 3d numpy array
def load_group(filenames, prefix=''):
    loaded = list()
    for name in filenames:
        data = load_file(prefix + name)
        loaded.append(data)
    # stack group so that features are the 3rd dimension
    loaded = dstack(loaded)
    return loaded

# load a dataset group, such as train or test
def load_dataset_group(group, prefix=''):
    filepath = prefix + group + '/Inertial Signals/'
    # load all 9 files as a single array
    filenames = list()
    # total acceleration
    filenames += ['total_acc_x_'+group+'.txt', 'total_acc_y_'+group+'.txt', 'total_acc_z_'+group+'.txt']
    # body acceleration
    filenames += ['body_acc_x_'+group+'.txt', 'body_acc_y_'+group+'.txt', 'body_acc_z_'+group+'.txt']
    # body gyroscope
    filenames += ['body_gyro_x_'+group+'.txt', 'body_gyro_y_'+group+'.txt', 'body_gyro_z_'+group+'.txt']
    # load input data
    X = load_group(filenames, filepath)
    # load class output
    y = load_file(prefix + group + '/y_'+group+'.txt')
    return X, y

# load the dataset, returns train and test X and y elements
def load_dataset(prefix=''):
    # load all train
    trainX, trainy = load_dataset_group('train', prefix + 'HARDataset/')
    print(trainX.shape, trainy.shape)
    # load all test
    testX, testy = load_dataset_group('test', prefix + 'HARDataset/')
    print(testX.shape, testy.shape)
    # zero-offset class values
    trainy = trainy - 1
    testy = testy - 1
    # one hot encode y
    trainy = to_categorical(trainy)
    testy = to_categorical(testy)
    print(trainX.shape, trainy.shape, testX.shape, testy.shape)
    return trainX, trainy, testX, testy

# fit and evaluate a model
def evaluate_model(trainX, trainy, testX, testy):
    verbose, epochs, batch_size = 0, 10, 32
    n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
    # head 1
    inputs1 = Input(shape=(n_timesteps,n_features))
    conv1 = Conv1D(filters=64, kernel_size=3, activation='relu')(inputs1)
    drop1 = Dropout(0.5)(conv1)
    pool1 = MaxPooling1D(pool_size=2)(drop1)
    flat1 = Flatten()(pool1)
    # head 2
    inputs2 = Input(shape=(n_timesteps,n_features))
    conv2 = Conv1D(filters=64, kernel_size=5, activation='relu')(inputs2)
    drop2 = Dropout(0.5)(conv2)
    pool2 = MaxPooling1D(pool_size=2)(drop2)
    flat2 = Flatten()(pool2)
    # head 3
    inputs3 = Input(shape=(n_timesteps,n_features))
    conv3 = Conv1D(filters=64, kernel_size=11, activation='relu')(inputs3)
    drop3 = Dropout(0.5)(conv3)
    pool3 = MaxPooling1D(pool_size=2)(drop3)
    flat3 = Flatten()(pool3)
    # merge
    merged = concatenate([flat1, flat2, flat3])
    # interpretation
    dense1 = Dense(100, activation='relu')(merged)
    outputs = Dense(n_outputs, activation='softmax')(dense1)
    model = Model(inputs=[inputs1, inputs2, inputs3], outputs=outputs)
    # save a plot of the model
    plot_model(model, show_shapes=True, to_file='multichannel.png')
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # fit network
    model.fit([trainX,trainX,trainX], trainy, epochs=epochs, batch_size=batch_size, verbose=verbose)
    # evaluate model
    _, accuracy = model.evaluate([testX,testX,testX], testy, batch_size=batch_size, verbose=0)
    return accuracy

# summarize scores
def summarize_results(scores):
    print(scores)
    m, s = mean(scores), std(scores)
    print('Accuracy: %.3f%% (+/-%.3f)' % (m, s))

# run an experiment
def run_experiment(repeats=10):
    # load data
    trainX, trainy, testX, testy = load_dataset()
    # repeat experiment
    scores = list()
    for r in range(repeats):
        score = evaluate_model(trainX, trainy, testX, testy)
        score = score * 100.0
        print('>#%d: %.3f' % (r+1, score))
        scores.append(score)
    # summarize results
    summarize_results(scores)
```

```
# run the experiment
run_experiment()
```

Running the example prints the performance of the model each repeat of the experiment and then summarizes the estimated score as the mean and standard deviation, as we did in the first case with the simple 1D CNN.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

We can see that the average performance of the model is about 91.6% classification accuracy with a standard deviation of about 0.8.

This example may be used as the basis for exploring a variety of other models that vary different model hyperparameters and even different data preparation schemes across the input heads.

It would not be an apples-to-apples comparison to compare this result with a single-headed CNN given the relative tripling of the resources in this model. Perhaps an apples-to-apples comparison would be a model with the same architecture and the same number of filters across each input head of the model.

```
>#1: 91.788
>#2: 92.942
>#3: 91.551
>#4: 91.415
>#5: 90.974
>#6: 91.992
>#7: 92.162
>#8: 89.888
>#9: 92.671
>#10: 91.415

[91.78825924669155, 92.94197488971835, 91.55072955548015, 91.41499830335935, 90.97387173396675, 91.99185612487275, 92.16152019002375, 89.88802171700034, 92.670

Accuracy: 91.680% (+/-0.823)
```

Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- **Date Preparation**. Explore other data preparation schemes such as data normalization and perhaps normalization after standardization.
- **Network Architecture**. Explore other network architectures, such as deeper CNN architectures and deeper fully-connected layers for interpreting the CNN input features.
- **Diagnostics**. Use simple learning curve diagnostics to interpret how the model is learning over the epochs and whether more regularization, different learning rates, or different batch sizes or numbers of epochs may result in a better performing or more stable model.

If you explore any of these extensions, I'd love to know.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Papers

- A Public Domain Dataset for Human Activity Recognition Using Smartphones, 2013.
- Human Activity Recognition on Smartphones using a Multiclass Hardware-Friendly Support Vector Machine, 2012.

Articles

- Human Activity Recognition Using Smartphones Data Set, UCI Machine Learning Repository
- Activity recognition, Wikipedia
- Activity Recognition Experiment Using Smartphone Sensors, Video.

Summary

In this tutorial, you discovered how to develop one-dimensional convolutional neural networks for time series classification on the problem of human activity recognition.

Specifically, you learned:

- How to load and prepare the data for a standard human activity recognition dataset and develop a single 1D CNN model that achieves excellent performance on the raw data.
- How to further tune the performance of the model, including data transformation, filter maps, and kernel sizes.
- How to develop a sophisticated multi-headed one-dimensional convolutional neural network model that provides an ensemble-like result.

Do you have any questions?
Ask your questions in the comments below and I will do my best to answer.

How to use 1d convolutional neural network (conv1d) to predict DNA sequence binding to protein

Sep 9, 2023 · 11 min read · 0 Comments (https://divingintogeneticsandgenomics.com/post/how-to-use-1d-convolutional-neural-network-conv1d-to-predict-dna-sequence-binding-to-protein/#disqus_thread) · deeplearning (https://divingintogeneticsandgenomics.com/categories/deeplearning/), bioinformatics (https://divingintogeneticsandgenomics.com/categories/bioinformatics/)

ding%20to%20protein&url=https%3a%2f%2fdivingintogeneticsandgenomics.com%2fpost%2fhow-to-

neural-network-conv1d-to-predict-dna-sequence-binding-to-protein%2f)

onvolutional-neural-network-conv1d-to-predict-dna-sequence-binding-to-uence%20binding%20to%20protein)

onal-neural-network-conv1d-to-predict-dna-sequence-binding-to-uence%20binding%20to%20protein)

binding%20to%20protein&body=https%3a%2f%2fdivingintogeneticsandgenomics.com%2fpost%2fhow-

In the mysterious world of DNA, where the secrets of life are encoded, scientists are harnessing the power of cutting-edge technology to decipher the language of genes. One of the remarkable tools they're using is the 1D Convolutionary Neural Network, or 1D CNN, which might sound like jargon from a sci-fi movie, but it's actually a game-changer in DNA sequence analysis.

Imagine DNA as a long, intricate string of letters, like a never-ending alphabet book. Each letter holds a unique piece of information that dictates our traits and characteristics. Understanding this code is crucial for unraveling genetic mysteries, detecting diseases, and even designing personalized medicine.

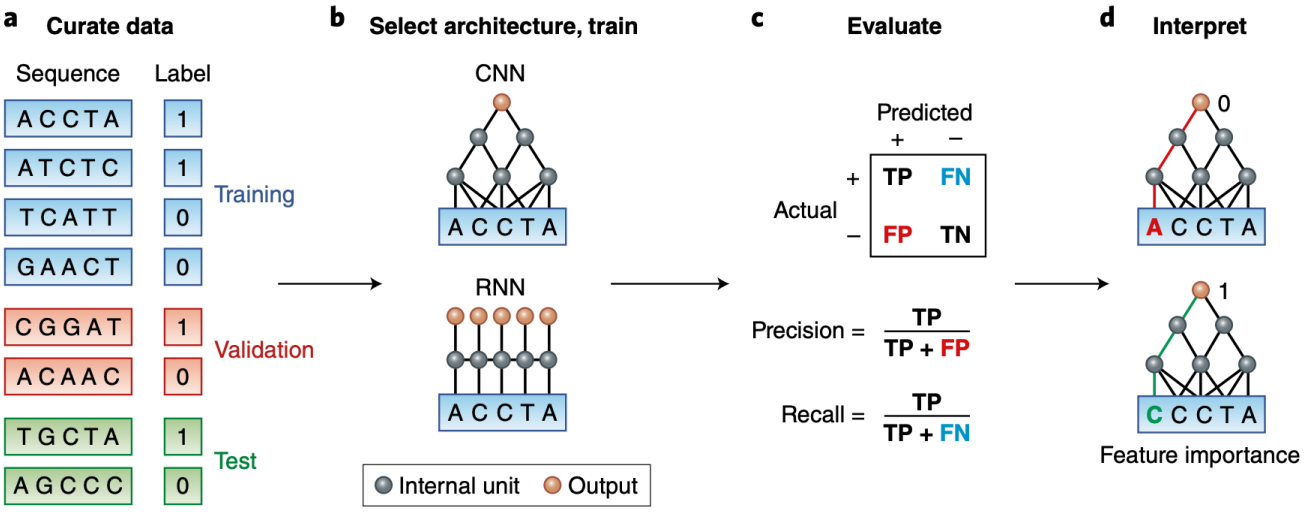
In this blog post, I will walk you through an example to predict DNA sequences binding to proteins using 1D CNN.

The original dataset is described in this paper A primer on deep learning in genomics (https://www.nature.com/articles/s41588-018-0295-5) and the python implementation can be found at:

https://github.com/abidlabs/deep-learning-genomics-primer/tree/master (https://github.com/abidlabs/deep-learning-genomics-primer/tree/master)

> We will use simulated data that consists of DNA sequences of length 50 bases (chosen to be artificially short so that the data is easy to play around with), and is labeled with 0 or 1 depending on the result of the assay. Our goal is to build a classifier that can predict whether a particular sequence will bind to the protein and discover the short motif that is the binding site in the sequences that are bound to the protein.

We will follow it from the paper:



```
library(keras)
library(reticulate)
library(ggplot2)
use_condaenv("r-reticulate")
library(readr)
library(tictoc) #monitoring the time

# Set a random seed in R to make it more reproducible
set.seed(123)

# Set the seed for Keras/TensorFlow
tensorflow::set_random_seed(123)
```

```
SEQUENCES_URL<-  'https://raw.githubusercontent.com/abidlabs/deep-learning-genomics-prime

LABELS_URL<-  'https://raw.githubusercontent.com/abidlabs/deep-learning-genomics-primer/m

sequences<- read_tsv(SEQUENCES_URL, col_names = FALSE)
labels<- read_tsv(LABELS_URL, col_names = FALSE)

head(sequences)
```

```
#> # A tibble: 6 × 1
#>   X1
#>   <chr>
#> 1 CCGAGGGCTATGGTTTGGAAGTTAGAACCCTGGGGCTTCTCGCGGACACC
#> 2 GAGTTTATATGGCGCGAGCCTAGTGGTTTTTGTACTTGTTTGTCGCGTCG
#> 3 GATCAGTAGGGAAACAAACAGAGGGCCCAGCCACATCTAGCAGGTAGCCT
#> 4 GTCCACGACCGAACTCCCACCTTGACCGCAGAGGTACCACCAGAGCCCTG
#> 5 GGCGACCGAACTCCAACTAGAACCTGCATAACTGGCCTGGGAGATATGGT
#> 6 AGACATTGTCAGAACTTAGTGTGCGCCGCACTGAGCGACCGAACTCCGAC
```

```
head(labels)
```

```
#> # A tibble: 6 × 1
#>      X1
#>   <dbl>
#> 1     0
#> 2     0
#> 3     0
#> 4     1
#> 5     1
#> 6     1
```

```
nrow(sequences)
```

```
#> [1] 2000
```

```
sequences<- sequences$X1
labels<- labels$X1
```

One-hot encoding for the DNA sequences.

```
one_hot_encode_dna <- function(sequence) {
  # Define a dictionary for mapping nucleotides to one-hot encoding
  # no dictionary/hash in R, use list
  nucleotide_dict <- list(A = c(1, 0, 0, 0),
                          T = c(0, 1, 0, 0),
                          C = c(0, 0, 1, 0),
                          G = c(0, 0, 0, 1))

  # Split the sequence into individual nucleotides
  nucleotides <- unlist(strsplit(sequence, ""))

  # Initialize an empty matrix to store the one-hot encoding
  one_hot_matrix <- matrix(0, nrow = length(nucleotides), ncol = 4)

  # Fill the one-hot matrix based on the sequence
  for (i in 1:length(nucleotides)) {
    one_hot_matrix[i,] <- nucleotide_dict[[nucleotides[i]]]
  }

  return(one_hot_matrix)
}



# Perform one-hot encoding
one_hot_matrix <- one_hot_encode_dna(sequences[1])

# Print the one-hot encoded matrix
print(one_hot_matrix)
```

```
#>       [,1] [,2] [,3] [,4]
#>  [1,]    0    0    1    0
#>  [2,]    0    0    1    0
#>  [3,]    0    0    0    1
#>  [4,]    1    0    0    0
#>  [5,]    0    0    0    1
#>  [6,]    0    0    0    1
#>  [7,]    0    0    0    1
#>  [8,]    0    0    1    0
#>  [9,]    0    1    0    0
#> [10,]    1    0    0    0
#> [11,]    0    1    0    0
#> [12,]    0    0    0    1
#> [13,]    0    0    0    1
#> [14,]    0    1    0    0
#> [15,]    0    1    0    0
#> [16,]    0    1    0    0
#> [17,]    0    0    0    1
#> [18,]    0    0    0    1
#> [19,]    1    0    0    0
#> [20,]    1    0    0    0
#> [21,]    0    0    0    1
#> [22,]    0    1    0    0
#> [23,]    0    1    0    0
#> [24,]    1    0    0    0
#> [25,]    0    0    0    1
#> [26,]    1    0    0    0
#> [27,]    1    0    0    0
#> [28,]    0    0    1    0
#> [29,]    0    0    1    0
#> [30,]    0    0    1    0
#> [31,]    0    1    0    0
#> [32,]    0    0    0    1
#> [33,]    0    0    0    1
#> [34,]    0    0    0    1
#> [35,]    0    0    0    1
#> [36,]    0    0    1    0
#> [37,]    0    1    0    0
#> [38,]    0    1    0    0
#> [39,]    0    0    1    0
#> [40,]    0    1    0    0
#> [41,]    0    0    1    0
#> [42,]    0    0    0    1
#> [43,]    0    0    1    0
#> [44,]    0    0    0    1
#> [45,]    0    0    0    1
#> [46,]    1    0    0    0
#> [47,]    0    0    1    0
#> [48,]    1    0    0    0
#> [49,]    0    0    1    0
#> [50,]    0    0    1    0
```

Do it for all the sequences

```
dna_data<- purrr::map(sequences, one_hot_encode_dna)
```

`dna_data` is a list of matrix, each entry is one DNA sequence

```
dna_data[[1]]
```

```
#>       [,1] [,2] [,3] [,4]
#>  [1,]    0    0    1    0
#>  [2,]    0    0    1    0
#>  [3,]    0    0    0    1
#>  [4,]    1    0    0    0
#>  [5,]    0    0    0    1
#>  [6,]    0    0    0    1
#>  [7,]    0    0    0    1
#>  [8,]    0    0    1    0
#>  [9,]    0    1    0    0
#> [10,]    1    0    0    0
#> [11,]    0    1    0    0
#> [12,]    0    0    0    1
#> [13,]    0    0    0    1
#> [14,]    0    1    0    0
#> [15,]    0    1    0    0
#> [16,]    0    1    0    0
#> [17,]    0    0    0    1
#> [18,]    0    0    0    1
#> [19,]    1    0    0    0
#> [20,]    1    0    0    0
#> [21,]    0    0    0    1
#> [22,]    0    1    0    0
#> [23,]    0    1    0    0
#> [24,]    1    0    0    0
#> [25,]    0    0    0    1
#> [26,]    1    0    0    0
#> [27,]    1    0    0    0
#> [28,]    0    0    1    0
#> [29,]    0    0    1    0
#> [30,]    0    0    1    0
#> [31,]    0    1    0    0
#> [32,]    0    0    0    1
#> [33,]    0    0    0    1
#> [34,]    0    0    0    1
#> [35,]    0    0    0    1
#> [36,]    0    0    1    0
#> [37,]    0    1    0    0
#> [38,]    0    1    0    0
#> [39,]    0    0    1    0
#> [40,]    0    1    0    0
#> [41,]    0    0    1    0
#> [42,]    0    0    0    1
#> [43,]    0    0    1    0
#> [44,]    0    0    0    1
#> [45,]    0    0    0    1
#> [46,]    1    0    0    0
#> [47,]    0    0    1    0
#> [48,]    1    0    0    0
#> [49,]    0    0    1    0
#> [50,]    0    0    1    0
```

The input of the 1D convolutionary neural network is a 3D tensor (sample, time, feature). In this case, it will be (sample, position_in_DNA_sequence, 4_DNA_bases/A,T,C,G).

It can be confusing because R and python fill in the matrix in column-wise and row-wise manner, respectively.

read my previous blog post on tensor reshaping in `R` : https://divingintogeneticsandgenomics.com/post/basic-tensor-array-manipulations-in-r/ (https://divingintogeneticsandgenomics.com/post/basic-tensor-array-manipulations-in-r/)

```
dna_data_tensor<- array_reshape(lapply(dna_data,
                                       function(x) x %>%t() %>% c()) %>% unlist(),
                                dim = c(2000, 50, 4))

# the first entry
# dna_data_tensor[1,,]

all.equal(dna_data[[1]], dna_data_tensor[1,,])
```

```
#> [1] TRUE
```

Split training set and testing set.

```
# save 25% for testing
testing_len<- length(dna_data)*0.25

test_index<- sample(1:length(dna_data), testing_len)

train_x<- dna_data_tensor[-test_index,,]
train_y<- labels[-test_index]

test_x<- dna_data_tensor[test_index,, ]
test_y<- labels[test_index]

dim(train_x)
```

```
#> [1] 1500   50    4
```

```
dim(test_x)
```

```
#> [1] 500  50   4
```

1D convolutionary neural network

2D convolutions, which involved extracting 2D patches from image tensors and applying the same transformation to each patch. Similarly, you can employ 1D convolutions to extract local

1D patches or subsequences from sequences, as illustrated below.

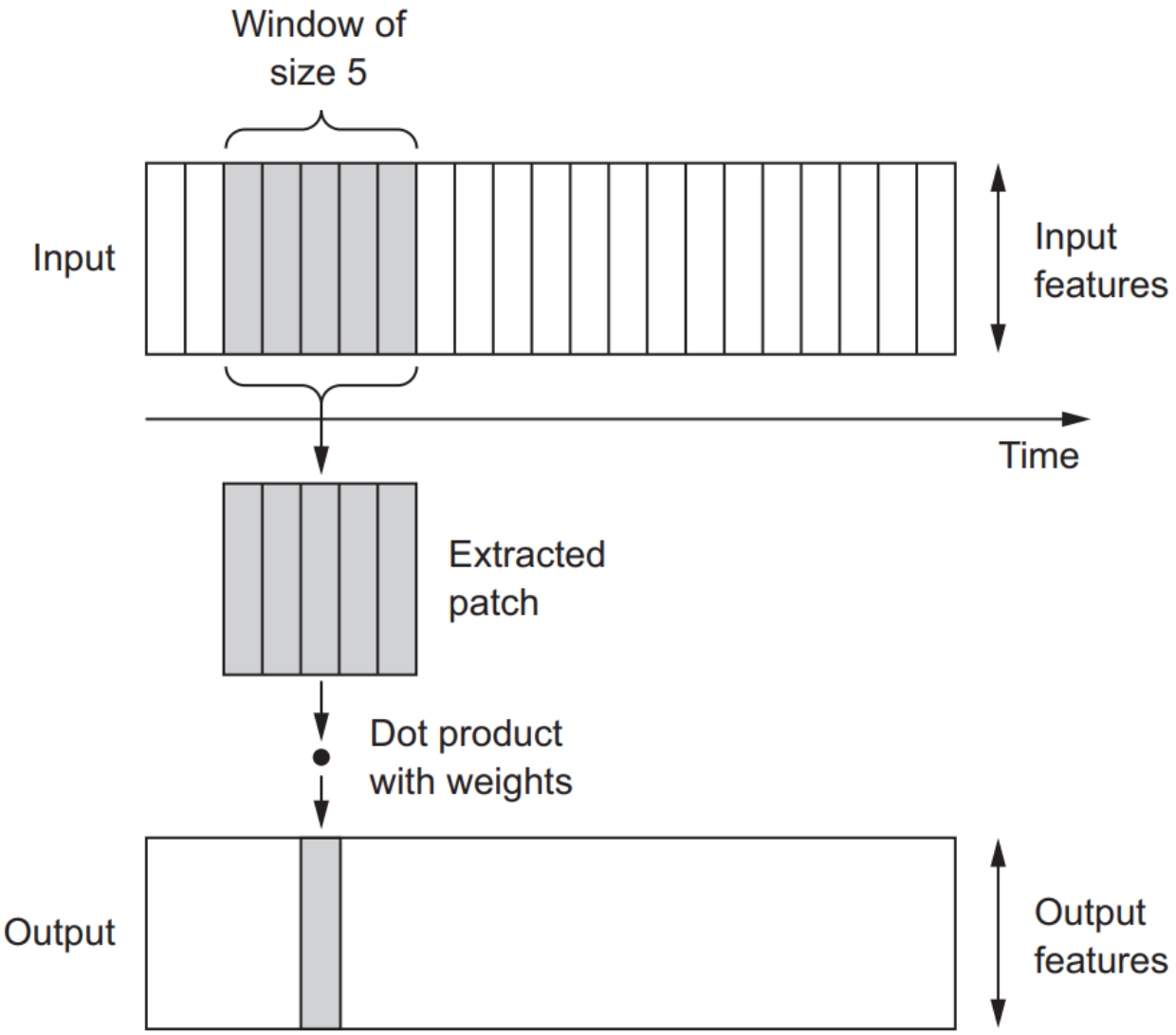


Image from Deep Learning with R (https://www.manning.com/books/deep-learning-with-r).

These 1D convolution layers excel at recognizing local patterns within a sequence. Since they apply the same transformation to every patch, a pattern learned at one position in a sequence can subsequently be identified at a different location. This property enables 1D convolutional networks with DNA motif recognition.

For example, when processing sequences of DNA using 1D convolutions with a window size of 5, the network should be capable of learning motifs or DNA fragments of up to 5 characters in length. Moreover, it can recognize these words in any context within an input sequence.

Spoiler alert: the true regulatory motif is `CGACCGAACTCC` which is 12 bases in length in the

dataset.

That's why we choose the `kernal_size` to 12. Again, understanding the biology is key. If you know the motifs that you are trying to find, you can feed the conv1D the right parameters.

If you want to understand more on the 2D convolutionary neural network on `filters` and `max pooling` , read my previous blog post (https://divingintogeneticsandgenomics.com/post/how-to-classify-mnist-images-with-convolutional-neural-network/).

```
model<- keras_model_sequential() %>%
  layer_conv_1d(filters = 32, kernel_size = 12, activation = "relu",
                input_shape = c(50, 4)) %>%
  layer_max_pooling_1d(pool_size = 4) %>%
  layer_flatten() %>%
  layer_dense(units = 16, activation = "relu") %>%
  layer_dense(units = 1, activation= "sigmoid")

summary(model)
```

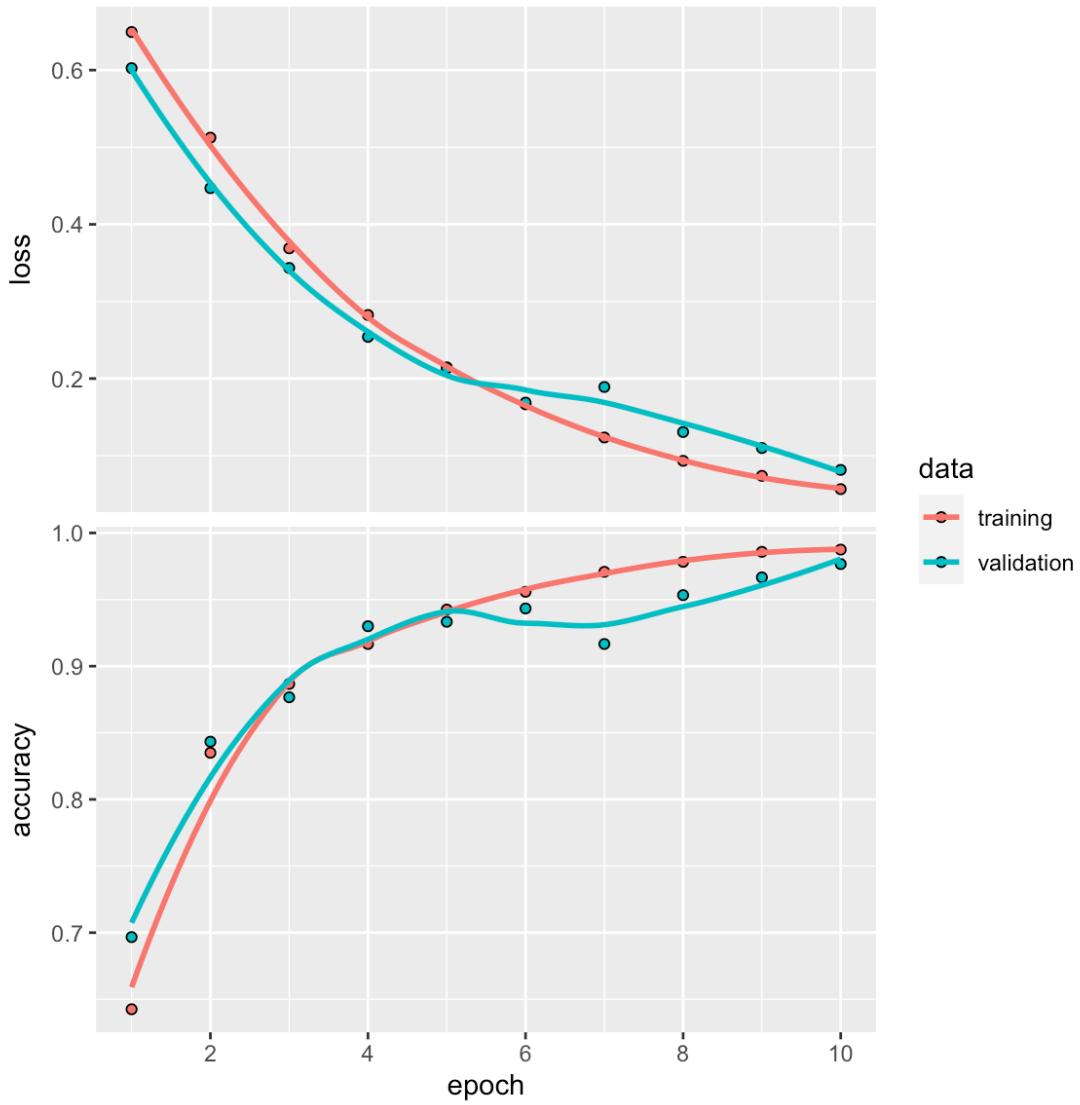
```
#> Model: "sequential"
#> ________________________________________________________________________________
#> Layer (type)                        Output Shape                    Param #
#> ================================================================================
#> conv1d (Conv1D)                     (None, 39, 32)                  1568
#> ________________________________________________________________________________
#> max_pooling1d (MaxPooling1D)        (None, 9, 32)                   0
#> ________________________________________________________________________________
#> flatten (Flatten)                   (None, 288)                     0
#> ________________________________________________________________________________
#> dense_1 (Dense)                     (None, 16)                      4624
#> ________________________________________________________________________________
#> dense (Dense)                       (None, 1)                       17
#> ================================================================================
#> Total params: 6,209
#> Trainable params: 6,209
#> Non-trainable params: 0
#> ________________________________________________________________________________
```

Compile the model.

```
model %>% compile(
        optimizer = "rmsprop",
        loss = "binary_crossentropy",
        metrics = c("accuracy")
)
```

training

```
history<- model %>%
  fit(
    train_x, train_y,
    epochs = 10,
    batch_size = 32,
    validation_split = 0.2
  )

plot(history)
```



train on full dataset

```
model %>%
        fit(train_x, train_y, epochs = 10, batch_size = 32)
```

validation on testing data

```
metrics<- model %>%
  evaluate(test_x, test_y)

metrics
```

```
#>       loss   accuracy
#> 0.01939072 0.99199998
```

```
res<- predict(model, test_x) %>%
        dplyr::bind_cols(test_y)

colnames(res)<- c("prob", "label")

threshold <- 0.5
res<- res %>%
  dplyr::mutate(.pred_class = ifelse(prob >= threshold, 1, 0)) %>%
  dplyr::mutate(.pred_class = factor(.pred_class)) %>%
  dplyr::mutate(label = factor(label))

library(tidymodels)
accuracy(res, truth = label, estimate = .pred_class)
```

```
#> # A tibble: 1 × 3
#>   .metric  .estimator .estimate
#>   <chr>    <chr>          <dbl>
#> 1 accuracy binary         0.992
```

```
res %>% conf_mat(truth = label, estimate = .pred_class)
```

```
#>           Truth
#> Prediction   0   1
#>          0 260   2
#>          1   2 236
```

In this simple example, the `conv_1d` only made 4 mistakes among the 500 testing DNA sequences with an accuracy of ~99%. That's pretty impressive.

We can also add another layer of `conv_1d` and `max_pool` again. The architecture of a deep neural network is an art.

random forest

In this paper Ten quick tips for deep learning in biology (https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1009803)

- Tip 1: Decide whether deep learning is appropriate for your problem
- Tip 2: Use traditional methods to establish performance baselines
- Tip 3: Understand the complexities of training deep neural networks
- Tip 4: Know your data and your question
- Tip 5: Choose an appropriate data representation and neural network architecture
- Tip 6: Tune your hyperparameters extensively and systematically
- Tip 7: Address deep neural networks' increased tendency to overfit the dataset
- Tip 8: Deep learning models can be made more transparent
- Tip 9: Don't over interpret predictions
- Tip 10: Actively consider the ethical implications of your work

Regression based methods and random forest are always my go-to baseline machine learning approach. Let's see how random forest perform for this problem.

```
library(tidymodels)

dim(train_x)
```

```
#> [1] 1500   50    4
```

```
# flatten the 50x4 matrix to a single vector
data_train<- array_reshape(train_x, dim = c(1500, 50*4))

dim(data_train)
```

```
#> [1] 1500  200
```

```
colnames(data_train)<- paste0("feature", 1:200)

data_train[1:5, 1:5]
```

```
#>      feature1 feature2 feature3 feature4 feature5
#> [1,]        0        0        1        0        0
#> [2,]        0        0        0        1        1
#> [3,]        0        0        0        1        1
#> [4,]        0        0        0        1        0
#> [5,]        0        0        0        1        0
```

```
# turn the label to a factor so tidymodel knows it is a classification problem
data_train<- bind_cols(as.data.frame(data_train), label = train_y %>%
        as.factor())
```

do the same for the testing data

```
data_test<- array_reshape(test_x, dim = c(500, 50*4))
colnames(data_test)<- paste0("feature", 1:200)
data_test<- bind_cols(as.data.frame(data_test), label = test_y %>%
        as.factor())
```

build the model using `tidymodels` . PS, read Tidy Modeling with R (https://www.tmwr.org/) for FREE.

```
tic()
rf_recipe <-
  recipe(formula = label ~ ., data = data_train)

## feature importance sore to TRUE, one can tune the trees and mtry
rf_spec <- rand_forest() %>%
  set_engine("randomForest", importance = TRUE) %>%
  set_mode("classification")

rf_workflow <- workflow() %>%
  add_recipe(rf_recipe) %>%
  add_model(rf_spec)
```

train the model

```
rf_fit <- fit(rf_workflow, data = data_train)
```

test the model

```
res<- predict(rf_fit, new_data = data_test) %>%
        bind_cols(data_test %>% select(label))

toc()
```

```
#> 24.129 sec elapsed
```

what's the accuracy?

```
accuracy(res, truth = label, estimate = .pred_class)
```

```
#> # A tibble: 1 × 3
#>   .metric  .estimator .estimate
#>   <chr>    <chr>          <dbl>
#> 1 accuracy binary         0.826
```

```
## confusion matrix,
res %>% conf_mat(truth = label, estimate = .pred_class)
```

```
#>            Truth
#> Prediction   0   1
#>          0 207  32
#>          1  55 206
```

An accuracy of this un-tuned random forest model gives ~82% accuracy. This is much worse than the `conv1d` model and is slower. This demonstrate where the deep learning models shine in sequence analysis.

deeplearning (https://divingintogeneticsandgenomics.com/tags/deeplearning/)

bioinformatics (https://divingintogeneticsandgenomics.com/tags/bioinformatics/)

Related

- Deep learning to predict cancer from healthy controls using TCRseq data (/post/deep-learning-to-predict-cancer-from-healthy-controls-using-tcrseq-data/)
- Long Short-term memory (LSTM) Recurrent Neural Network (RNN) to classify movie reviews (/post/long-short-term-memory-lstm-recurrent-neural-network-rnn-to-classify-movie-reviews/)
- Understand word embedding and use deep learning to classify movie reviews (/post/understand-word-embedding-and-use-deep-learning-to-classify-movie-reviews/)
- multi-omics data integration: a case study with transcriptomics and genomics mutation data (/post/multi-omics-data-integration-a-case-study-with-transcriptomics-and-genomics-mutation-data/)
- neighborhood/cellular niches analysis with spatial transcriptome data in Seurat and Bioconductor (/post/neighborhood-cellular-niches-analysis-with-spatial-transcriptome-data-in-seurat-and-bioconductor/)

NEXT

Omics Playground: Derive biological insights from your omics data at your fingertip (https://divingintogeneticsandgenomics.com/post/omics-playground-derive-biological-insights-from-your-omics-data-at-your-

PERSPECTIVE
https://doi.org/10.1038/s41588-018-0295-5
nature genetics

A primer on deep learning in genomics

James Zou[1,2,3*], Mikael Huss[4,5], Abubakar Abid[3], Pejman Mohammadi[6,7], Ali Torkamani[6,7] and Amalio Telenti[6,7*]

Deep learning methods are a class of machine learning techniques capable of identifying highly complex patterns in large datasets. Here, we provide a perspective and primer on deep learning applications for genome analysis. We discuss successful applications in the fields of regulatory genomics, variant calling and pathogenicity scores. We include general guidance for how to effectively use deep learning methods as well as a practical guide to tools and resources. This primer is accompanied by an interactive online tutorial.

Deep learning has made impressive recent advances in applications ranging from computer vision to natural-language processing. This primer discusses the main categories of deep learning methods and provides suggestions for how to effectively use deep learning in genomics. The primer is intended for bioinformaticians who are interested in applying deep learning approaches, and for genomicists and general biomedical researchers who seek a high-level understanding of this rapidly evolving field. Computer scientists may also use the primer as an introduction to the exciting applications of deep learning in genomics. However, we do not provide a survey of deep learning in the biomedical field, which has been broadly covered in recent reviews[1–5]. This paper is accompanied by an interactive tutorial that we have created for interested readers to build a convolutional neural network to discover DNA-binding motifs (see URLs).

Deep learning as a class of machine learning methods

Machine learning techniques have been extensively used in genomics research[3,6]. Machine learning tasks fall within two major categories: supervised and unsupervised. In supervised learning, the goal is predicting the label (classification) or response (regression) of each data point by using a provided set of labeled training examples. In unsupervised learning, such as clustering and principal component analysis, the goal is learning inherent patterns within the data themselves.

The ultimate goal in many machine learning tasks is to optimize model performance not on the available data (training performance) but instead on independent datasets (generalization performance). With this goal, data are randomly split into at least three subsets: training, validation and test sets. The training set is used for learning the model parameters (detailed discussion on parameter optimization in ref. [1]), the validation set is used to select the best model, and the test set is kept aside to estimate the generalization performance (Fig. 1). Machine learning must reach an appropriate balance between model flexibility and the amount of training data. An overly simple model will underfit and fail to let the data 'speak'. An overly flexible model will overfit to spurious patterns in the training data and will not generalize.

Large neural networks, a main form of deep learning, are a class of machine learning algorithms that can make predictions and perform dimensionality reduction. The key difference between deep learning and standard machine learning methods used in genomics—e.g., support vector machine and logistic regression—is that deep learning models have a higher capacity and are much more flexible. Typical deep learning models have millions of trainable parameters. However, this flexibility is a double-edged sword. With appropriately curated training data, deep learning can automatically learn features and patterns with less expert handcrafting. It also requires greater care to train on and to interpret the underlying biology. Box 1 summarizes the main messages of this primer on how to effectively use deep learning in genomics.

Setting up deep learning

Deep learning is an umbrella term that refers to the recent advances in neural networks and the corresponding training platforms (e.g., TensorFlow and PyTorch). The starting point of a neural network is an artificial neuron, which takes as input a vector of real values and computes the weighted average of these values followed by a nonlinear transformation, which can be a simple threshold[7]. The weights are the parameters of the model that are learned during training. The power of neural networks stems from individual neurons being highly modular and composable, despite their simplicity[8]. The output of one neuron can be directly fed as input into other neurons. By composing neurons together, a neural network is created.

The input into a neural network is typically a matrix of real values. In genomics, the input might be a DNA sequence, in which the nucleotides A, C, T and G are encoded as [1,0,0,0], [0,1,0,0], [0,0,1,0] and [0,0,0,1]. Neurons that directly read in the data input are called the first, or input, layer. Layer two consists of neurons that read in the outputs of layer one, and so on for deeper layers, which are also referred to as hidden layers. The output of the neural network is the prediction of interest, e.g., whether the input DNA is an enhancer. Box 2 describes key terms and concepts in deep learning.

There are three common families of architectures for connecting neurons into a network: feed-forward, convolutional and recurrent. Feed-forward is the simplest architecture[7]. Every neuron of layer i is connected only to neurons of layer $i + 1$, and all the connection edges can have different weights. Feed-forward architecture is suitable for generic prediction problems when there are no special relations among the input data features.

In a convolutional neural network (CNN), a neuron is scanned across the input matrix, and at each position of the input, the CNN

[1]Department of Biomedical Data Science, Stanford University, Palo Alto, CA, USA. [2]Chan–Zuckerberg Biohub, San Francisco, CA, USA. [3]Department of Electrical Engineering, Stanford University, Palo Alto, CA, USA. [4]Peltarion, Stockholm, Sweden. [5]Department of Learning, Informatics, Management and Ethics, Karolinska Institutet, Stockholm, Sweden. [6]Scripps Research Translational Institute, La Jolla, CA, USA. [7]Department of Integrative Structural and Computational Biology, The Scripps Research Institute, La Jolla, CA, USA. *e-mail: jamesz@stanford.edu; atelenti@scripps.edu

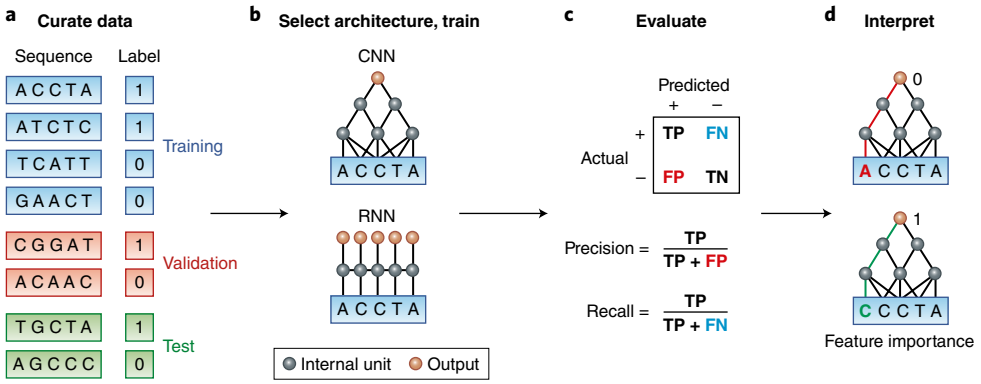


Fig. 1 | Deep learning workflow in genomics. a, A dataset should be randomly split into training, validation and test sets. The positive and negative examples should be balanced for potential confounders (for example, sequence content and location) so that the predictor learns salient features rather than confounders. **b**, The appropriate architecture is selected and trained on the basis of domain knowledge. For example, CNNs capture translation invariance, and RNNs capture more flexible spatial interactions. **c**, True positive (TP), false positive (FP), false negative (FN) and true negative (TN) rates are evaluated. When there are more negative than positive examples, precision and recall are often considered. **d**, The learned model is interpreted by computing how changing each nucleotide in the input affects the prediction. The interactive tutorial illustrates the four steps of this workflow (see URLs).

computes the local weighted sum and produces an output value[9]. This procedure is very similar to taking the position weight matrix of a motif and scanning it across the DNA sequence. CNNs are useful in settings in which some spatially invariant patterns in the input are expected.

Recurrent neural networks (RNN) are designed for sequential or time-series data[7]. At each point in the sequence, a neural network, which could be feed-forward or convolutional, is applied to generate an internal signal, which is also fed to the next step of the RNN. Hidden layers of the RNN can be viewed as memory states that retain information from the sequence previously observed and are updated at each time step.

In addition, there are neural network architectures used for unsupervised learning. The most common are autoencoders that perform nonlinear dimensionality reduction[10], in contrast to principal component analysis, which is linear. In an autoencoder, the output is set to be the input, and the network is encouraged to find a low-dimensional space that compresses the original information and reconstructs the inputs.

Training a neural network starts with a labeled dataset of (X_i, Y_i), where each X_i is the ith input, and Y_i is its output label. Each training point X_i is fed into the network, and the network's output is evaluated against the true label Y_i to produce a loss $L(Y'_i, Y_i)$. Loss is the sum of the errors made for each example. The lower the loss, the better the model. Commonly used loss functions include squared error and cross-entropy. Squared error measures the difference between predicted and actual output values, which is especially relevant when the output is a continuous value. Cross-entropy measures the difference between two probability distributions over the same set of underlying events or classes, as is appropriate when the output is categorical[7].

To train the network, the derivative of $-L(Y'_i, Y_i)$ is computed with respect to the parameters of the network, which are the collection of neuron weights. By updating the weights in a small step in the direction of the derivative, the network's prediction loss can be decreased, and its accuracy can be increased. The derivative can be efficiently computed for each training point via the chain rule from calculus; this process is commonly called back-propagation. In parallel, the network's accuracy is also evaluated on the validation data, which are not used to update the weights. A good practice to avoid overfitting is to stop the training updates when the validation accuracy begins to decrease. More sophisticated techniques such as L_2 regularization on the weights and dropout are also effective in controlling overfitting. L_2 regularization encourages simpler, and thus more generalizable, models by penalizing models with large weights[7]. Dropout is a training process that randomly ignores nodes to mitigate overfitting[11].

How to use deep learning effectively

The most important step in building an effective deep learning model is to first curate an appropriate training dataset and to select a suitable evaluation metric. The training set should be constructed to ensure that confounding biases that may artificially inflate performance are not introduced. For example, known pathogenic genetic variants may cluster in certain regions of the genome, i.e., exons or promoters, whereas known neutral variants may be more broadly distributed throughout the genome. A neural network naively applied to these unbalanced data may appear to perform well, but in reality it would probably learn to identify regions of the genome enriched in pathogenic variants without actually being able to distinguish neutral from pathogenic variants within these important genomic regions. Thus, it is important to design training datasets that are appropriately balanced for confounders that would detrimentally affect performance when applied to real-world-use cases[12].

Furthermore, genomics data are often highly imbalanced[13]. For example, there are many more variants that are not disease causing than are disease causing, or only a small fraction of the population may develop a particular disease to be predicted. Therefore it is often more meaningful to assess precision and recall, measures of classifier performance that account for the imbalance of classes in a dataset rather than simple accuracy[14].

Successful application of deep learning also requires domain knowledge, as with all other machine learning methods. For example, in support-vector-machine classification and logistic regression, domain knowledge is used to construct features from data, and in Bayesian models, expertise is incorporated into the prior distributions. In deep learning, domain knowledge is built into the design of the network architecture. The performance of the network crucially depends on understanding the assumptions and limitations behind different architectures.

As an illustration, suppose we want to build a model to predict whether a DNA sequence is an active enhancer[8]. Biological knowledge indicates that regulatory elements may be effective even after small spatial translations, thus suggesting that CNN might model them effectively. Moreover, the regulatory motifs are known to

Box 1 | Deep learning in genomics: key elements and guidance

- Large training datasets (typically thousands of examples), curated to remove confounders, are required.
- The main architectures—feed-forward, convolutional and recurrent—correspond to different assumptions about data.
- Most genomics data do not require very deep networks.
- Researchers must be wary of high accuracy due to data imbalance or bias that makes classification too easy.
- A good practice is to compare against simpler machine learning models on the same dataset.
- Deep learning can achieve high accuracy, but the interpretation of results is more challenging than for standard statistical models.

Box 2 | Vocabulary and concepts

Artificial neuron
A simple mathematical function that takes a vector as input and outputs a transformed weighted sum of the vector. The weights are the neuron's parameters.
Deep learning
Neural networks with multiple layers of artificial neurons. The output of one layer is fed as input into the next layer to achieve greater flexibility.
Cross-validation
A common machine learning strategy wherein the dataset is split multiple times into training and validation sets. The average validation performance across the multiple splits is used to select the final model.
Back-propagation
A common method to train neural networks by updating its parameters (i.e., weights) by using the derivative of the network's performance with respect to the parameters.
Feed-forward neural network
The most flexible class of neural networks, wherein each neuron can have arbitrary weights.
Convolutional neural network
A class of neural networks in which groups of artificial neurons are scanned across the input to identify translation-invariant patterns.
Recurrent neural network
A class of neural networks with cycles that can process inputs of varying lengths.
Autoencoder
A class of neural networks that performs nonlinear dimensionality reduction.
Feature importance
A saliency score for each input feature (for example, each nucleotide) that measures the extent to which changes in that feature affect the prediction.

have a tendency to be relatively short (<20 nt) thus suggesting that convolution filters should also be small (<20 nt). Finally, if enhancers are being modeled, most of the activity is known to have a tendency to be clustered in regions from several hundred base pairs to two kilobases. Consequently, a reasonable design would be to limit inputs to the network to <2 kb. Any choice of the network architecture places an implicit prior over the model, and, as with any other machine learning, mismatch between the model's prior and the underlying biology can lead to poor performance.

In computer vision, researchers have observed that performance can improve with very deep networks (more than 100 layers). In most genomics applications, fewer than five layers are sufficient[13,15]. Even relatively shallow networks can still have millions of parameters, and the most important factor that determines the success of a model is the availability of a large corpus of labeled training data. Most successful biological applications of deep learning have at least several thousand labeled examples[1]. It is always good practice to train simpler machine learning models—linear regression, least absolute shrinkage and selection operator (LASSO), gradient boosting or random forest—in parallel on the same dataset to compare against the deep learning models. Simpler models have fewer parameters and can work better when fewer training datasets are available[16].

Interpreting deep learning models

In many genomic applications, researchers are more interested in the biological mechanisms revealed by the predictive model rather than the prediction accuracy itself[13,17]. For example, the main motivation for building accurate deep learning models to predict chromatin patterns is a hope to learn new gene-regulation grammar by interpreting the trained model. Although deep learning can achieve state-of-the-art accuracy, it is more challenging to interpret than the more standard statistical models.

The simplest method to interpret a neural network is analogous to in silico mutagenesis. Given a particular data point X, each feature of X (for example, mutating each nucleotide) can be systematically varied while the rest of the features are fixed, and how the network's output changes can be tracked. This approach is easy to implement but can be computationally expensive—the network recomputes the output for every mutation of X. A computationally tractable approximation to mutagenesis is to take the derivative of the network output with respect to each feature of X. This derivative can be computed in one back-propagation pass, and it conveys the sensitivity of the output to small perturbations in input features. Features with large positive or negative derivatives may be more influential to the outcome.

In strict terms, the derivative is a valid measure of influence for only infinitesimally small perturbations to the input, whereas in practice, researchers are interested in larger changes (for example, mutation of A to C). Several variations of the derivative-based interpretation methods have been developed to partially address this limitation—for example, integrated gradient[18] and DeepLift[19]—and this goal is still currently an active area of research. Other interpretation methods, such as LIME[20] select a small number of features to explain why a prediction is made.

For a CNN, it is also possible to visualize each convolution filter as a heat map or position weight matrix–style logo image. These visualizations are useful to obtain a sense of what local features the network might be learning. A caveat is that multiple convolution filters might be learning partially redundant features, and how the local features interact is less clear, because such interaction depends on the higher layers of the network.

The interpretation methods discussed here should not be confused with causal models that attempt to pinpoint cause–effect relationships. Interpreting a prediction model can identify salient features and generate hypotheses, but inferring actual causality requires experimental perturbation.

Applications in genomics

A growing number of publications are presenting deep learning approaches and tools to study the genome (Fig. 2). Functional genomics is a leading application domain of deep learning[21]. Examples include predicting the sequence specificity of DNA- and RNA-binding proteins and of enhancer and cis-regulatory regions, methylation status, gene expression and control of splicing. These tools are based on data generated by DNase I sequencing, assay for

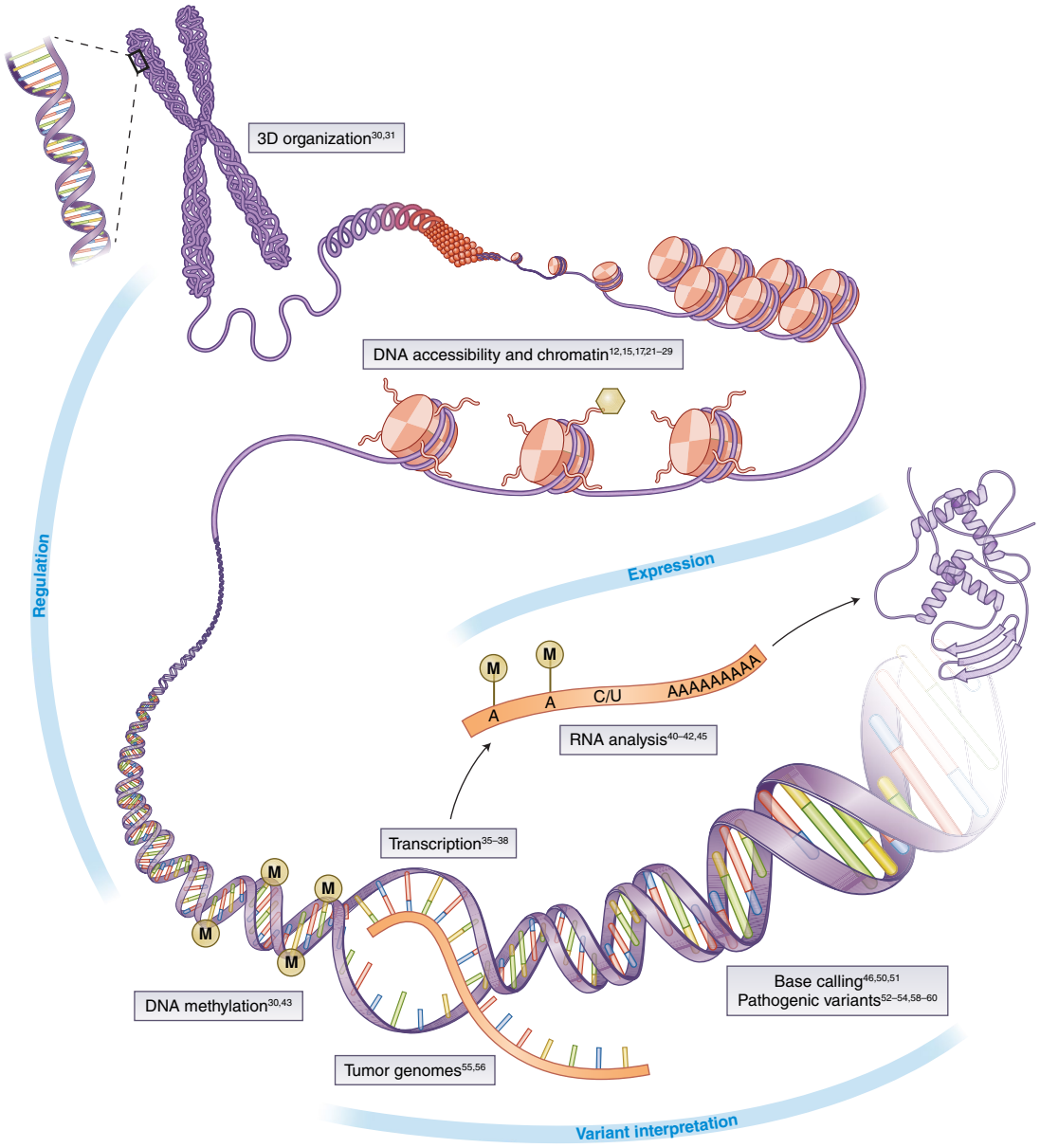


Fig. 2 | Applications of deep learning in genomics. The boxes highlight several application domains and references discussed in the text. Image adapted with permission from ref. [65], Springer Nature.

transposase-accessible chromatin using sequencing (ATAC–seq), chromatin immunoprecipitation (ChIP) sequencing, ChIP-on-chip, RNA immunoprecipitation sequencing, transcription-factor datasets and chromatin state[15,17,22–24]. Similarly, the identification of transcription start sites as well as cis-regulatory/enhancer elements[25–29] can be executed with the addition of features from the Encyclopedia of DNA Elements (ENCODE) project, as well as transcription-start-site sequencing and RNA-seq signals. The methylation state of DNA, which also influences the expression of genes, has been inferred from three-dimensional genome topology (on the basis of Hi-C) and DNA sequence patterns[30]. Nucleotide sequence and DNase I assay signals predict Hi-C contacts at 1-kb resolution[31].

Deep learning has been especially successful when applied to regulatory genomics, by using architectures directly adapted from modern computer vision and natural-language-processing applications[32]. Most approaches use CNN or RNN, which are well suited for the tasks of modeling regulatory elements, though genomics-specific modifications to the deep learning architecture can be useful. For example, Shrikumar et al.[33] have addressed the phenomenon in which, in double-stranded DNA, the same pattern may appear identically on one strand and its reverse complement, owing to complementary base-pairing. Conventional deep learning models that do not explicitly model this property can produce different predictions on forward- and reverse-complement versions of the same DNA sequence.

Different tools are able to extract transcriptome patterns from large sets of gene expression data with the goal of estimating how much RNA is produced from a DNA template in a particular cell or condition[34–37]. Deep learning can build predictive models of gene expression from genotype data[38] and can be used for studying the splicing-code model[39] as well as for the identification of long noncoding RNAs[40–42]. Finally, deep learning has been used for the interpretation of regulatory control in single cells; for example, the detection of DNA methylation in single cells[30,43], and for the identification of subgroups of cells by improving the representation of single-cell RNA-seq data[44,45].

Box 3 | Deep learning resources

All the resources, except for those listed in the 'specific for genomics' block, are relevant for deep learning in general. Because the field is developing rapidly, this information is likely to be considerably different in the future.

Resource type	Name	URL	Comment
Cloud platform	Amazon EC2	https://aws.amazon.com/ec2/	Most popular cloud platform
	Microsoft Azure	https://azure.microsoft.com/	Second-largest cloud platform
Plug-and-play cloud GPU services	FloydHub	https://www.floydhub.com/	All startups in the GPU service space; pay-by-the-hour model on top of basic monthly subscriptions
	PaperSpace	https://www.paperspace.com/	
	Valohai	https://valohai.com/	
	Google CloudML	https://cloud.google.com/ml-engine/	Can run your own models on Google's hardware, including tensor processor units
	Google Colaboratory	https://colab.research.google.com/	Notebook environment with free GPUs (during 12 h)
Design services for deep learning models	Fabrik	https://github.com/Cloud-CV/Fabrik/	Model export to Keras code; no training
	IBM Data Cloud	https://datascience.ibm.com/	Model export to Keras, PyTorch, TensorFlow or Caffe
	DeepCognition	http://deepcognition.ai/	Training and evaluation included
Prebuilt images with CUDA support	Docker Hub	https://hub.docker.com/r/nvidia/cuda/	Docker images from NVIDIA with CUDA/cuDNN GPU support
	Amazon Deep Learning AMIs	https://aws.amazon.com/machine-learning/amis/	Amazon Machine Images (AMIs) with GPU support
Software libraries (general)	Keras	https://keras.io/	More high-level than TensorFlow (below) but can be integrated with it in many ways
	TensorFlow	https://www.tensorflow.org/	Developed by Google; most popular deep learning framework
	PyTorch	http://pytorch.org/	Developed by Facebook
Software libraries (specific for genomics)	DragoNN	https://kundajelab.github.io/dragonn/	Tutorials included
	Kipoi	http://kipoi.org/	Model zoo for deep learning in genomics
Educational resources	fast.ai	http://www.fast.ai/	E.g., Deep Learning for Coders 1 and 2
	Coursera	https://www.coursera.org/specializations/deep-learning/	Deep-learning-specialization course package
	Textbook	http://neuralnetworksanddeeplearning.com/	Free online textbook with example code
	Fast.ai tips on configuring a deep learning environment	https://github.com/reshamas/fastai_deeplearn_part1/blob/master/README.md#platforms-for-using-fastai-gpu-required/	Instructions for configuring deep learning frameworks for a variety of platforms; from the fast.ai course but general; the details of these procedures change quickly
	Setting up TensorFlow with GPU on Google Cloud Engine	https://medium.com/google-cloud/jupyter-tensorflow-nvidia-gpu-docker-google-compute-engine-4a146f085f17/	Recipe for Docker-based setup of Google Cloud instance with TensorFlow, GPU support and Jupiter Notebooks

Predicting phenotypes from genetic data is also a major area of interest of deep learning. A first step in performing these types of predictions is to specify what genetic variants are present in an individual genome. This problem has been addressed by DeepVariant, which applies a CNN to make variant calls from short-read sequencing. The method treats DNA alignments as an image with a performance that appears to exceed that of standard variant callers[46]. Other methods for variant calling using more traditional DNA representations have also been developed[47–49]. Long-read-sequencing technologies also use deep learning for base calling[50,51].

Prioritizing variants on the basis of the likelihood that they are pathogenic is important. Several methods that predict the pathogenicity of coding variants have been proposed[52–54]. These approaches are essentially aggregators that combine prior non–deep learning predictors as well as features known to be useful for the prediction of variant pathogenicity. A more challenging problem, the prediction of functional consequences of noncoding variants, has seen some success via DeepSEA[13], a CNN trained to predict functional genomics features that can also be adapted to predict variant effects. In cancer genomics, deep learning can extract the high-level features between combinatorial somatic mutations and cancer types[55] and learn prognostic information from multicancer datasets[56]. Preliminary phenotype prediction in agricultural applications appears promising[57]. However, a demonstration of the prediction of common complex human disease phenotypes is only now emerging: Zhou et al. have recently reported the extension of DeepSEA to the study of regulatory variants in autism spectrum disorder[58]. The same team has published ExPecto, the ab initio prediction of gene

expression levels and variant effects from sequences from more than 200 tissues and cell types[59]. Also recently, Sundaram et al. have trained a DNN by using hundreds of common variants from population sequencing of nonhuman primate species to identify pathogenic variants in rare human diseases[60].

Resources

Implementation of deep learning approaches requires familiarity with new tools and resources. These include access to computing power and general and genomics-specific software libraries (Box 3).

Cloud platforms. Recent advances in deep learning have, to a considerable degree, been driven by developments in graphical processing unit (GPU) technology. GPUs can significantly increase training speed because the way in which neural networks are trained lends itself well to their architecture, thus allowing for fast vector and matrix multiplications, owing to the large number of processing units and high memory bandwidth in GPUs. Some academic high-performance computing clusters offer access to GPUs, but because progress in GPU technology is rapid, those processors are at risk of becoming outdated. The major cloud-computing platforms, such as Amazon EC2, Google Cloud Engine, Microsoft Azure and IBM Cloud, can be more flexible in offering on-demand GPU access. However, even these cloud platforms require some degree of configuration from the user, such as installing and compiling an appropriate version of CUDA (a popular parallel computing platform and programming toolkit) for general GPU programming, upon which many deep learning frameworks depend for GPU acceleration.

For users wishing to avoid semimanual setup procedures, there are specialized platforms offering 'plug-and-play' GPU access, for instance FloydHub, Valohai, Paperspace and Google CloudML. Typically, these platforms provide configuration-free access to GPUs on Amazon or Google cloud infrastructures for a modest markup compared with running 'raw' cloud instances directly. Perhaps the simplest alternative at the time of writing of this paper is Google Colaboratory, a Python notebook environment that provides free use of a K80 GPU during 12 consecutive hours. The tutorial accompanying this paper (see URLs) is built on this platform. In some cases, using lightweight virtual machines such as Docker or Singularity containers is convenient for packaging software and dependencies so that they can be directly run on a cloud instance with minimal setup. All these solutions, however, still require users to write the code for the models.

Some emerging platforms are offering the possibility to design deep learning models without coding. For example, IBM Data Cloud has recently introduced a neural network designer in which users can build a model from building blocks and export the finished model to runnable code. The open-source Fabrik framework offers a similar capability. DeepCognition offers a platform in which users can design, train and evaluate models in the same framework.

Software libraries. Many software libraries are commonly used for deep learning. Whereas earlier models were often written in frameworks developed at universities, such as Theano, Torch and Caffe, recent years have seen a strong showing of open-source Python libraries developed in corporate laboratories, for example TensorFlow (Google) and PyTorch (Facebook). Some frameworks were initially developed independently but subsequently received explicit backing from a company, for example, Keras (Google) and MXNet (Amazon). Keras has gained its popularity from being designed to be at a higher level of abstraction than most other frameworks; consequently, it allows for model construction in larger conceptual blocks such as network layers without requiring users to keep track of all the details.

Because of the relatively recent entry of deep learning into genomics, there are few genomics-specific software libraries (Box 3). One example is DragoNN, a toolkit to teach and learn about deep learning for genomics, which includes tutorials in Python notebook format, a command-line interface, tools for running models on cloud CPU or GPU, and a model-interpretation module. Kipoi is a repository of ready-to-use trained models in genomics. It currently contains 2,031 different models ('model zoo'), covering canonical predictive tasks in transcriptional and post-transcriptional gene regulation[61]. Kipoi uses Conda virtual environments to install the correct dependencies for each model.

Conclusions and perspectives

Although deep learning has demonstrated impressive potential in genomics, a number of outstanding issues remain[62]. First is the challenge of how to design deep learning systems that best augment and complement human experience in making medical decisions (for example, genome interpretation). Second is the challenge of how to avoid biases in training sets and how to interpret predictions. Interpretation and robustness are two important directions of method development[63]. Finally, there is a need for iterative experimentation, in which deep learning predictions can be validated by functional laboratory tests or by formal clinical assessment.

The most successful applications of deep learning in genomics to date has been in supervised learning, i.e., making predictions[8]. It is important to not confuse high prediction accuracy with the ultimate objective of extracting biomedical insights from data and making robust assessment in diverse settings that might be different from the training data. Beyond making predictions, deep learning can potentially become a powerful tool for synthetic biology by learning to automatically generate new DNA sequences and new proteins with desirable properties[64]. Such generative models are an exciting new frontier of research.

URLs. Interactive tutorial to build a convolutional neural network to discover DNA-binding motifs, https://colab.research.google.com/drive/17E4h5aAOioh5DiTo7MZg4hpL6Z_0FyWr.

Received: 14 May 2018; Accepted: 26 September 2018;
Published online: 26 November 2018

References

1. Angermueller, C., Pärnamaa, T., Parts, L. & Stegle, O. Deep learning for computational biology. *Mol. Syst. Biol.* **12**, 878 (2016).
2. Ching, T. et al. Opportunities and obstacles for deep learning in biology and medicine. *J. R. Soc. Interface* **15**, 20170387 (2018).
3. Telenti, A., Lippert, C., Chang, P. C. & DePristo, M. Deep learning of genomic variation and regulatory network data. *Hum. Mol. Genet.* **27**, R63–R71 (2018).
4. Yue, T. & Wang, H. Deep learning for genomics: a concise overview. Preprint at https://arxiv.org/abs/1802.00810 (2018).
5. Camacho, D. M., Collins, K. M., Powers, R. K., Costello, J. C. & Collins, J. J. Next-generation machine learning for biological networks. *Cell* **173**, 1581–1592 (2018).
6. Libbrecht, M. W. & Noble, W. S. Machine learning applications in genetics and genomics. *Nat. Rev. Genet.* **16**, 321–332 (2015).
7. Goodfellow, I., Bengio, Y., Courville, A. & Bengio, Y. *Deep Learning* Vol. 1 (MIT Press, Cambridge, 2016).
8. LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* **521**, 436–444 (2015).
9. Krizhevsky, A., Sutskever, I. & Hinton, G. E. Imagenet classification with deep convolutional neural networks. *Adv. Neural Inf. Process. Syst.* **1**, 1097–1105 (2012).
10. Hinton, G. E. & Salakhutdinov, R. R. Reducing the dimensionality of data with neural networks. *Science* **313**, 504–507 (2006).
11. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* **15**, 1929–1958 (2014).
12. Khodabandelou, G., Mozziconacci, J. & Routhier, E. Genome functional annotation using deep convolutional neural network. Preprint at https://www.biorxiv.org/content/early/2018/05/25/330308 (2018).
13. Zhou, J. & Troyanskaya, O. G. Predicting effects of noncoding variants with deep learning-based sequence model. *Nat. Methods* **12**, 931–934 (2015).
14. Powers, D. M. W. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *J. Mach. Learn. Technol.* **2**, 37–63 (2011).